Butterfly-accelerated multiple scattering in two dimensions

Samuel F. Potter
Joint work with Leslie Greengard and Mike O'Neil

Courant Institute of Mathematical Sciences
New York University

What's the point of this work?

Ultimate goal: high-frequency fast direct solvers for Helmholtz.

To that end: evaluate the butterfly factorization[1] for this purpose. Compare how efficient it is relative to the high-frequency fast multipole method (HF-FMM).

There is no readily available implementation of the butterfly algorithm, and the original paper is unclear on the details of how it should be programmed.

Today's goal: Good implementation + applied to realistic problem + comparison with HF-FMM (Flatiron's FMM2D).

---

[1] Michielssen & Boag, "A Multilevel Matrix Decomposition Algorithm for Analyzing Scattering from Large Structures."

Model problem: sound-soft scattering in 2D

Solve Helmholtz equation with Dirichlet (sound-soft) BCs on $\Omega \subseteq \mathbb{R}^2$:

$$\begin{cases} \Delta u(x) + k^2 u(x) = 0, & x \in \Omega \\ u(x) = g(x), & x \in \partial\Omega \end{cases} \tag{1}$$

Seek solution of the form:

$$u = u_{\text{in}} + u_{\text{scat}}, \tag{2}$$

where u_{in} is an incident field which satisfies the Helmholtz equation.

Solution by CFIE representation

Green's function for 2D Helmholtz is:

$$G(x,y) = \frac{i}{4} H_0^{(1)}(k\|x-y\|). \tag{3}$$

Use the combined-field integral equation (CFIE) representation. Let solution be of the form:

$$u_{\text{scat}}(x) = \int_{\partial\Omega} \left[\alpha G(x,y) + \beta \frac{\partial G(x,y)}{\partial \nu(y)} \right] \sigma(y) dy, \tag{4}$$

where $\alpha = -ik$ and $\beta = -1$.

CFIE makes sound-soft scattering problem uniquely solvable and well-conditioned for all k.

Solution using CFIE representation

Satisfies boundary integral equation of the form:

$$\frac{1}{2}\sigma(x) + \int_{\partial\Omega} \left[\alpha G(x,y) + \beta \frac{\partial G(x,y)}{\partial \nu(y)}\right] \sigma(y) dy = -u_{\text{in}}(x). \quad (5)$$

Nÿstrom method: discretize using Kapur-Rokhlin quadrature. Leads to a dense linear system of form:

$$\left(\frac{1}{2} I + K\right) \sigma = -u_{\text{in}}. \quad (6)$$

Error in σ is $O(h^p)$ where spacing between nodes is $O(h)$.

Iterative solution

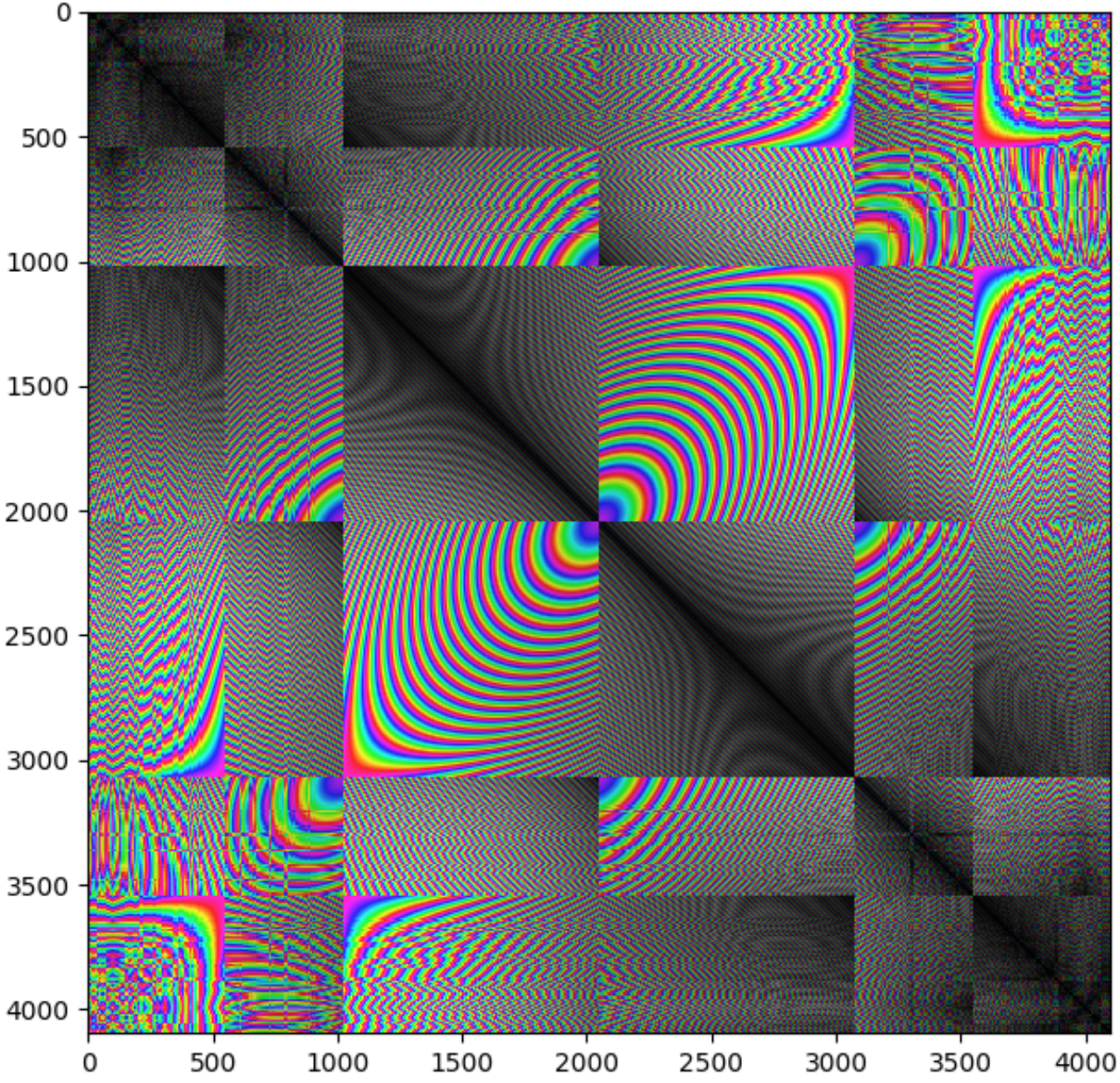
Linear system is nonsymmetric and indefinite, use GMRES to solve.

If system has $O(n)$ nodes, iterative solution using dense matrix vector product is $O(kn^2)$ since K is dense. For high-frequency ($k \sim n$) problem, solution takes $O(n^3)$ time, no better than GE.

Need a fast algorithm for multiplication by K and a preconditioner:

- **Fast algorithm**: our focus: butterfly vs. FMM.
- **Preconditioner**: problem-specific (block Jacobi).

Can we “butterfly” the Helmholtz kernel matrix?



Key tool: large k *a priori* rank estimate

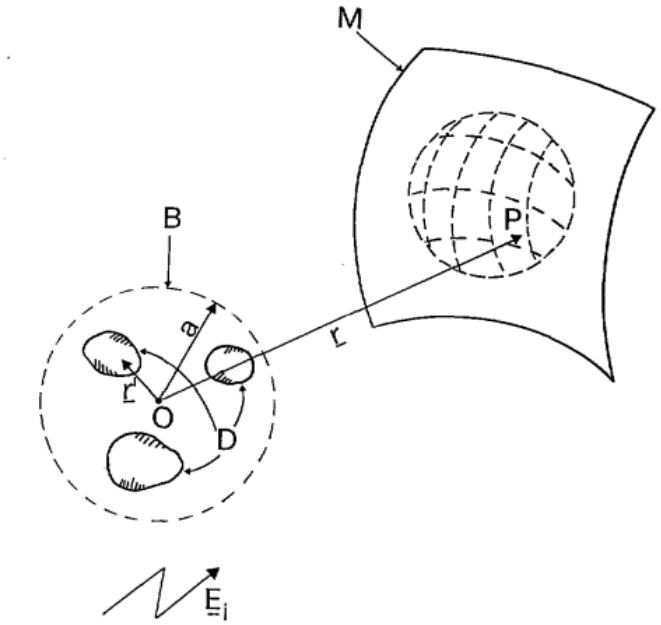


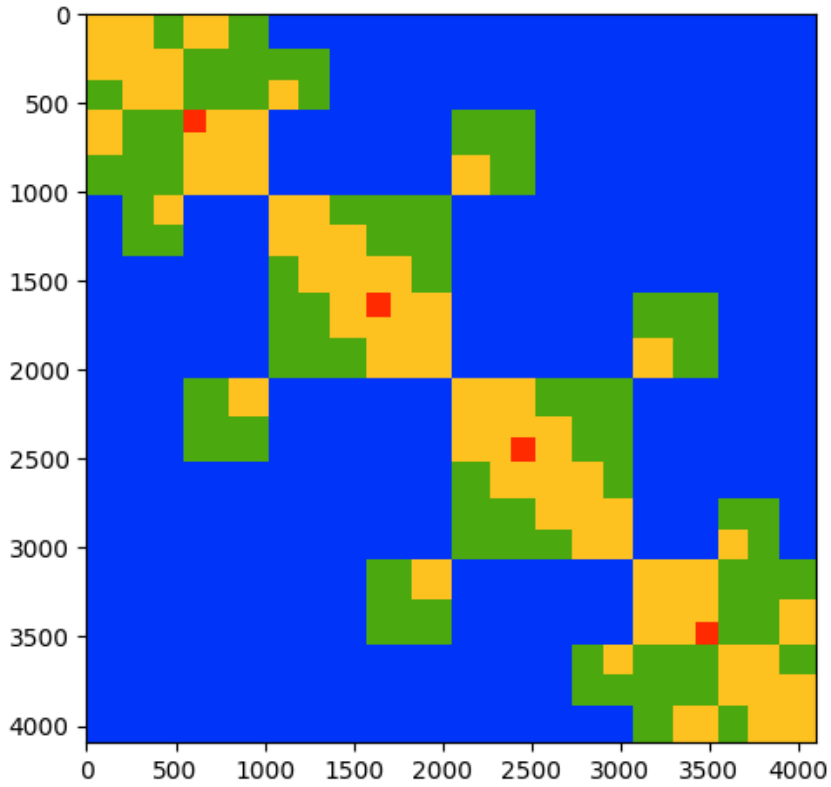
Fig. 1. Geometry of the scattering problem.

Theorem. Let $k > 0$. For 2D scatterer with radius r_1 and domain with radius r_2, if R is the separation distance, then ϵ-rank of Helmholtz kernel matrix is $kr_1r_2/(R - r_1 - r_2) + O(\log(1/\epsilon))$ as $k \to \infty$.[2]

[2] Bucci and Franceschetti, "On the Spatial Bandwidth of Scattered Fields", IEEE Transactions on Antennas and Propagation, 1987.

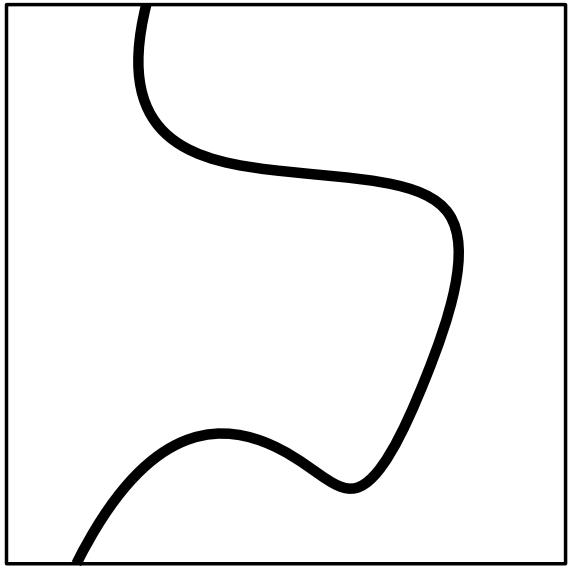
Multilevel butterfly factorization

Can’t butterfly K directly, can only butterfly blocks corresponding to well-separated sets of points. Use a quadtree to generate these sets.

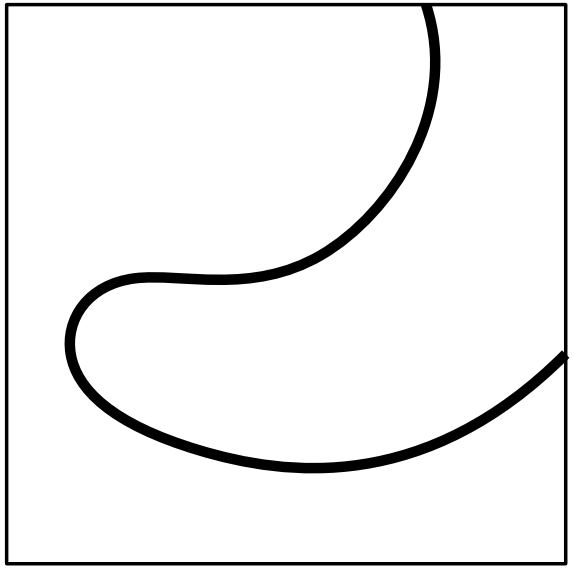


How to butterfly a block of K

Start with a pair of well-separated boxes at the same level of the quadtree containing a subset of the boundary:



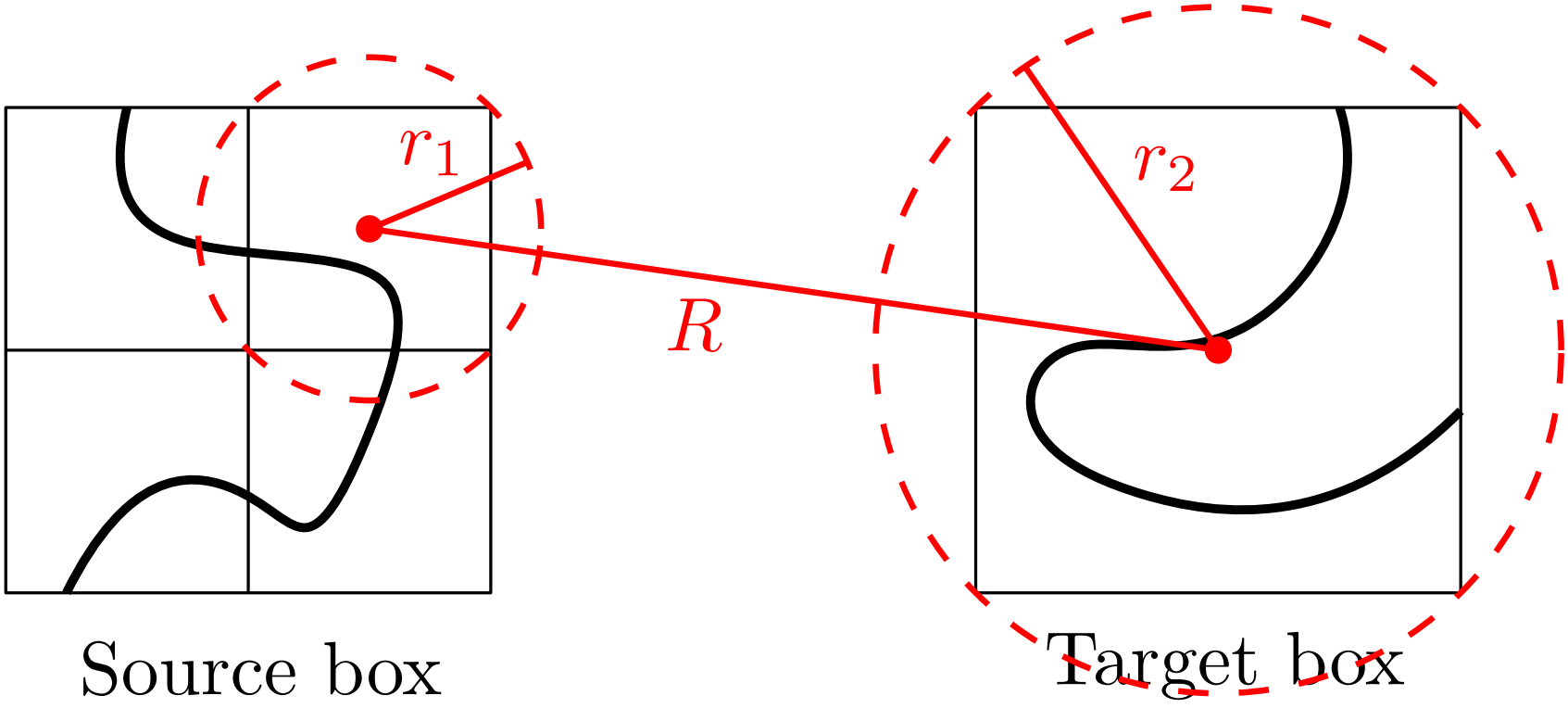
Source box



Target box

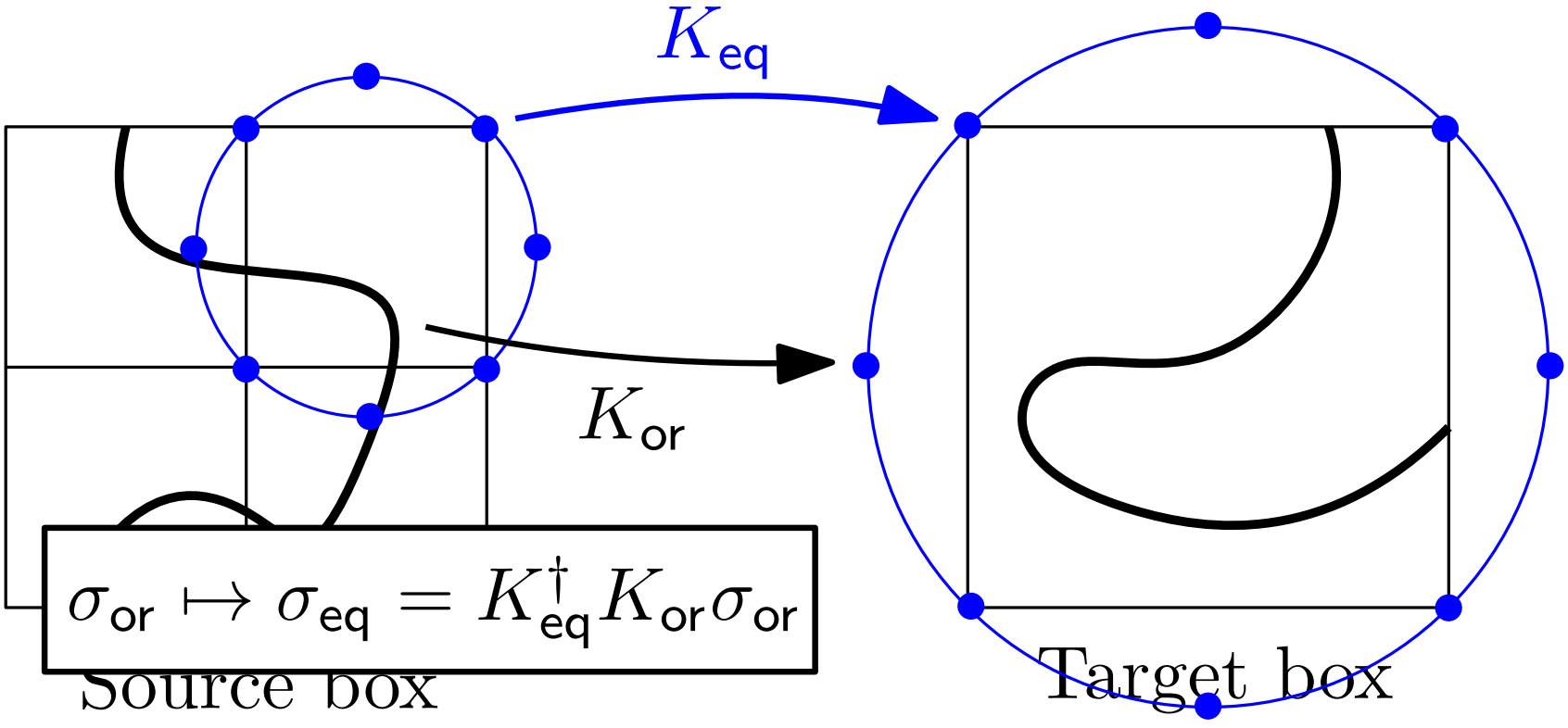
How to butterfly a block of K

Push down the subtree beneath the source box until the rank estimate is $O(1)$ for all source leaf boxes paired with the target box:



How to butterfly a block of K

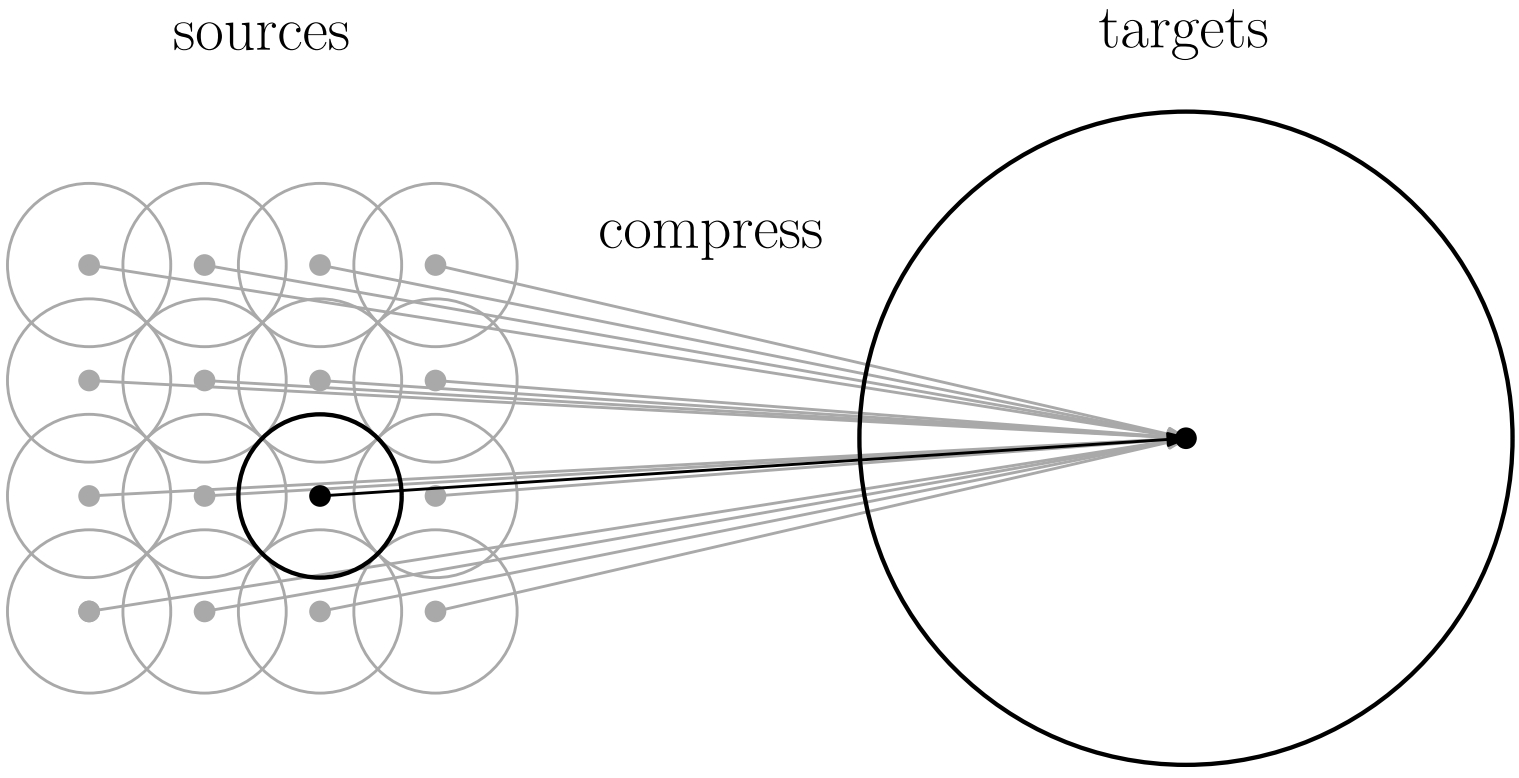
The rank estimate says we can use $O(\log \epsilon^{-1})$ proxy points to reexpand the layer density in the child box with the same number of collocation points (do this by solving a least squares problem using the SVD).



.

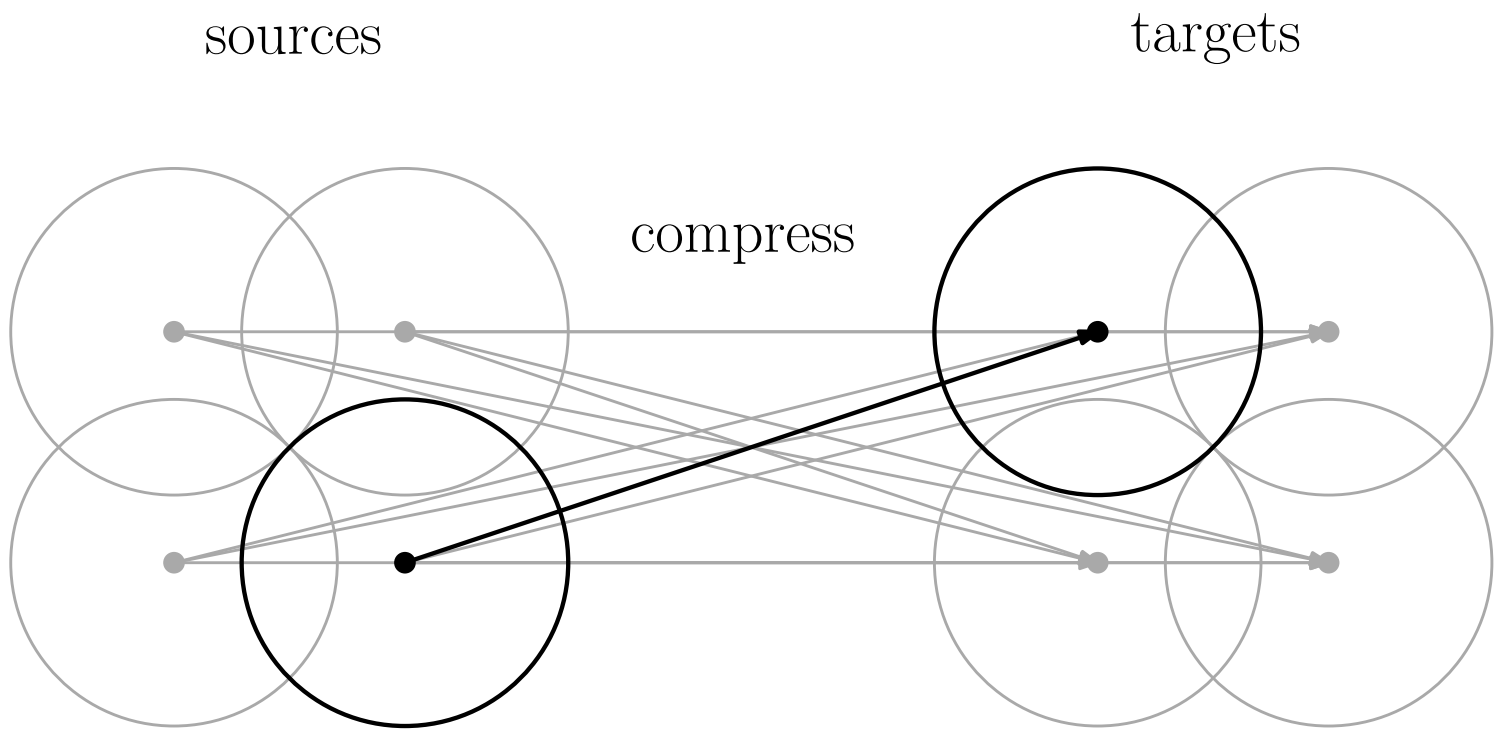
How to butterfly a block of K

To “butterfly” this block of the kernel matrix, move up the source tree and down the target tree simultaneously, reexpanding using proxy points, merging, and splitting the evaluation domains:



How to butterfly a block of K

To “butterfly” this block of the kernel matrix, move up the source tree and down the target tree simultaneously, reexpanding using proxy points, merging, and splitting the evaluation domains:

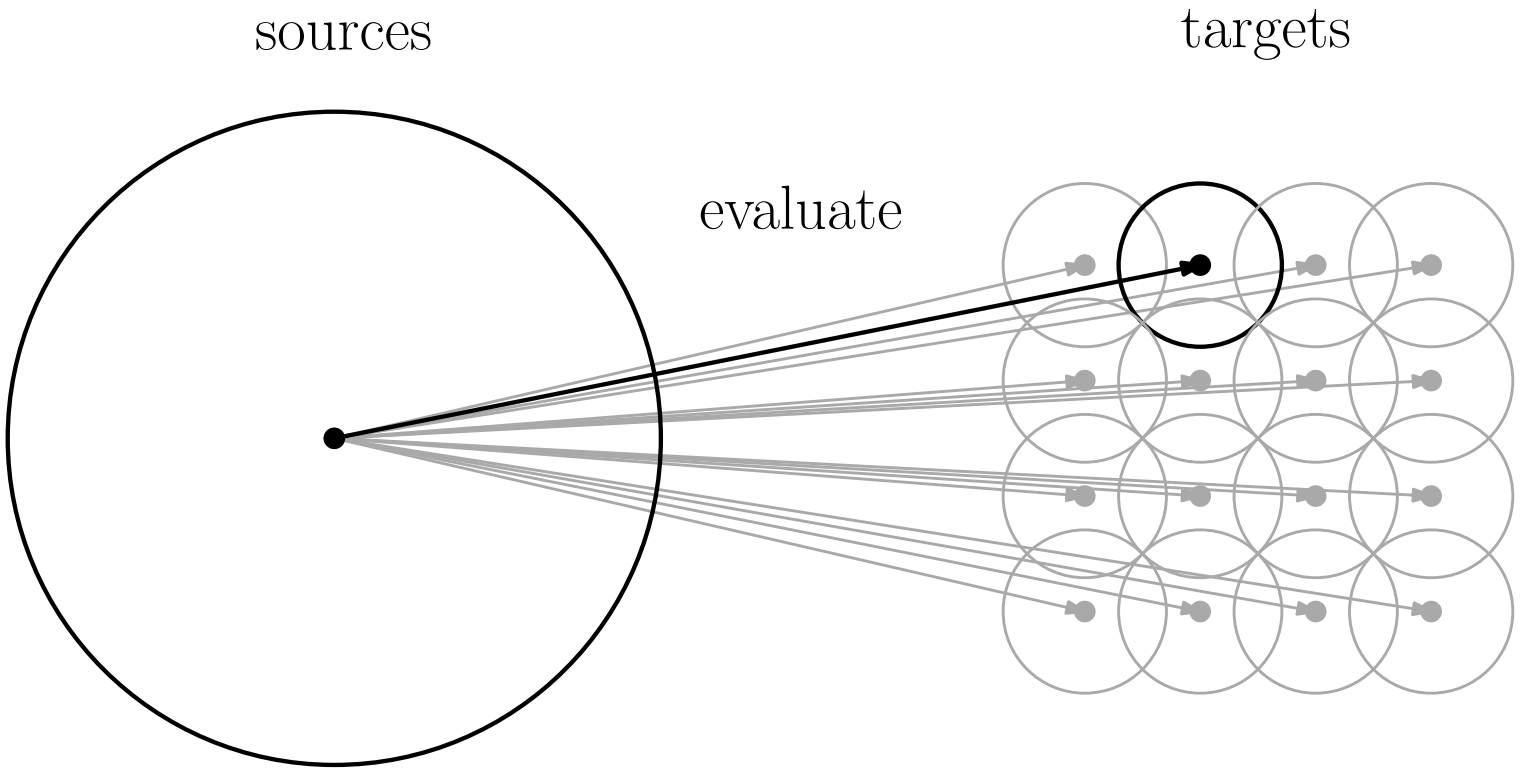


equivalent surfaces are nested

observation domains are nested

How to butterfly a block of K

To “butterfly” this block of the kernel matrix, move up the source tree and down the target tree simultaneously, reexpanding using proxy points, merging, and splitting the evaluation domains:



equivalent surfaces are nested

observation domains are nested

How to butterfly a block of K

Data flow is the same as the FFT.

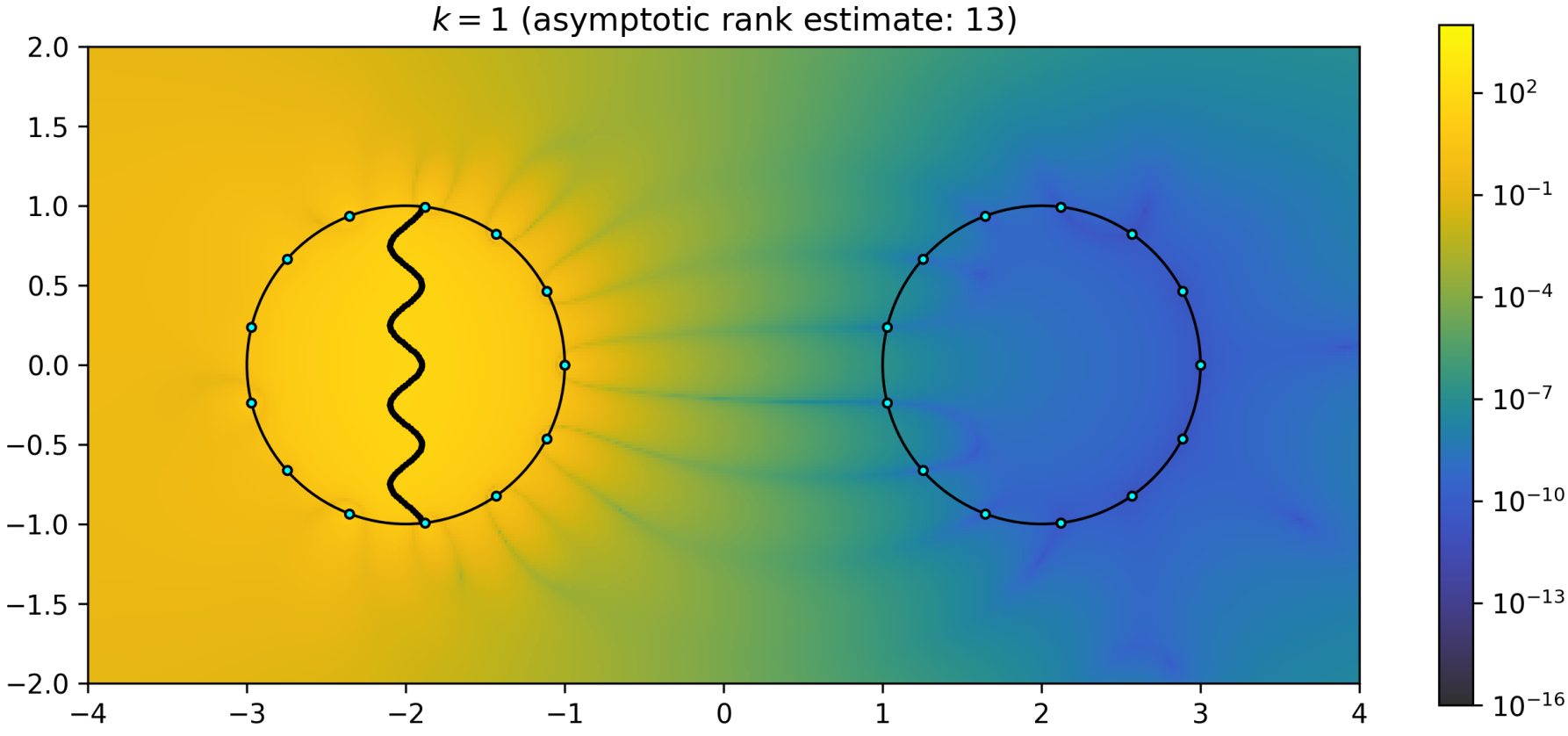
At the end, have factorized subblock into matrix product with $O(\log k)$ factors, each with $O(n)$ nonzero entries.

Key point! Assembly is only $O(n \log k)$ using proxy points. If $k = O(1)$, the butterfly factorization reduces to a simple low rank factorization.

Essentially the same algorithm can be used for S, S′, D, and D′ or combinations thereof.

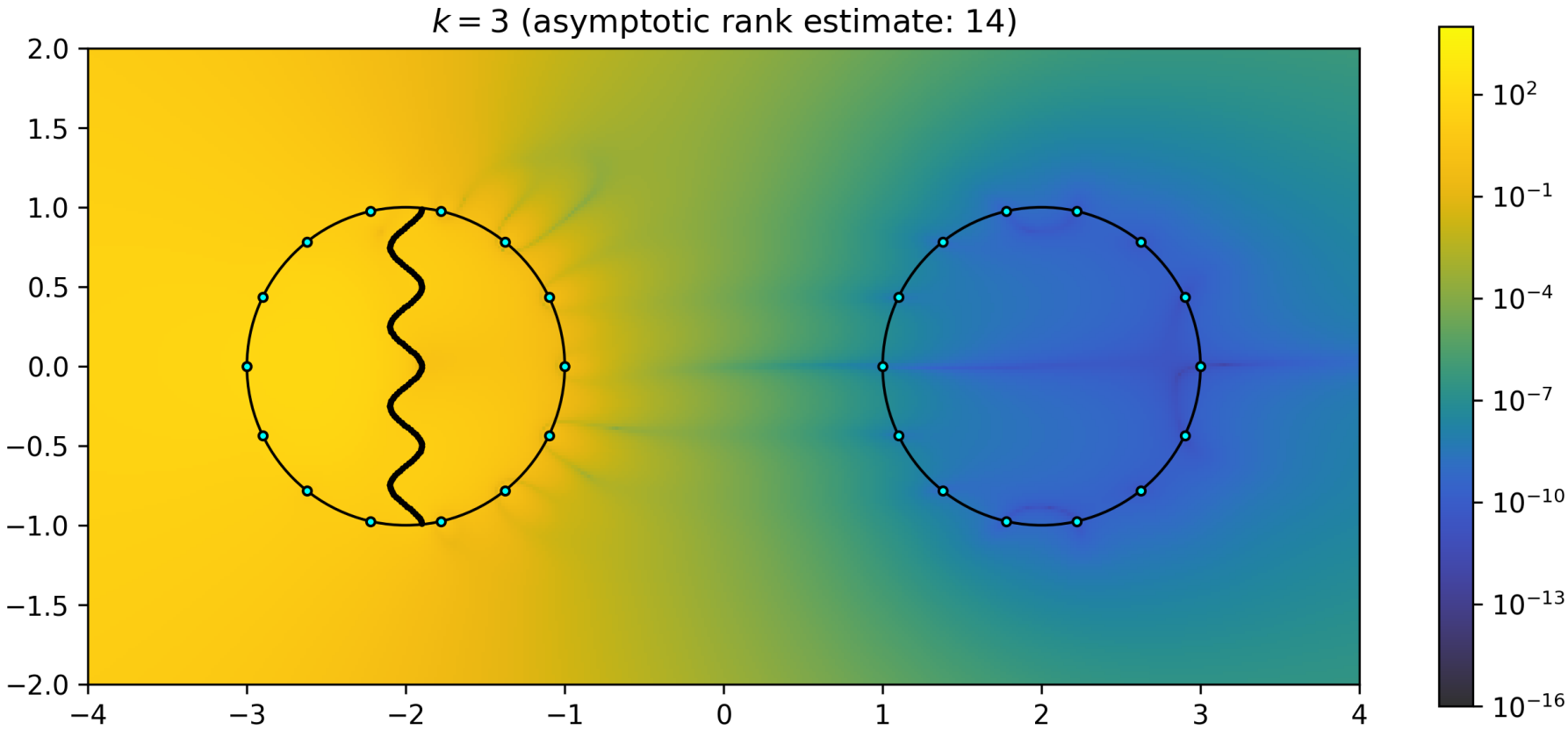
The a priori rank est. is wideband and tight

Requested error tolerance: $\epsilon = 10^{-12}$:



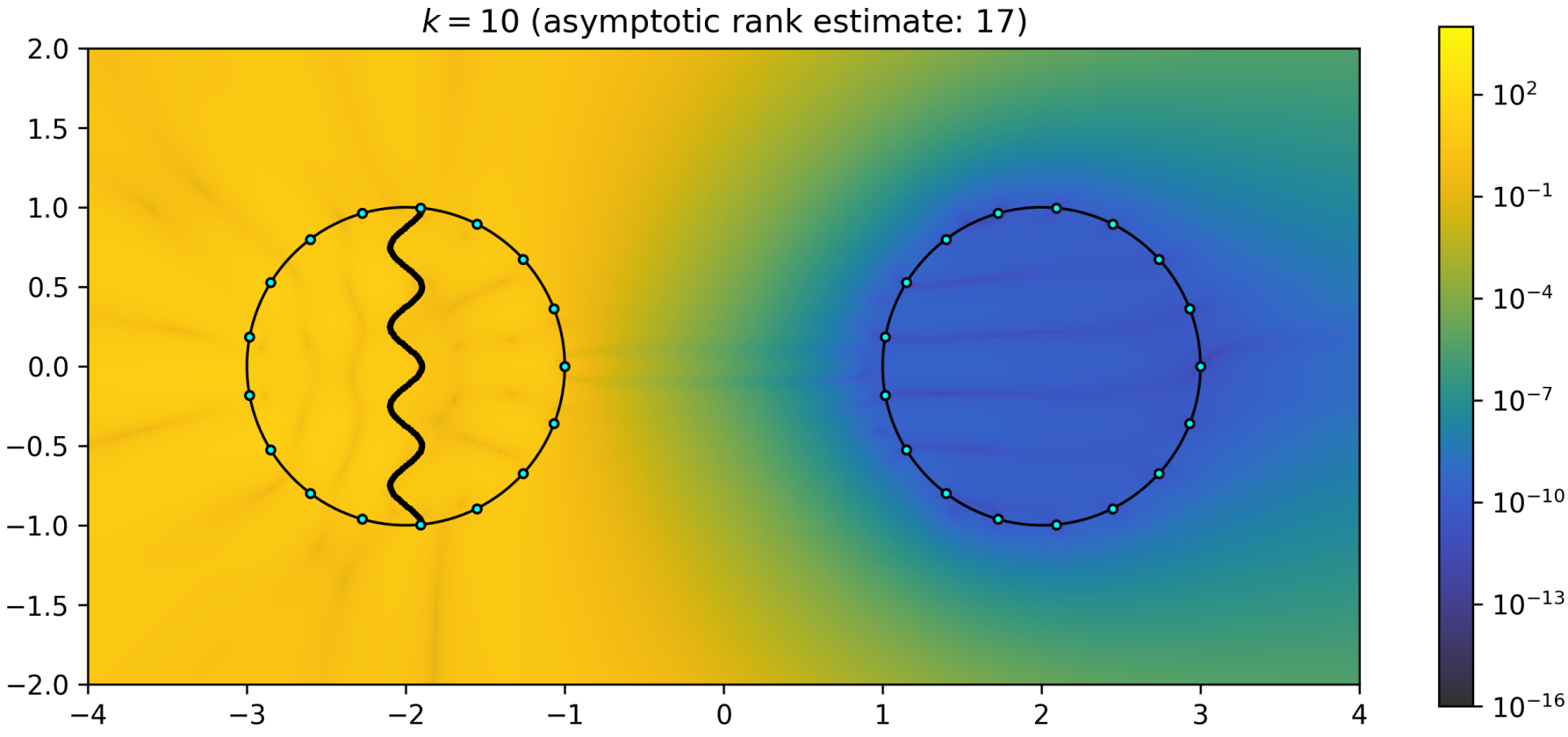
The a priori rank est. is wideband and tight

Requested error tolerance: $\epsilon = 10^{-12}$:



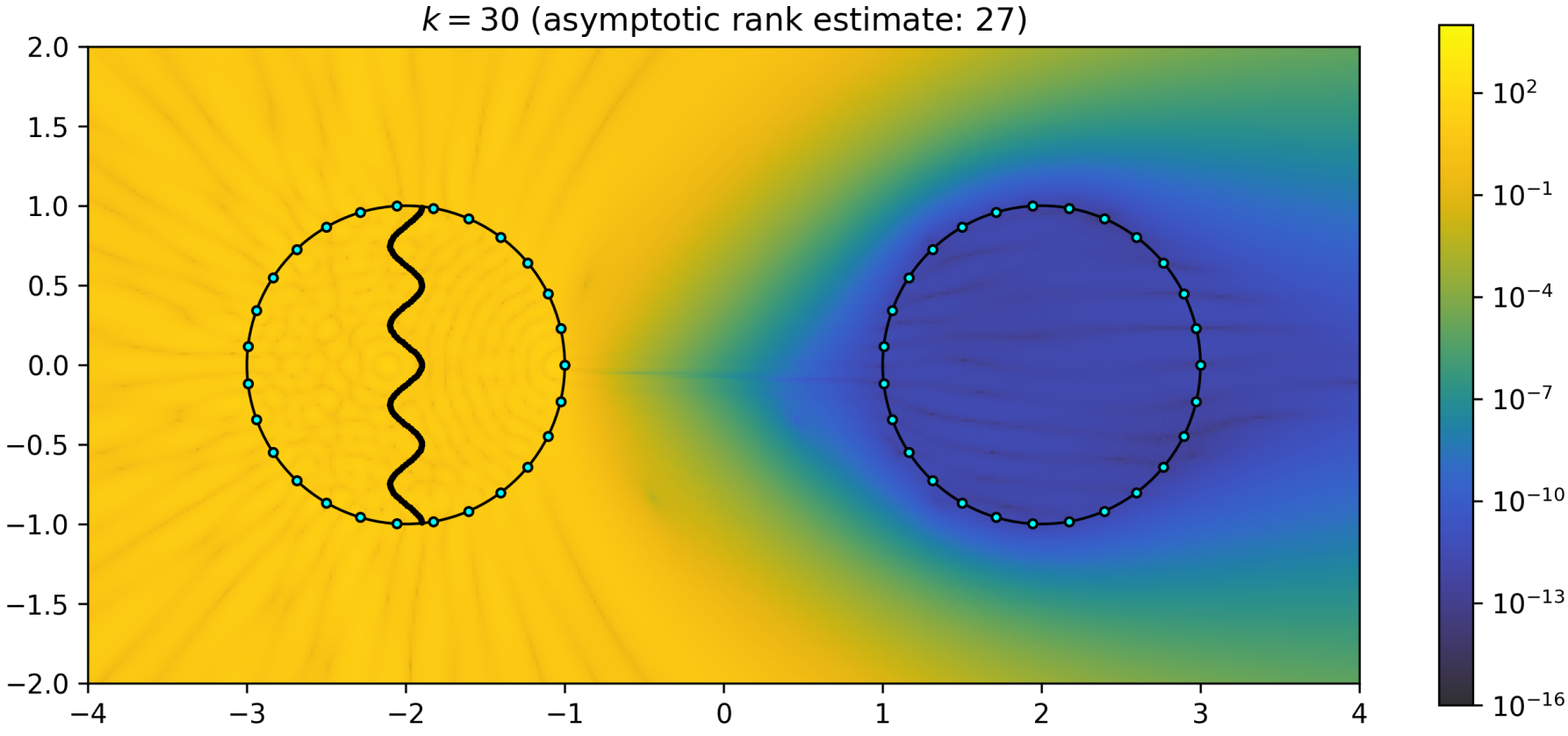
The a priori rank est. is wideband and tight

Requested error tolerance: $\epsilon = 10^{-12}$:



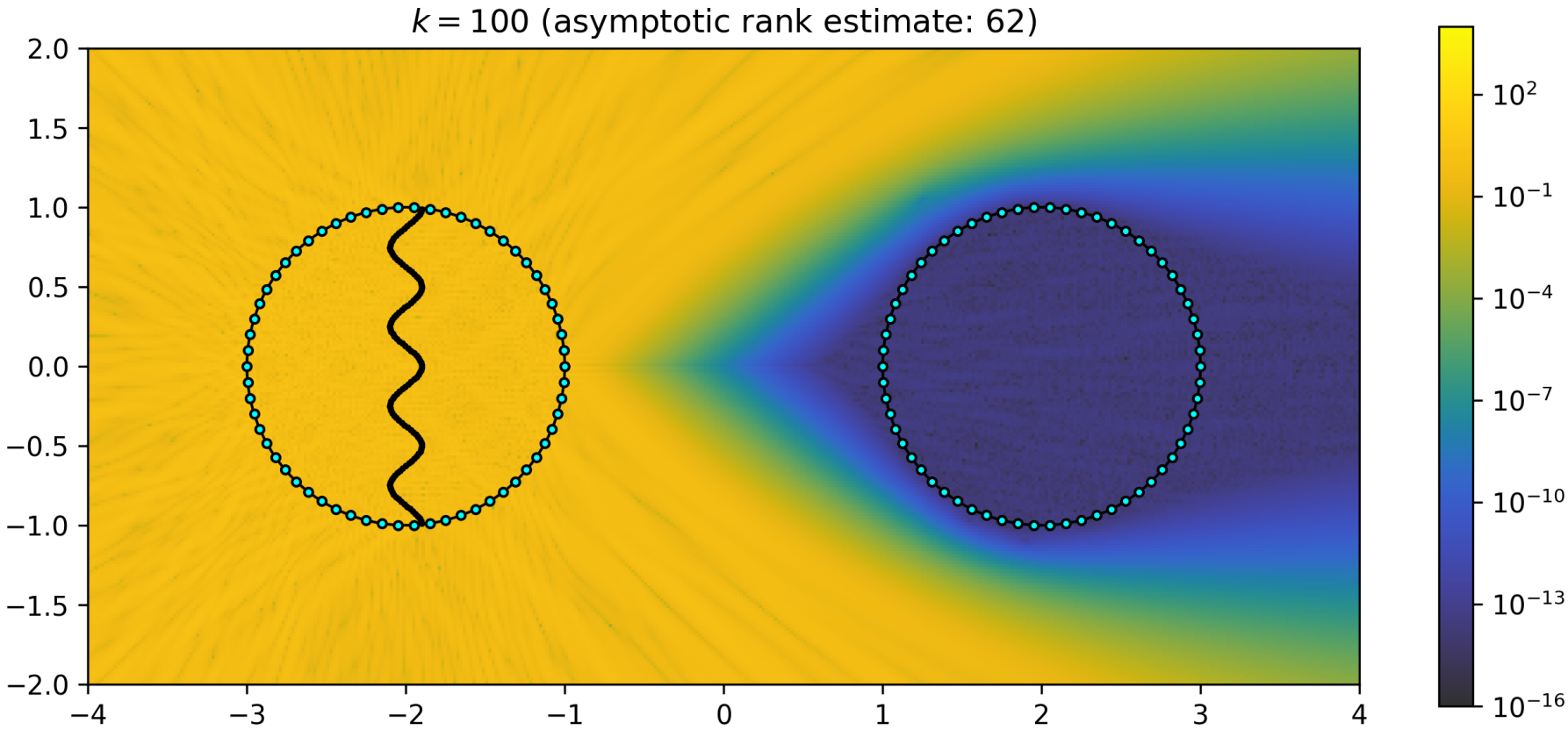
The a priori rank est. is wideband and tight

Requested error tolerance: $\epsilon = 10^{-12}$:



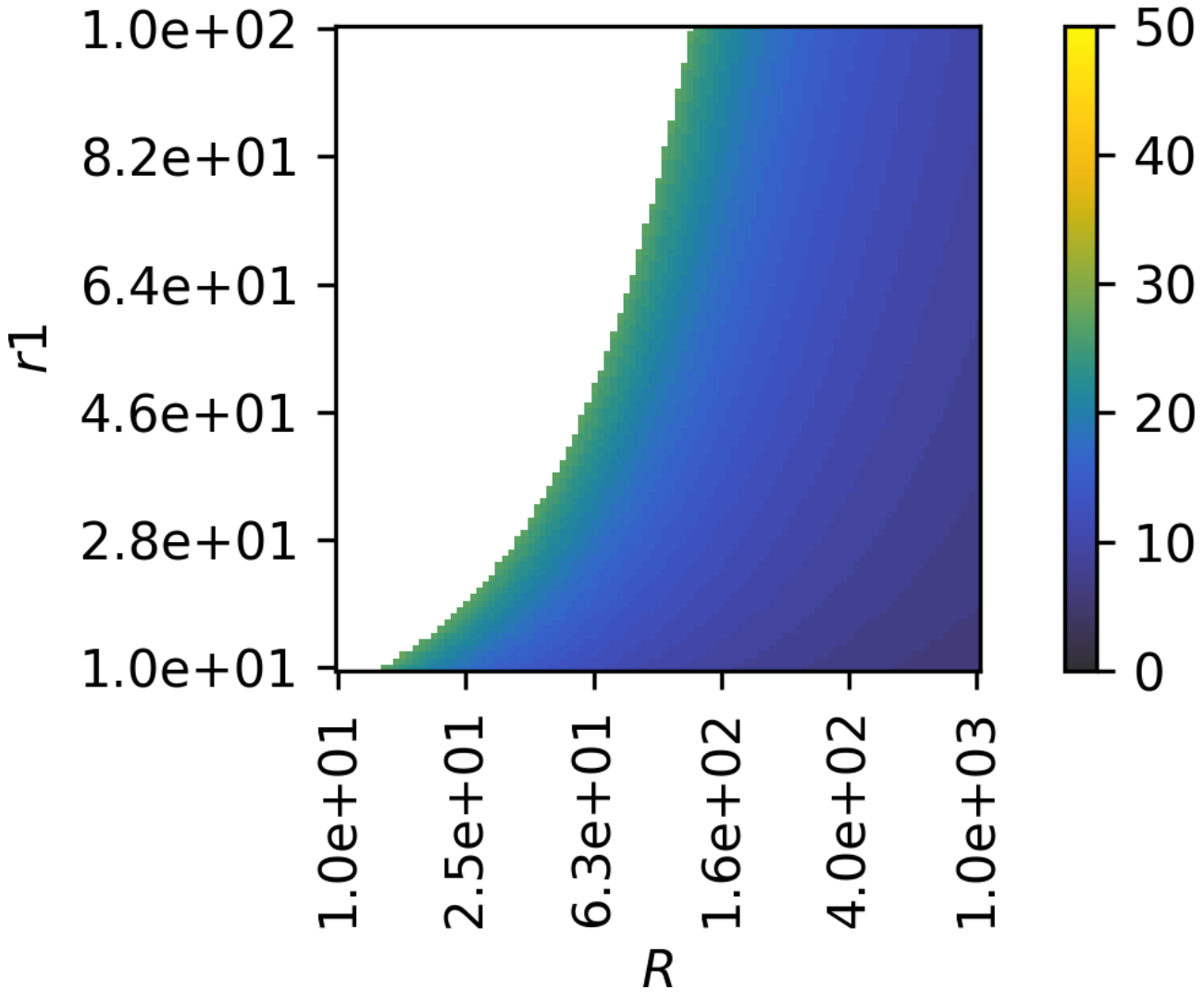
The a priori rank est. is wideband and tight

Requested error tolerance: $\epsilon = 10^{-12}$:



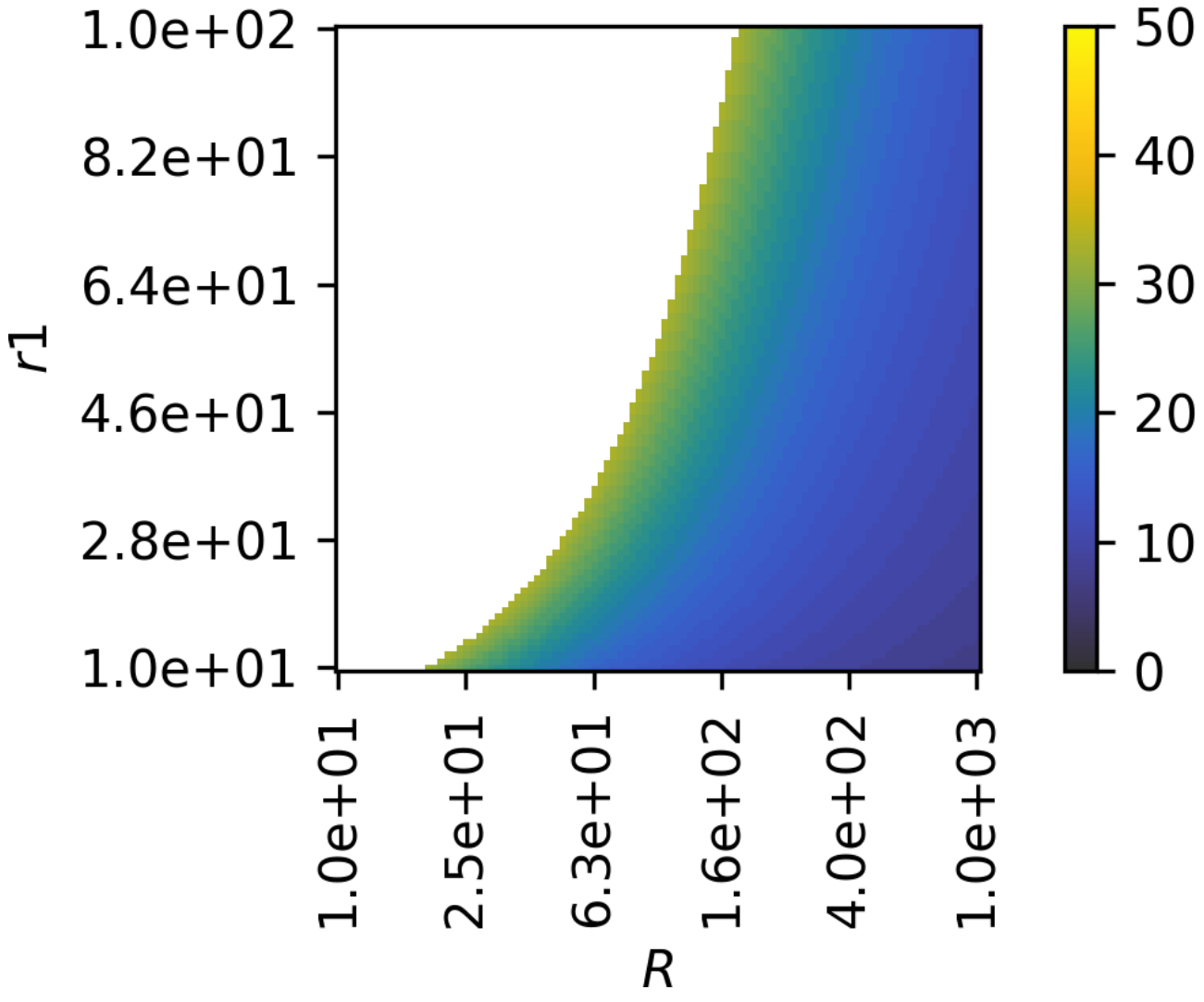
The a priori rank est. is wideband and tight

True rank of kernel subblock ($k = 10$, rank est. ≤ 50):



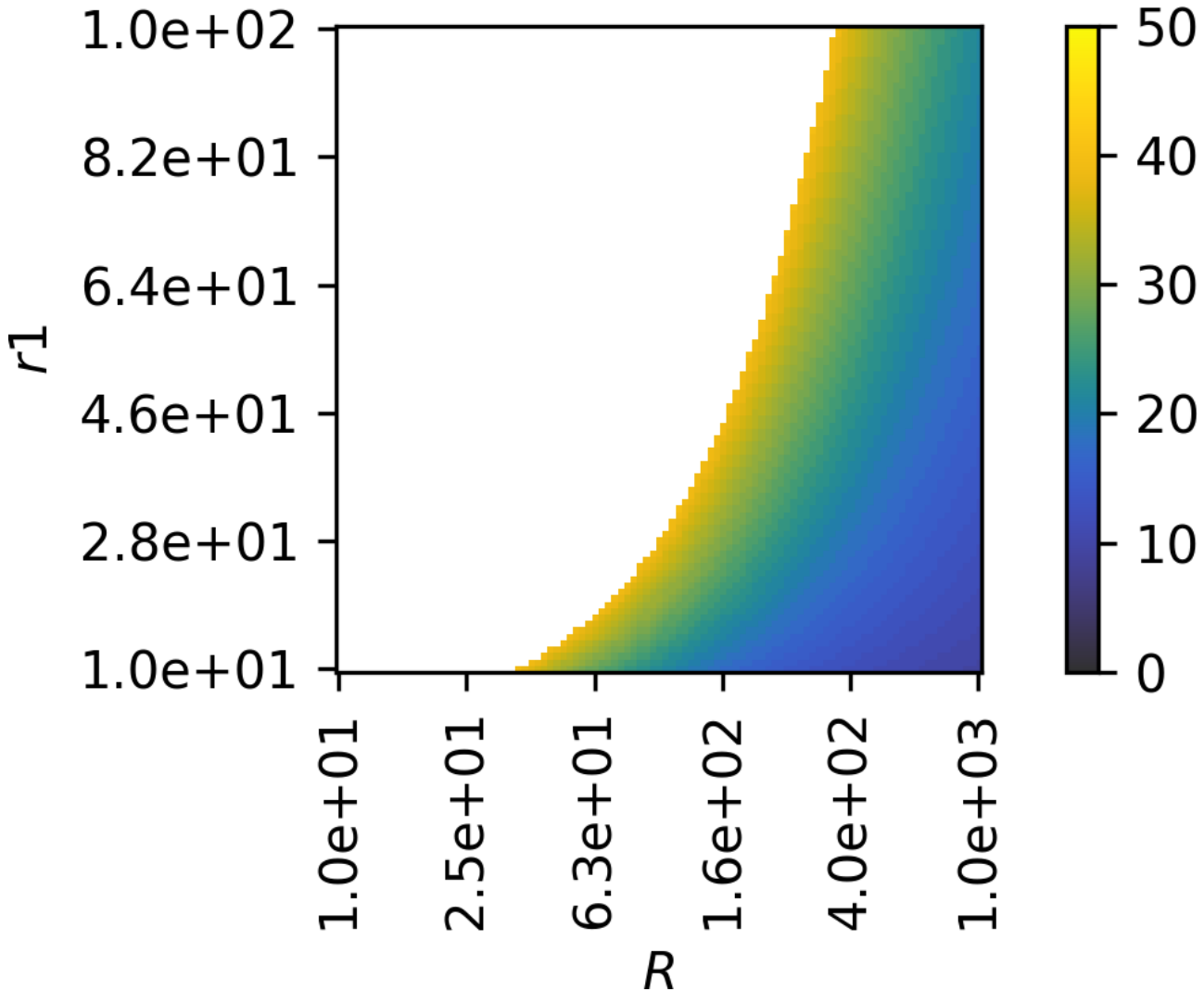
The a priori rank est. is wideband and tight

True rank of kernel subblock ($k = 30$, rank est. ≤ 50):



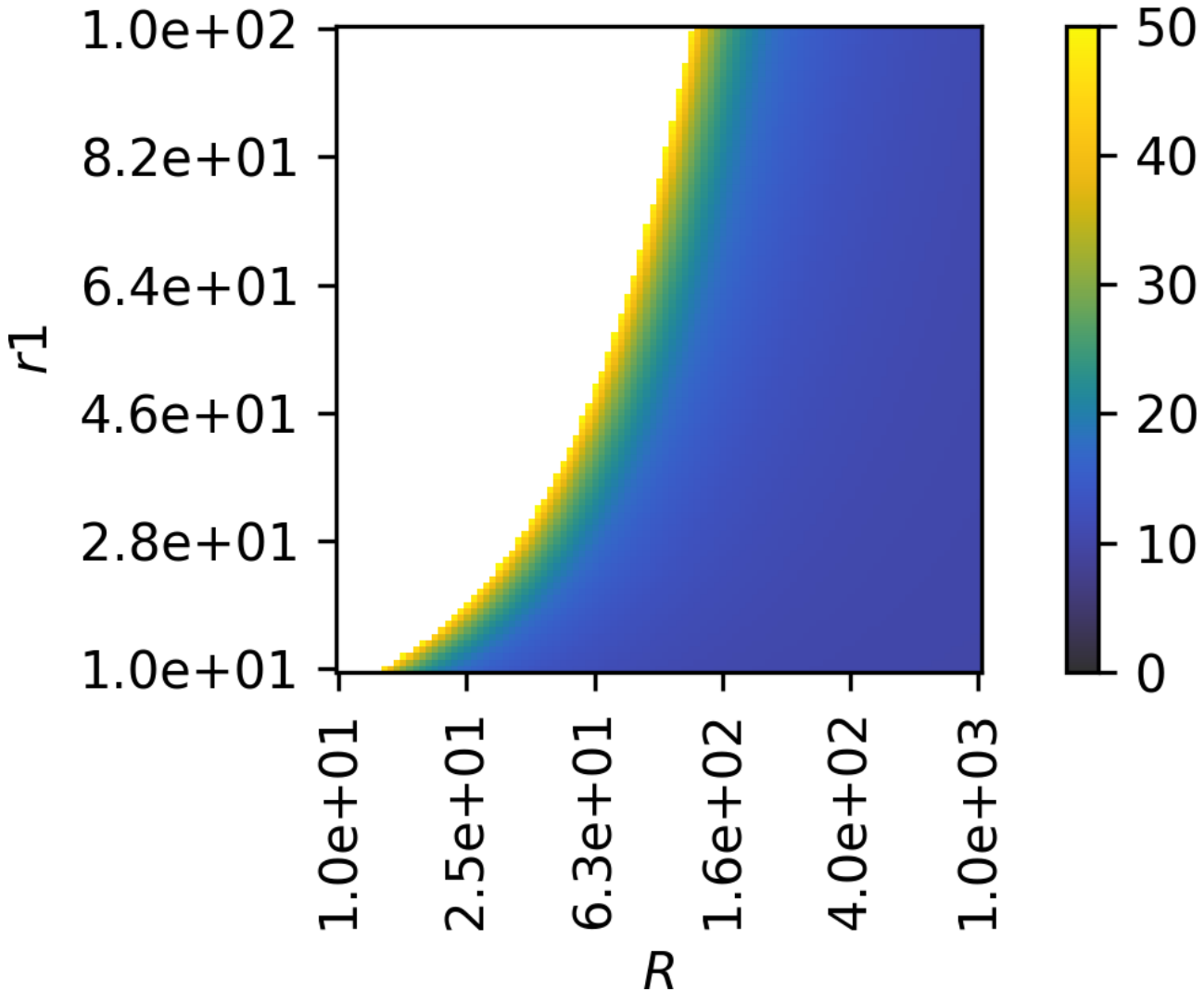
The a priori rank est. is wideband and tight

True rank of kernel subblock ($k = 100$, rank est. ≤ 50):



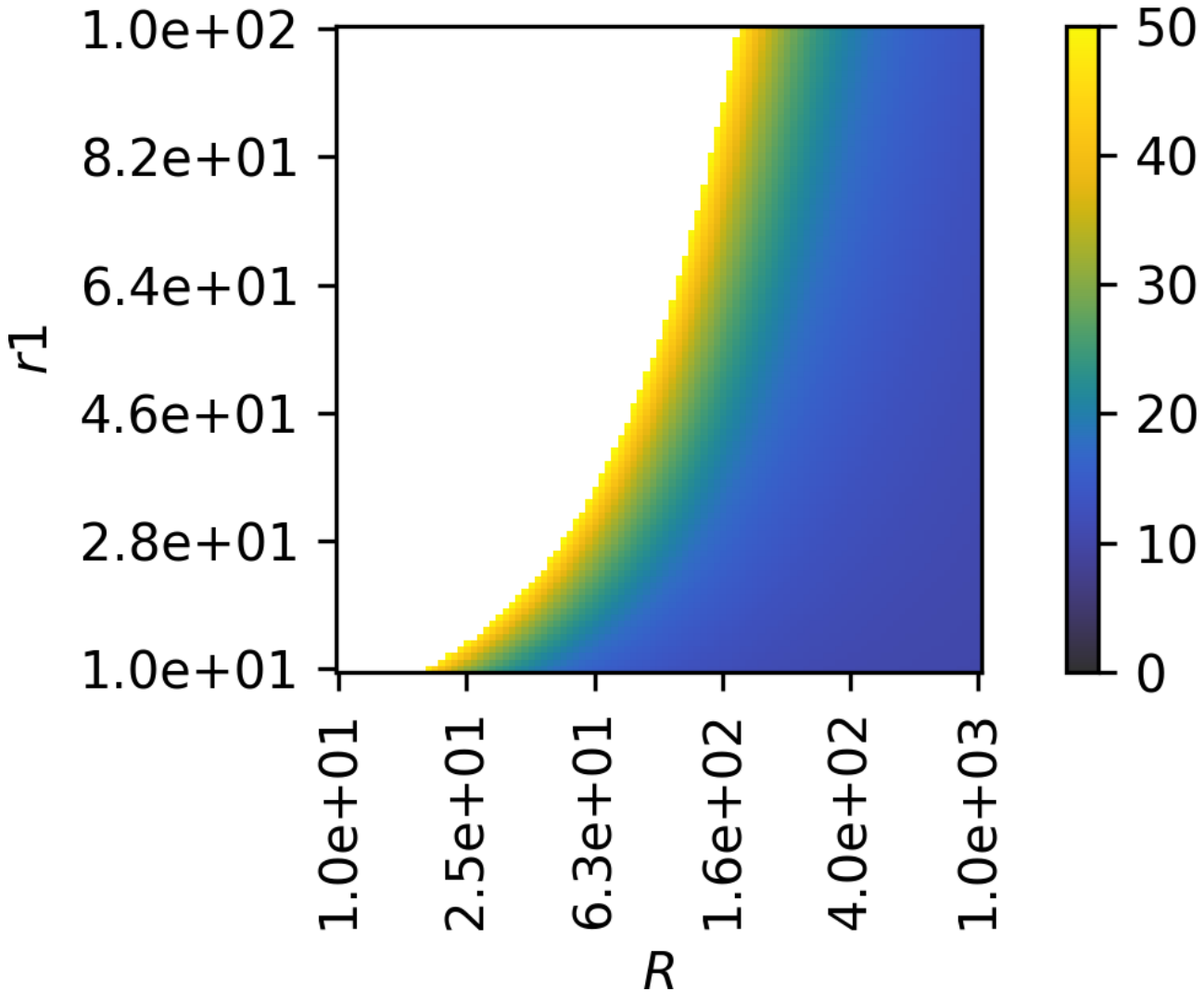
The a priori rank est. is wideband and tight

Est. rank of kernel subblock ($k = 10$, rank est. ≤ 50):



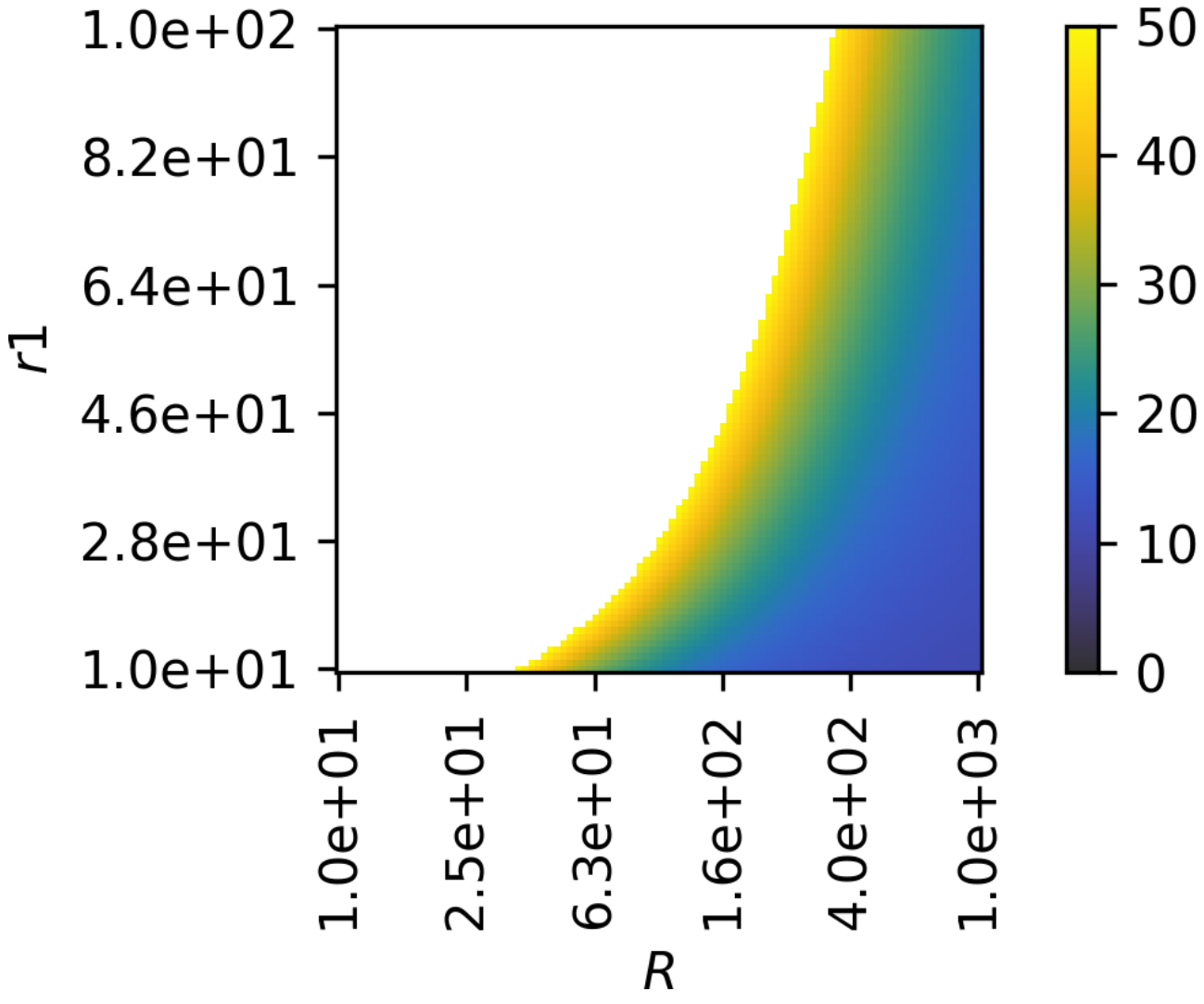
The a priori rank est. is wideband and tight

Est. rank of kernel subblock ($k = 30$, rank est. ≤ 50):



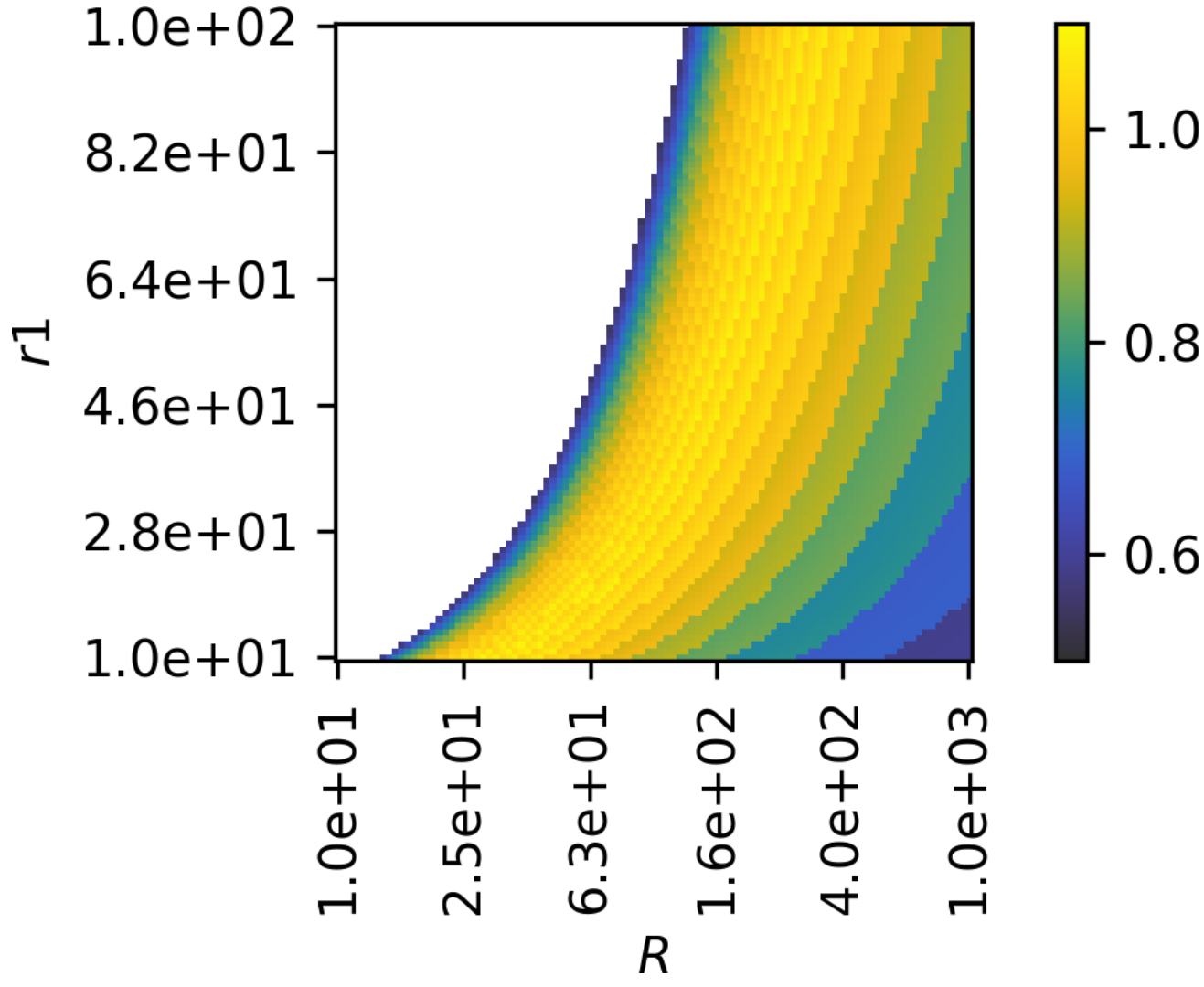
The a priori rank est. is wideband and tight

Est. rank of kernel subblock ($k = 100$, rank est. ≤ 50):



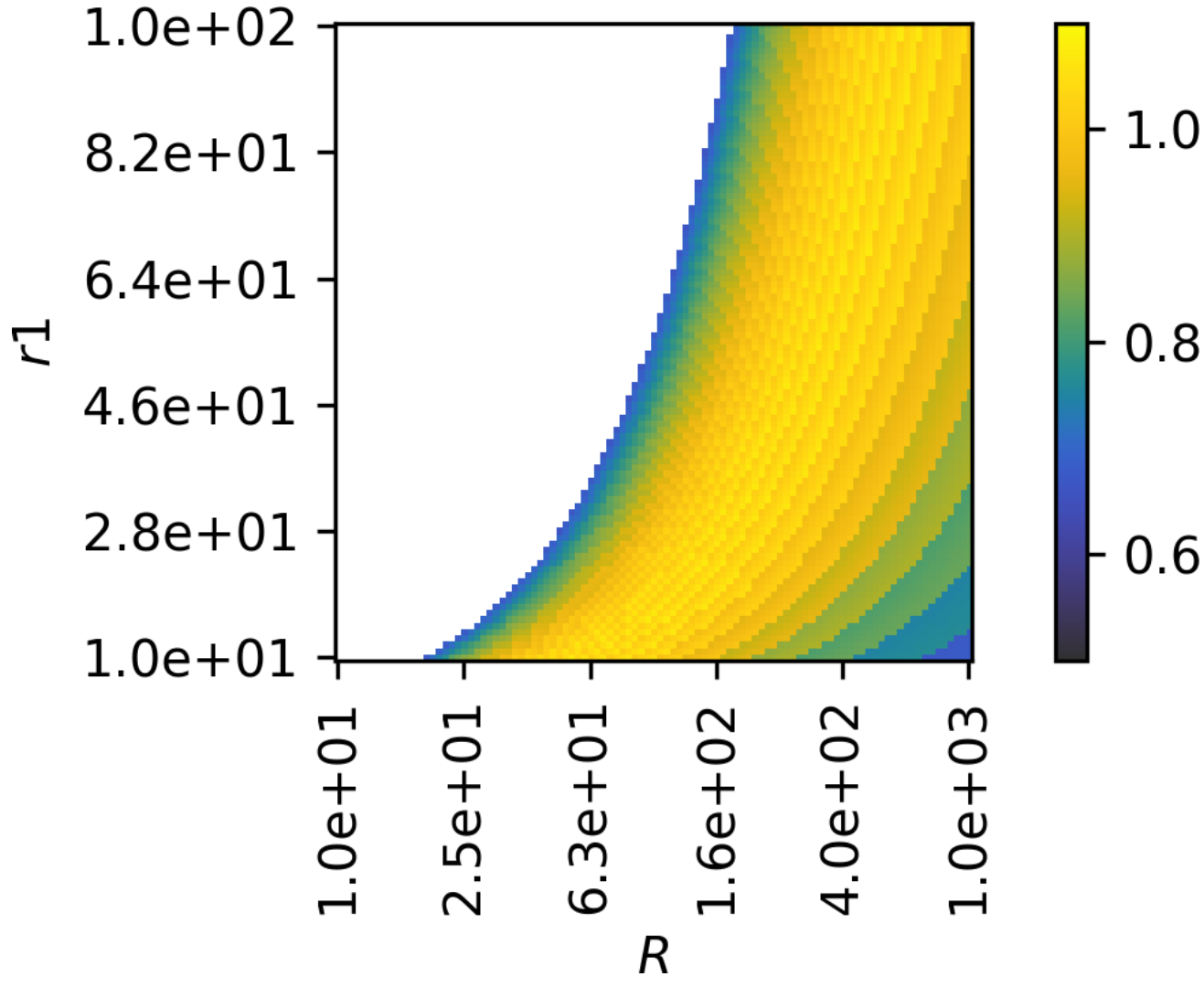
The a priori rank est. is wideband and tight

Rank divided by rank est. ($k = 10$, rank est. ≤ 50):



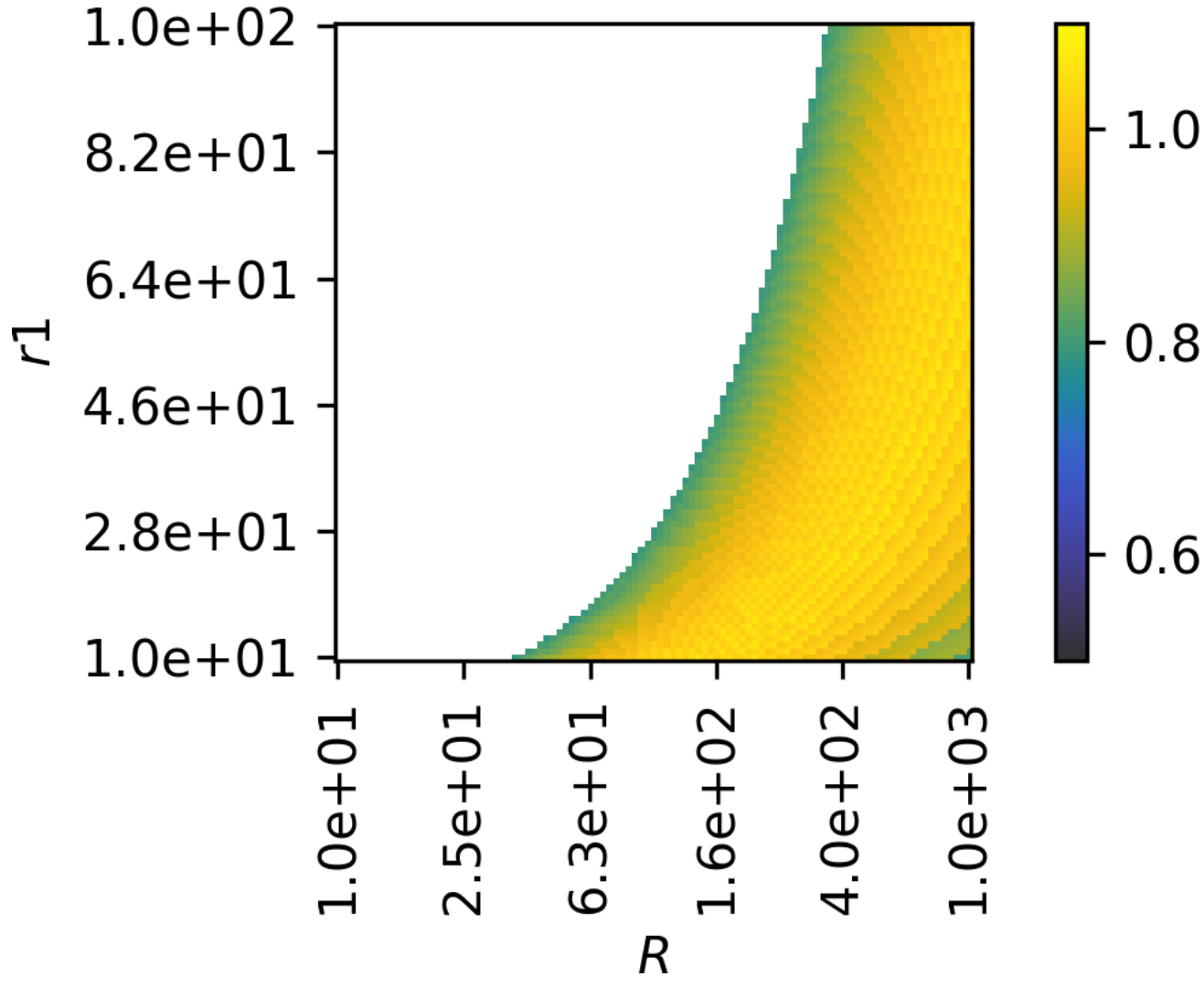
The a priori rank est. is wideband and tight

Rank divided by rank est. ($k = 30$, rank est. ≤ 50):



The a priori rank est. is wideband and tight

Rank divided by rank est. ($k = 100$, rank est. ≤ 50):



Multilevel butterfly factorization

- Butterfly each kernel subblock corresponding to well-separated nodes in the quadtree
- Time and space complexity of one butterfly factorization: $O(N \log k)$.
- ... of a multilevel butterfly factorization: $O(N \log N \log k)$.
- Same complexity for assembly *and* multiplication, but assembly is 1-2 orders of magnitude slower.
- If $k = O(1)$, then $O(N \log N)$ complexity.
- If $k = \Theta(N)$, then $O(N \log^2 N)$ complexity.

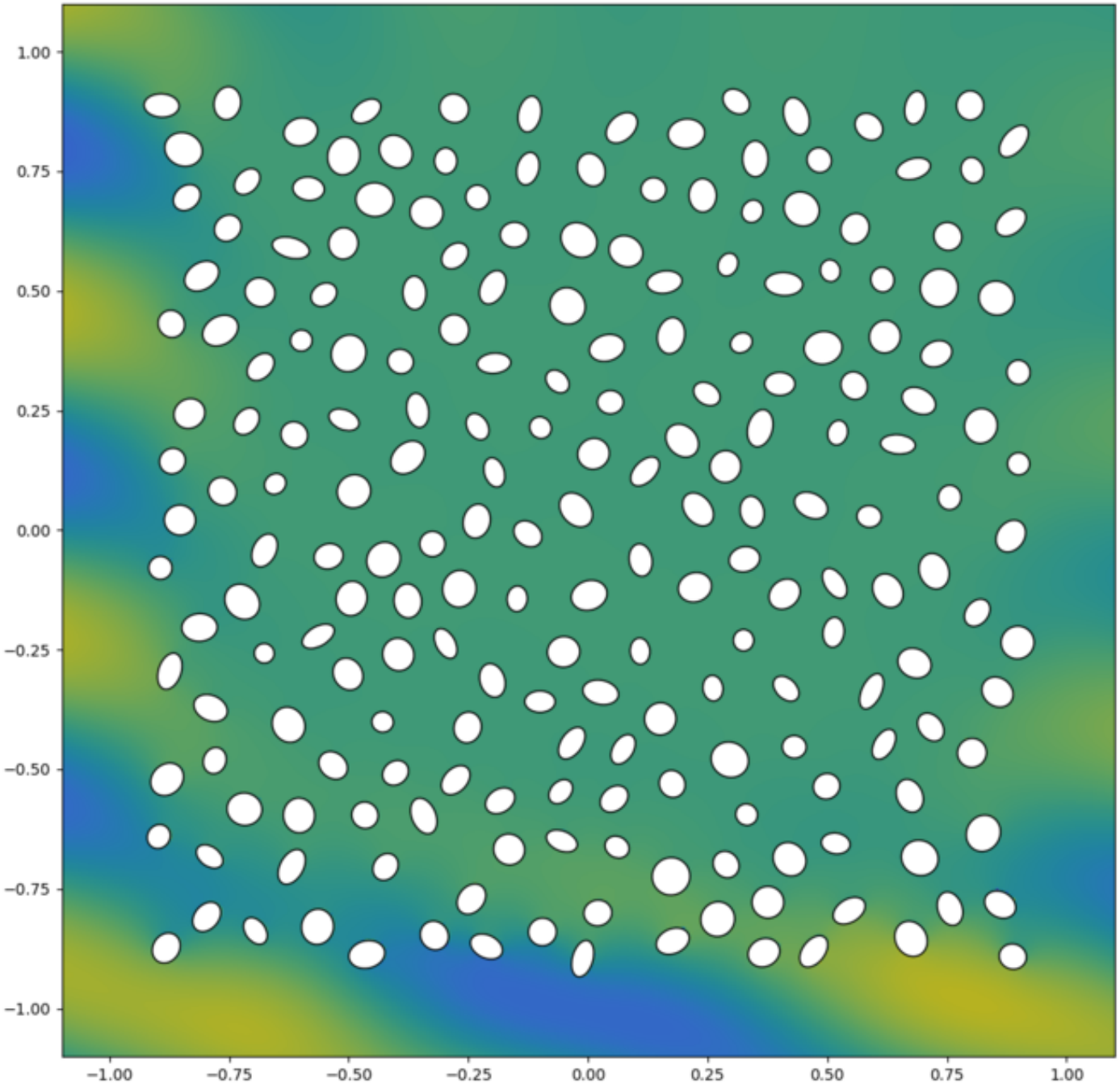
Note! the multilevel butterfly and the HF-FMM have the *same* time complexity, $O(N \log N \log k)$.

Test problem

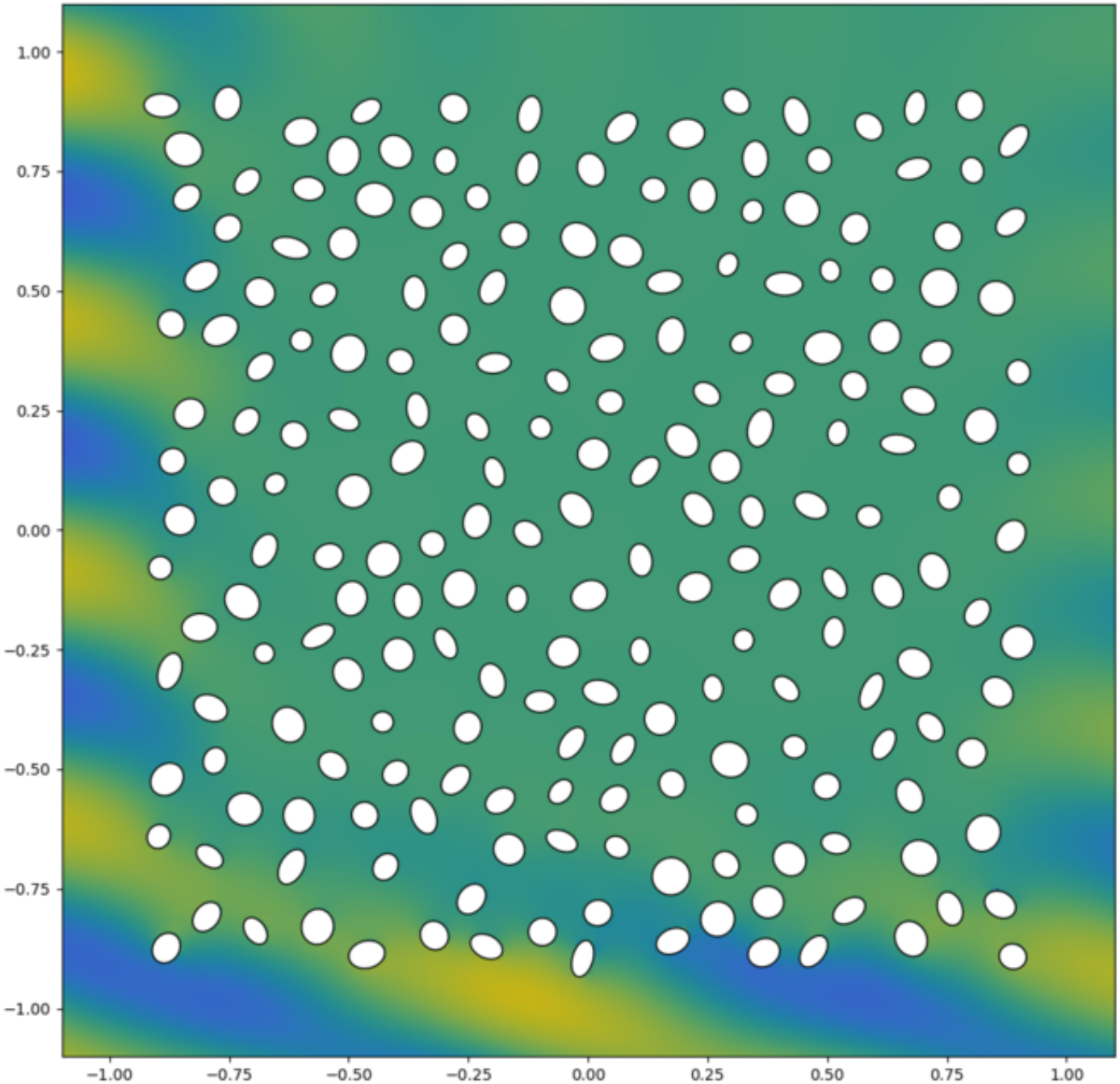
- Randomly sample nonoverlapping ellipses in the box $[-1,1] \times [-1,1]$ using Poisson disk sampling.
- Set $u_{\text{in}} = e^{ik\hat{d}\cdot x}$ for some choice of $\hat{d}$.
- Nÿstrom discretization using punctured trapezoid rule with 6th order Kapur-Rokhlin correction.
- Use a block Jacobi preconditioner to reduce number of GMRES iterations: for each ellipse, compute LU decomposition of corresponding block of K. (Requires each ellipse to have $O(\sqrt{n})$ points.)

Our focus is on the relative **performance** (speed, memory) of the algorithms. **Not** the accuracy of solving this particular problem.

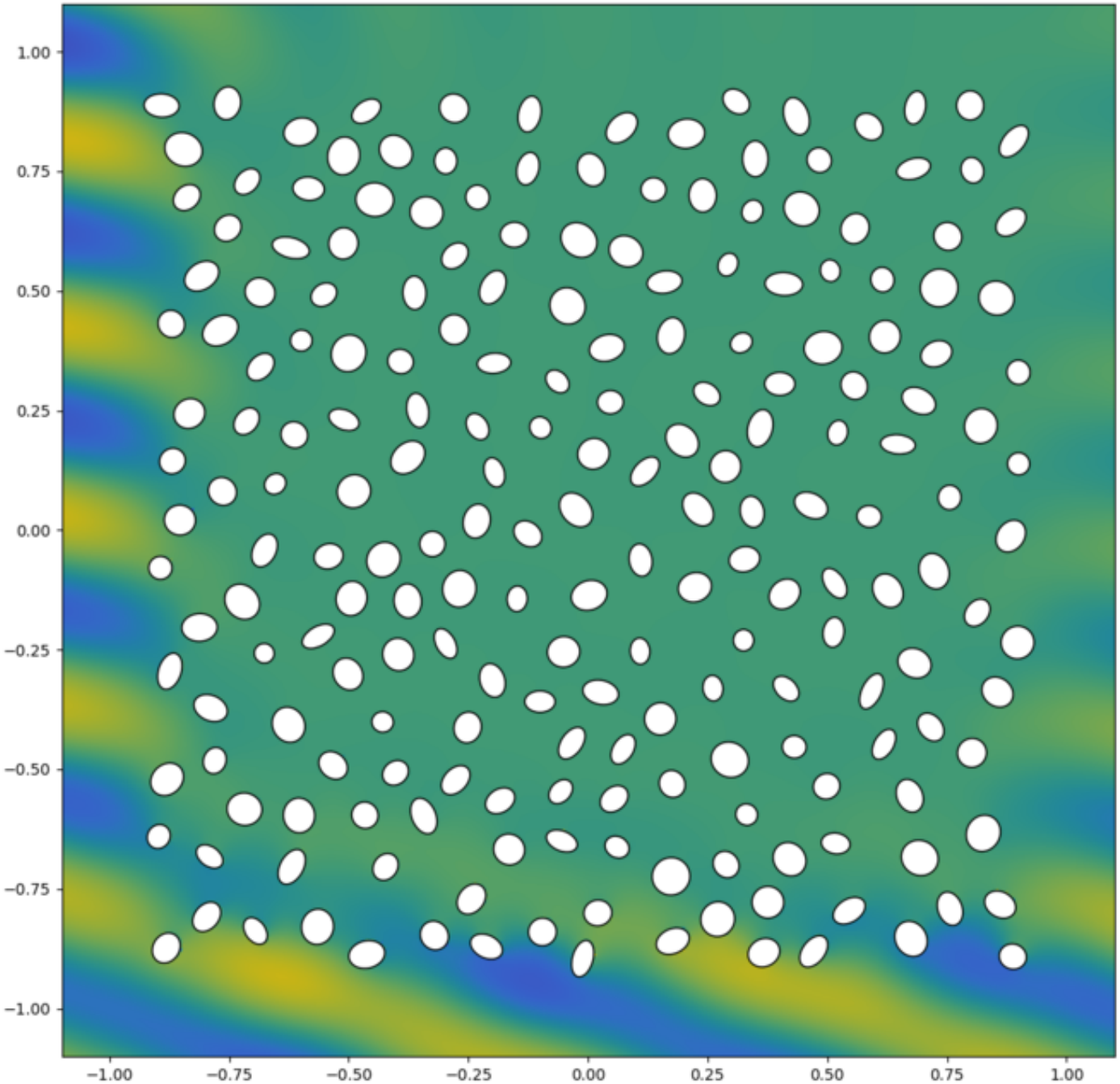
$k = 10$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

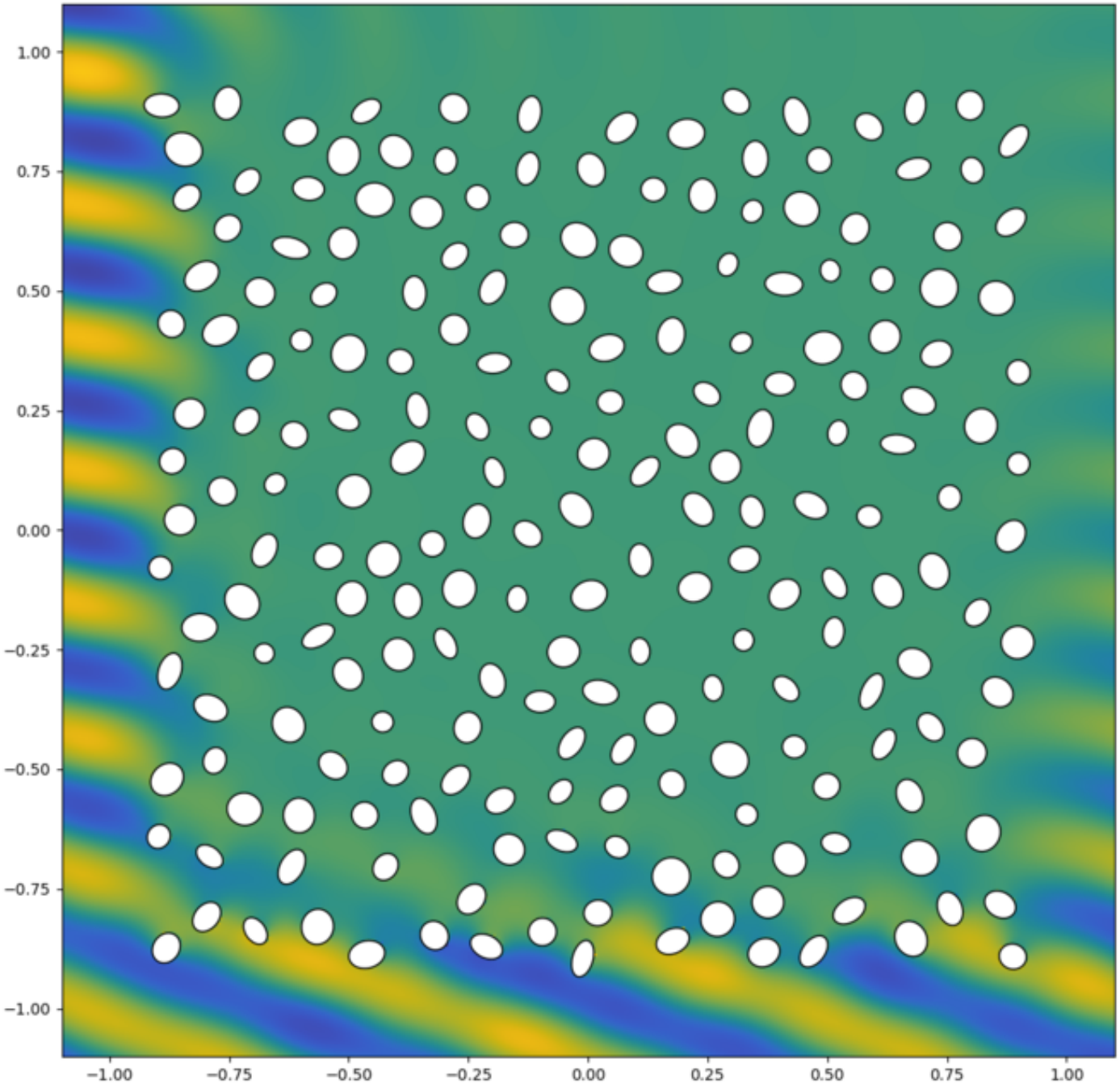
$k = 13$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

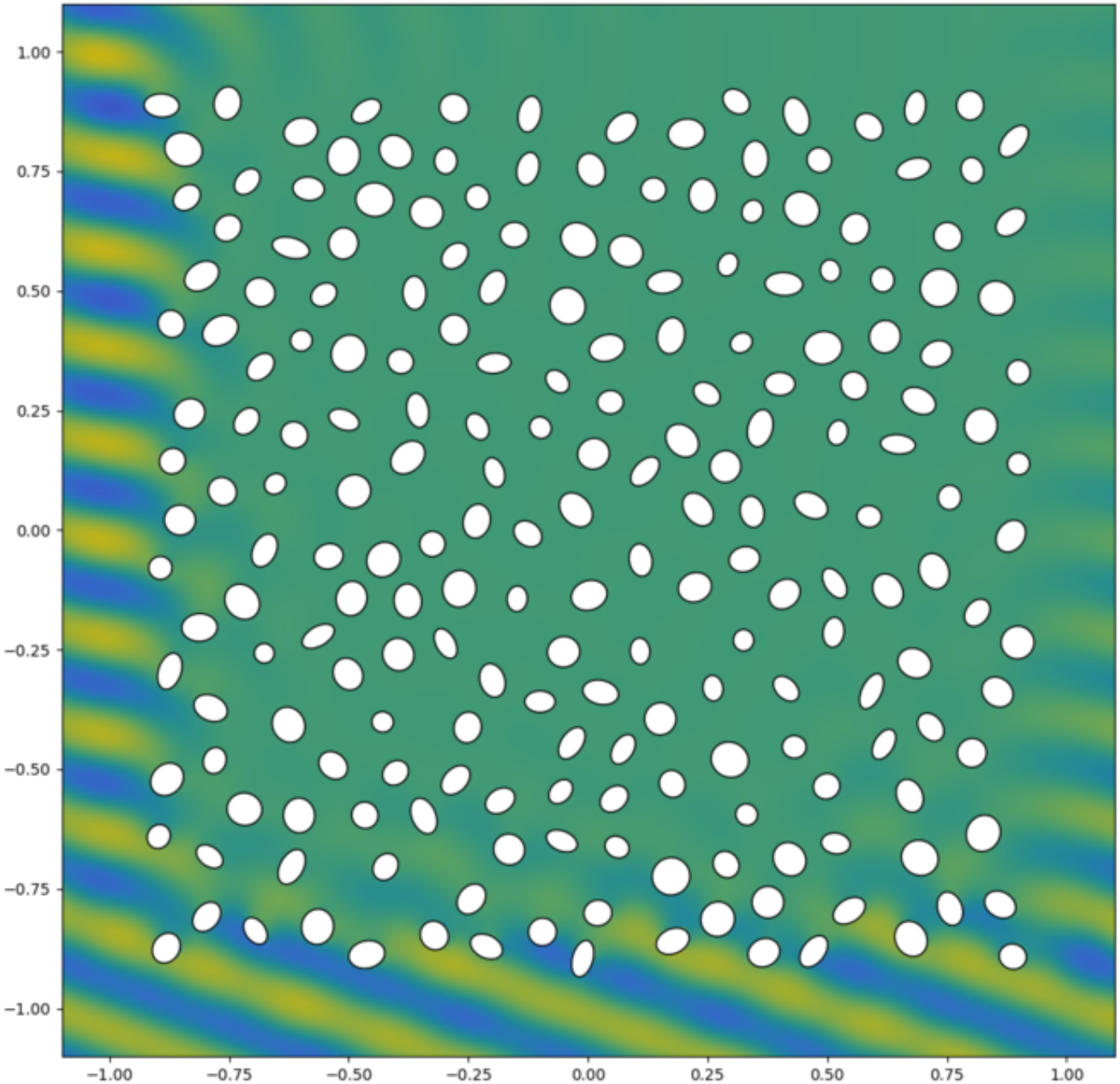
$k = 17$ $(\Re u)$


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

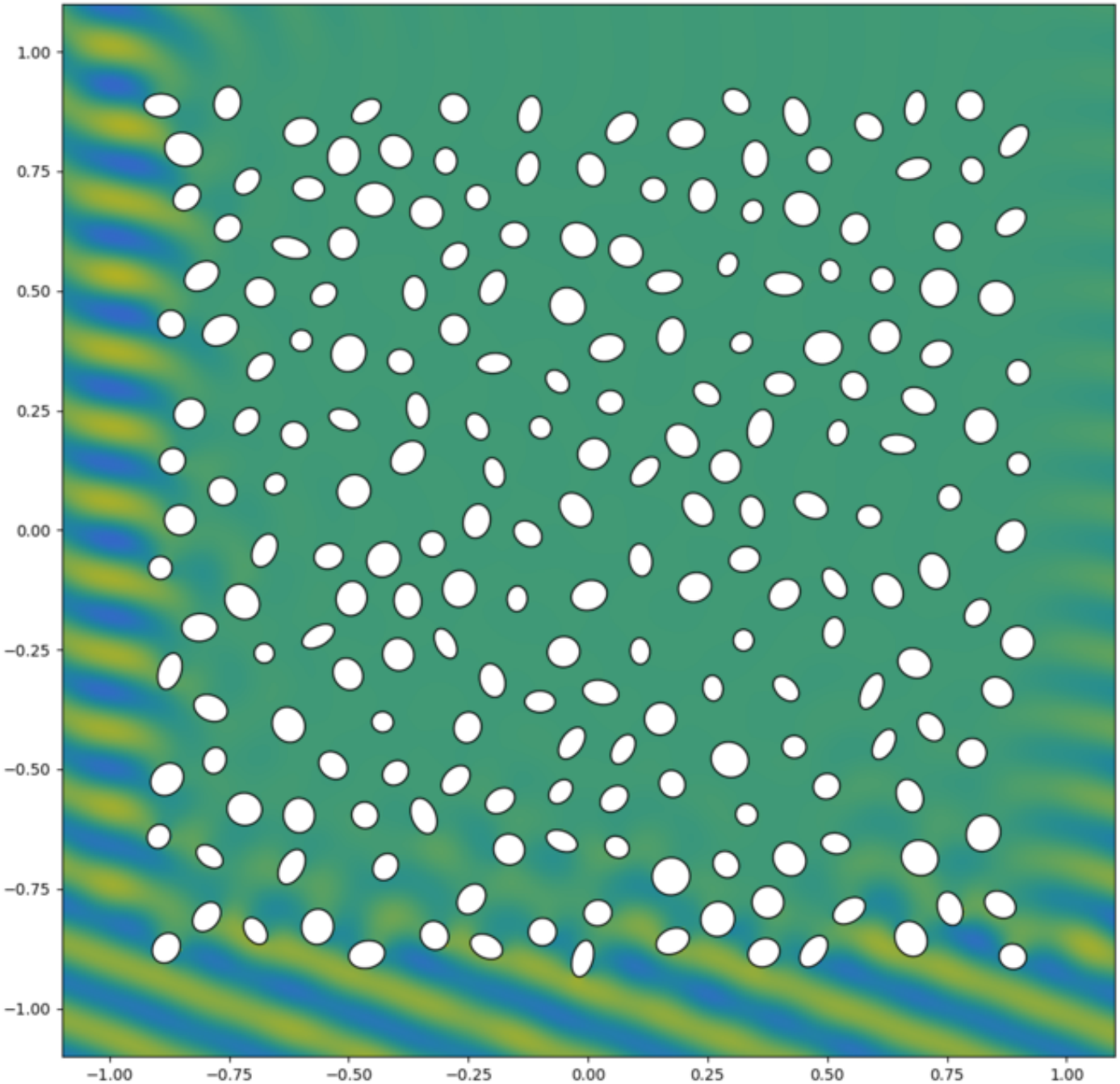
$k = 24$ $(\Re u)$


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

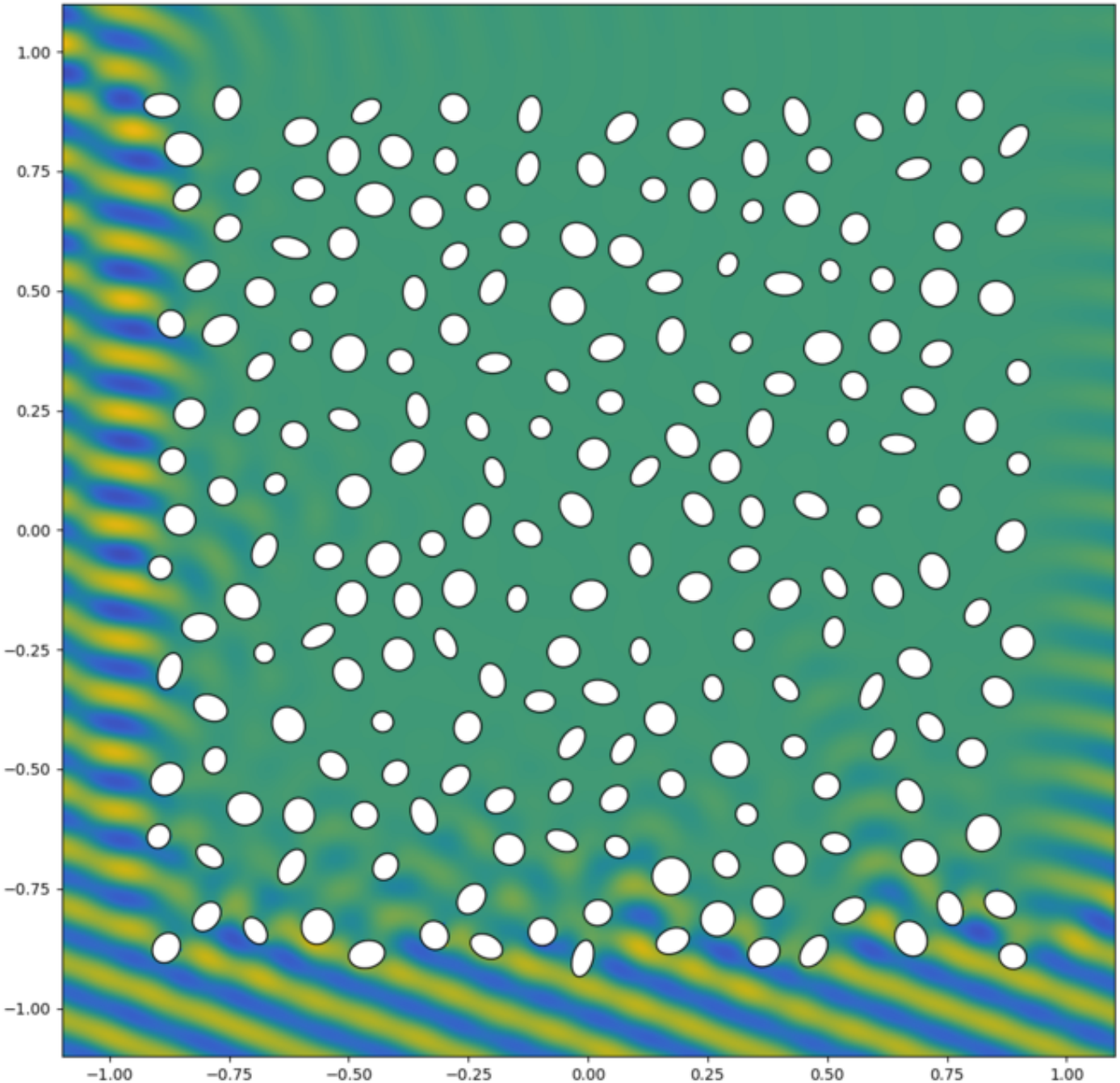
$k = 33$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

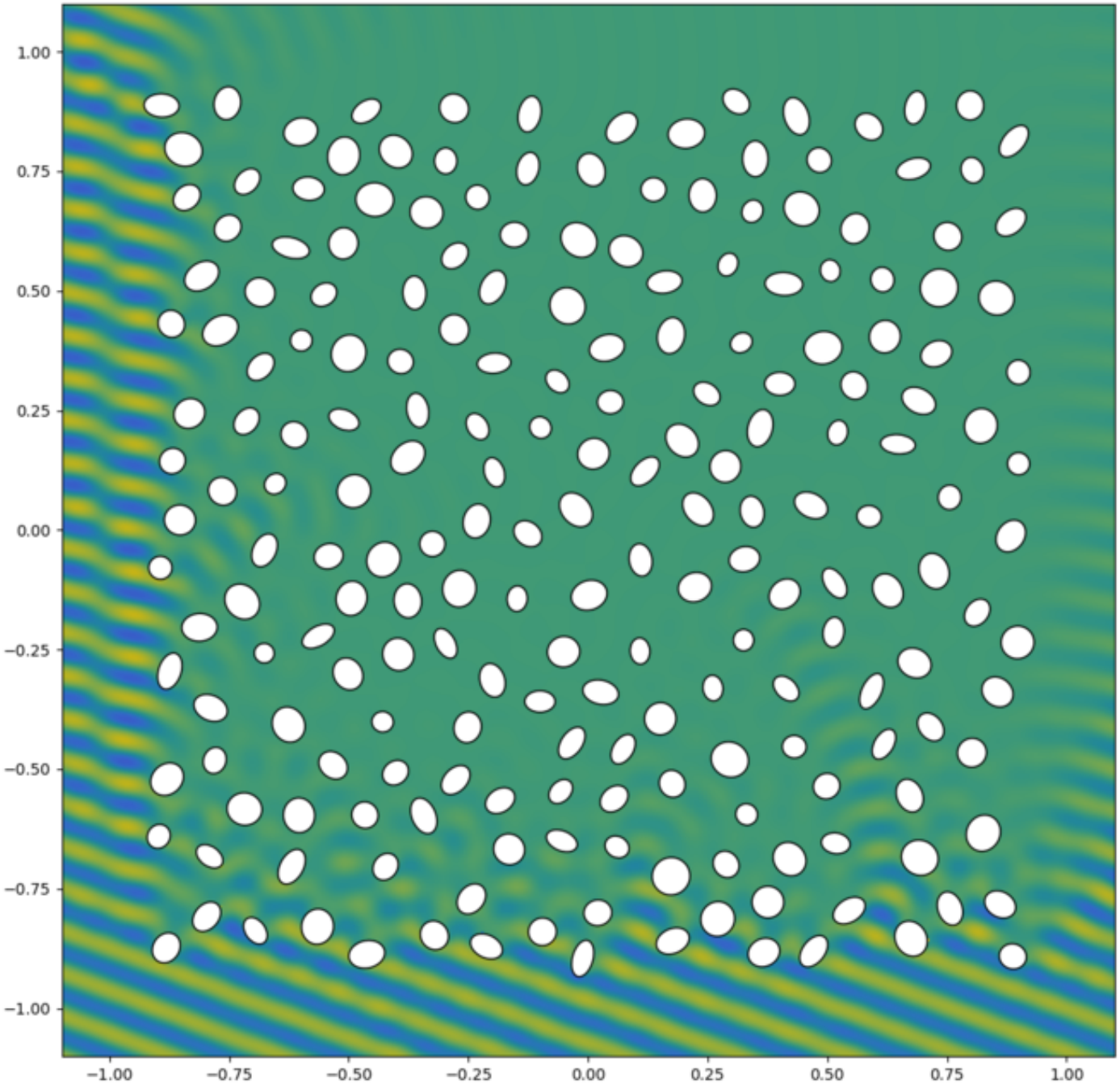
$k = 42$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

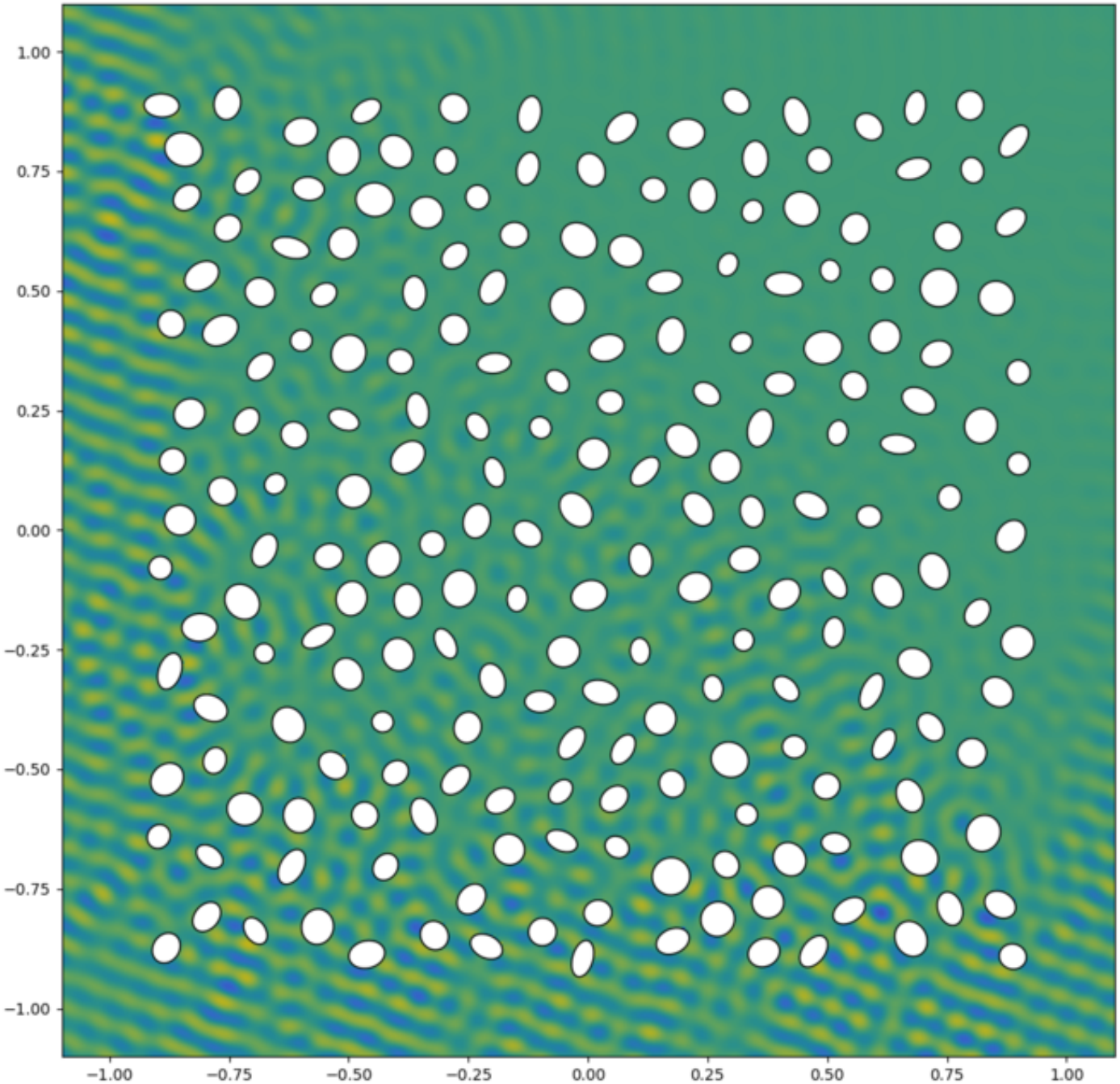
$k = 56$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

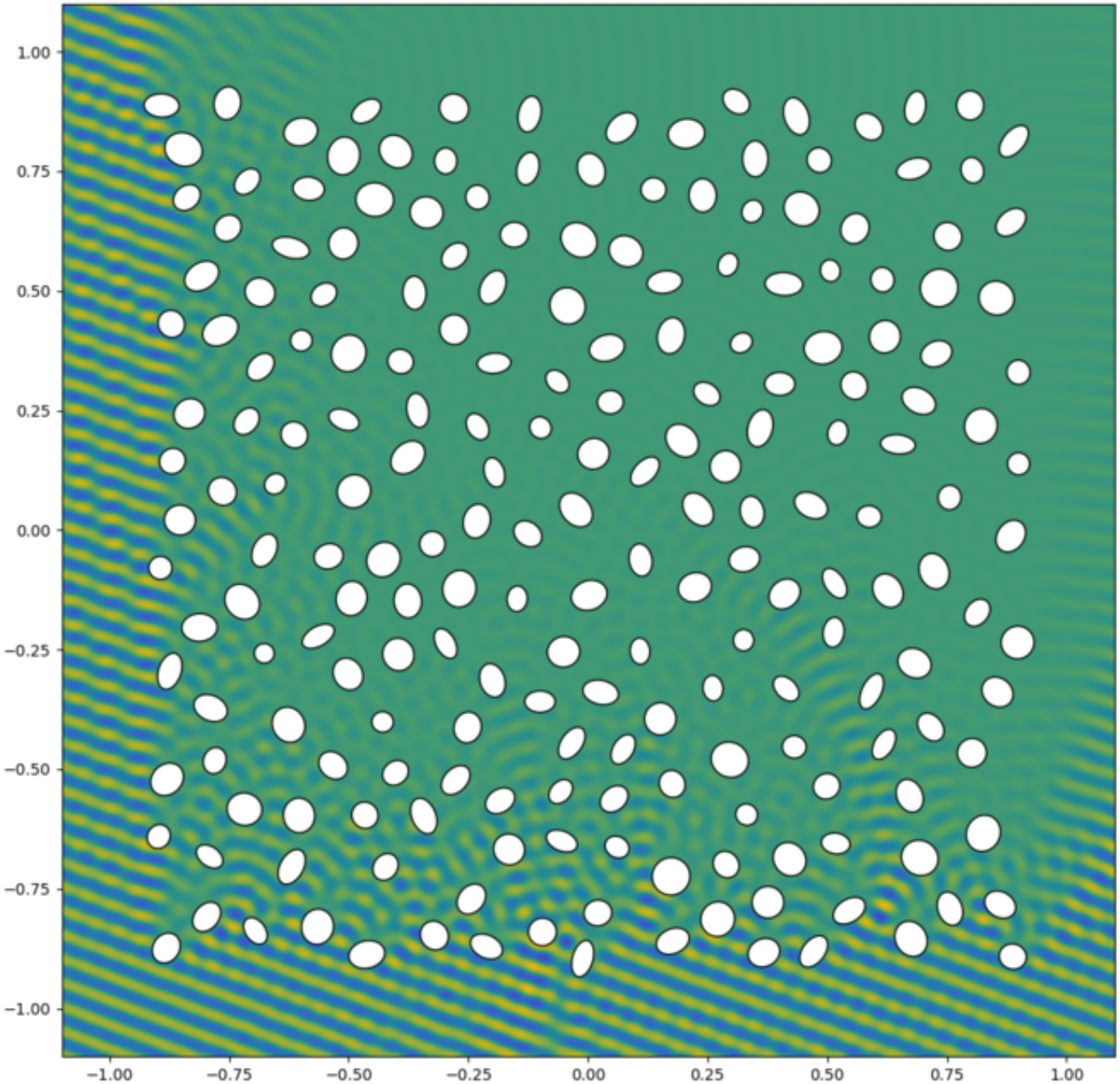
$k = 75$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

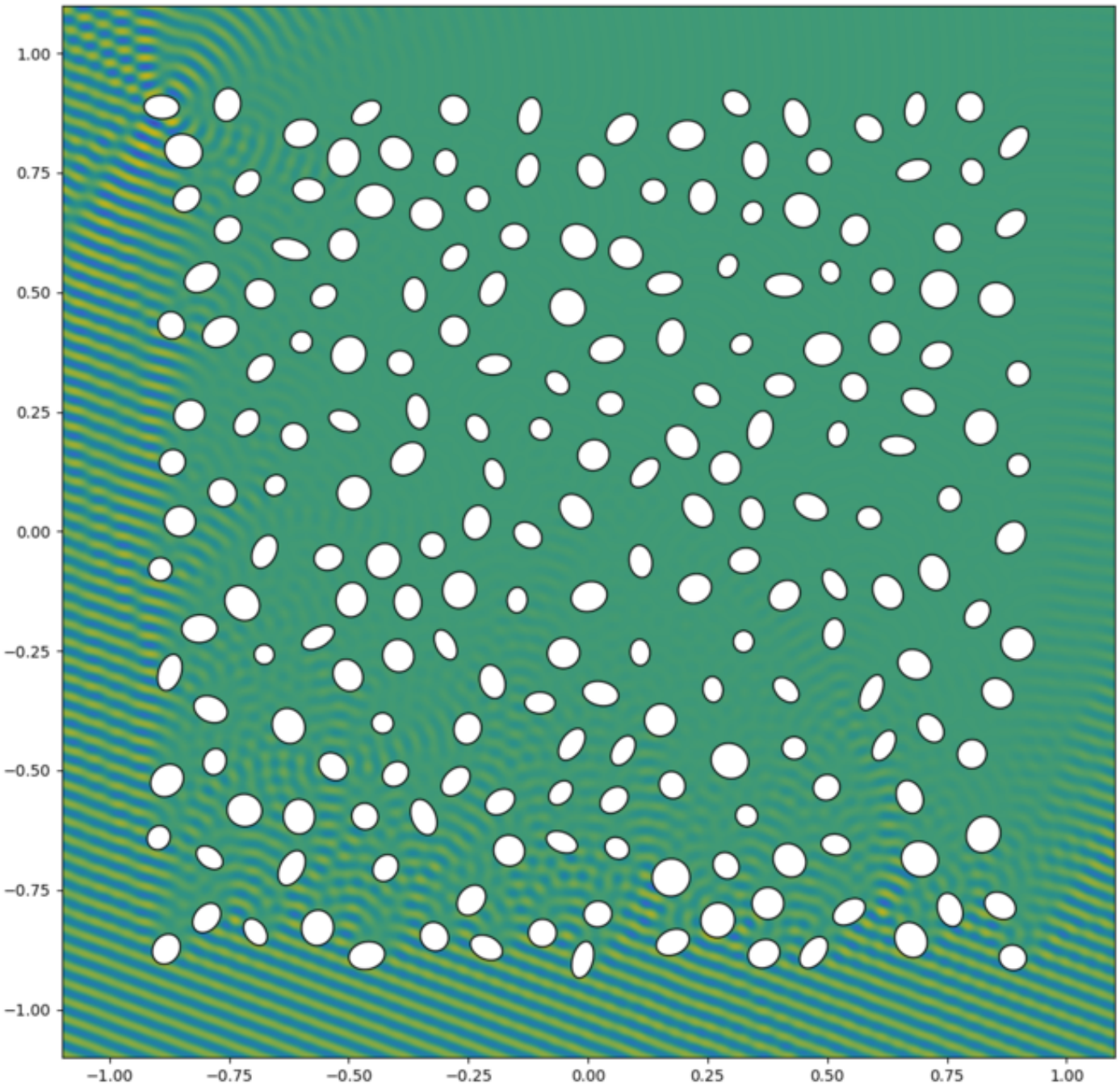
$k = 100$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

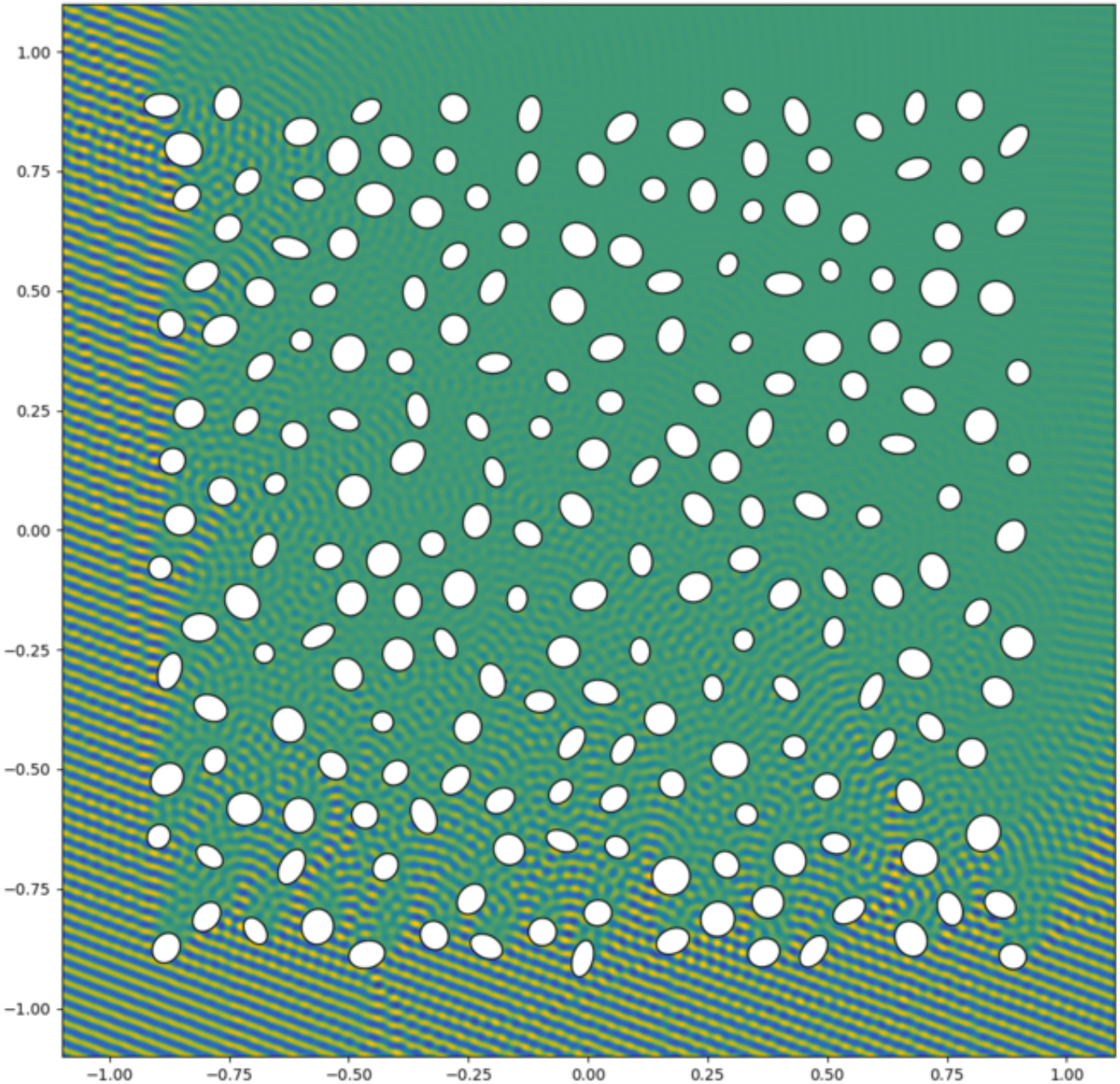
$k = 133$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

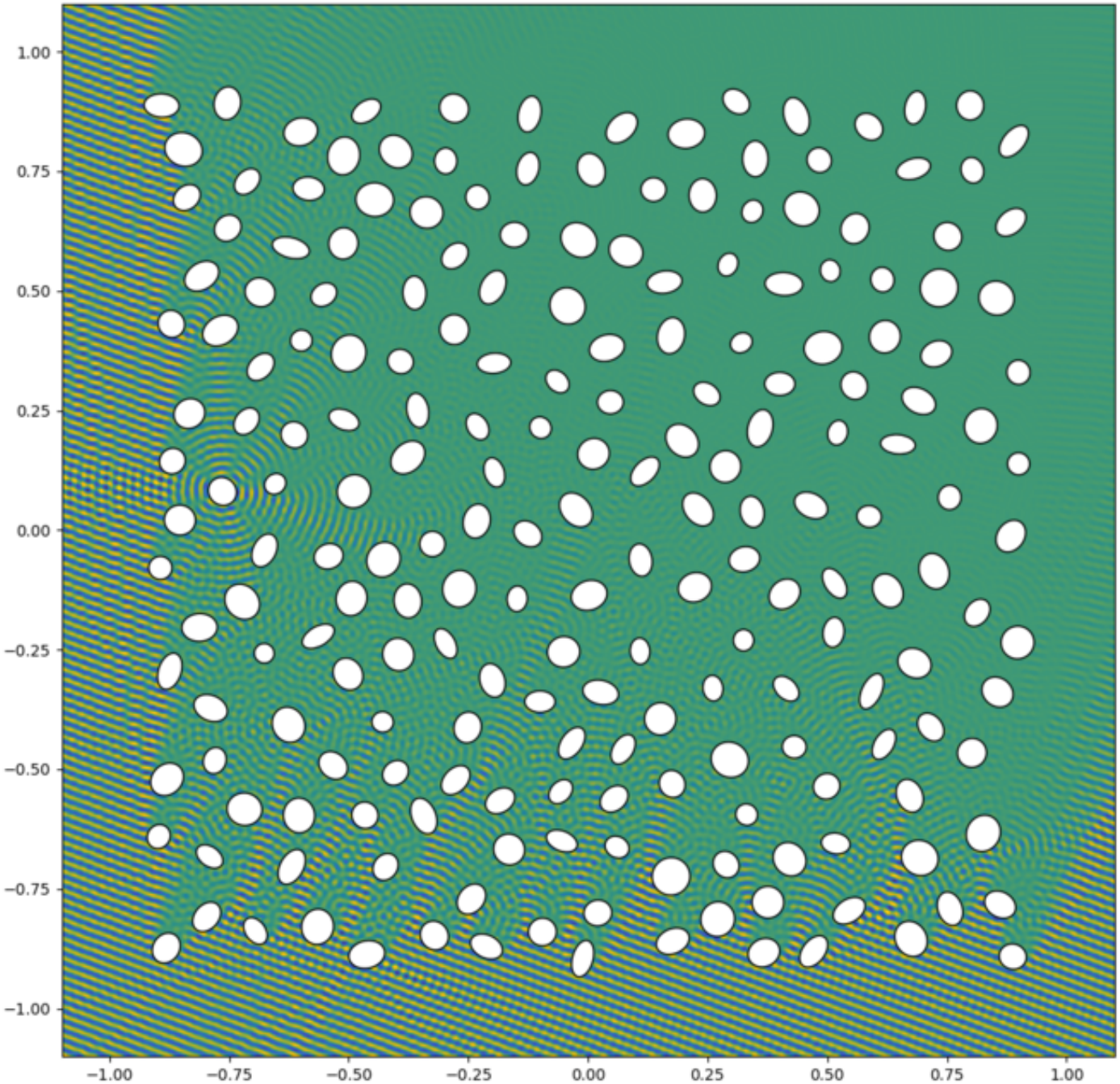
$k = 178$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

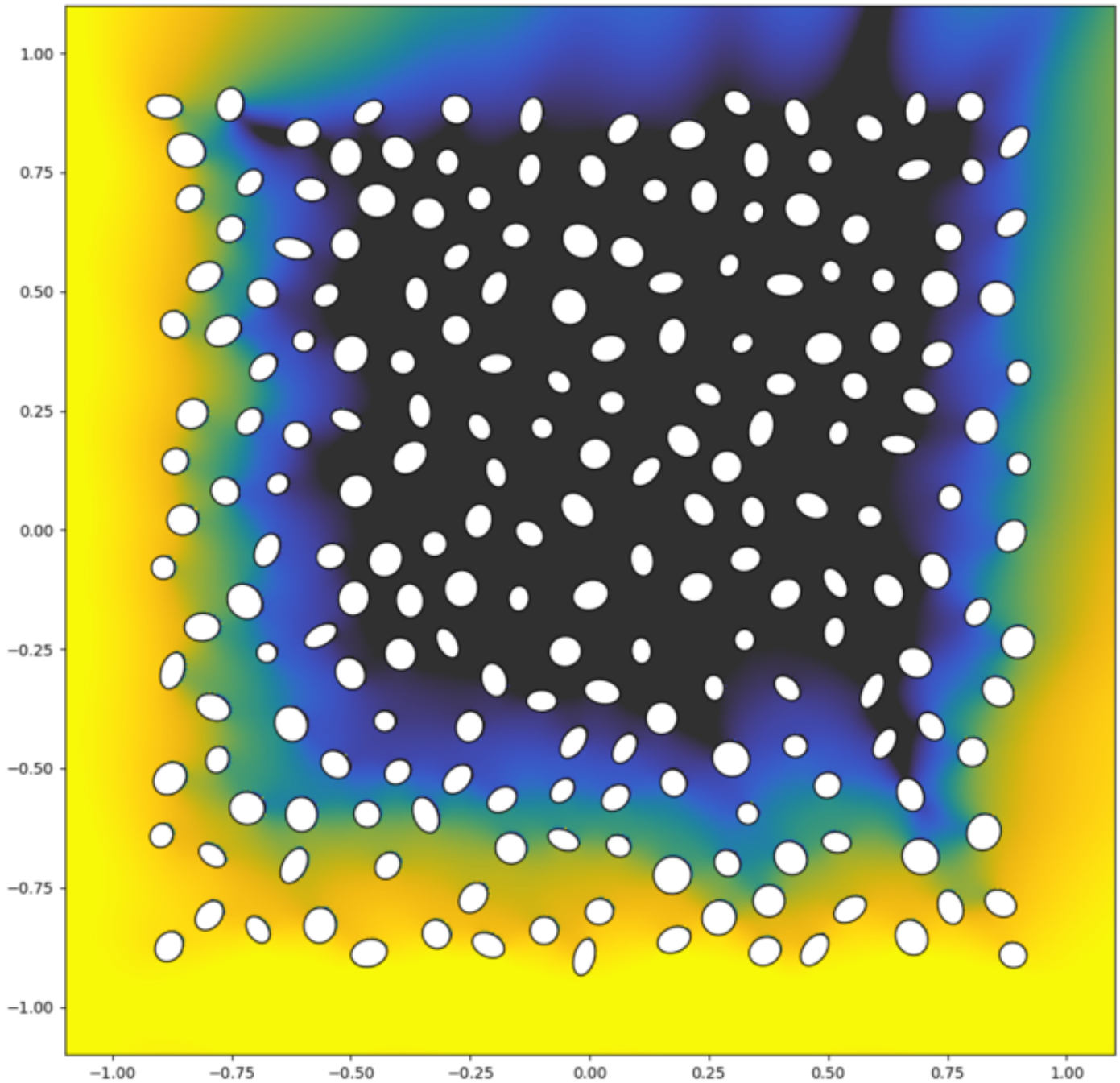
$k = 237$ $(\Re u)$


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

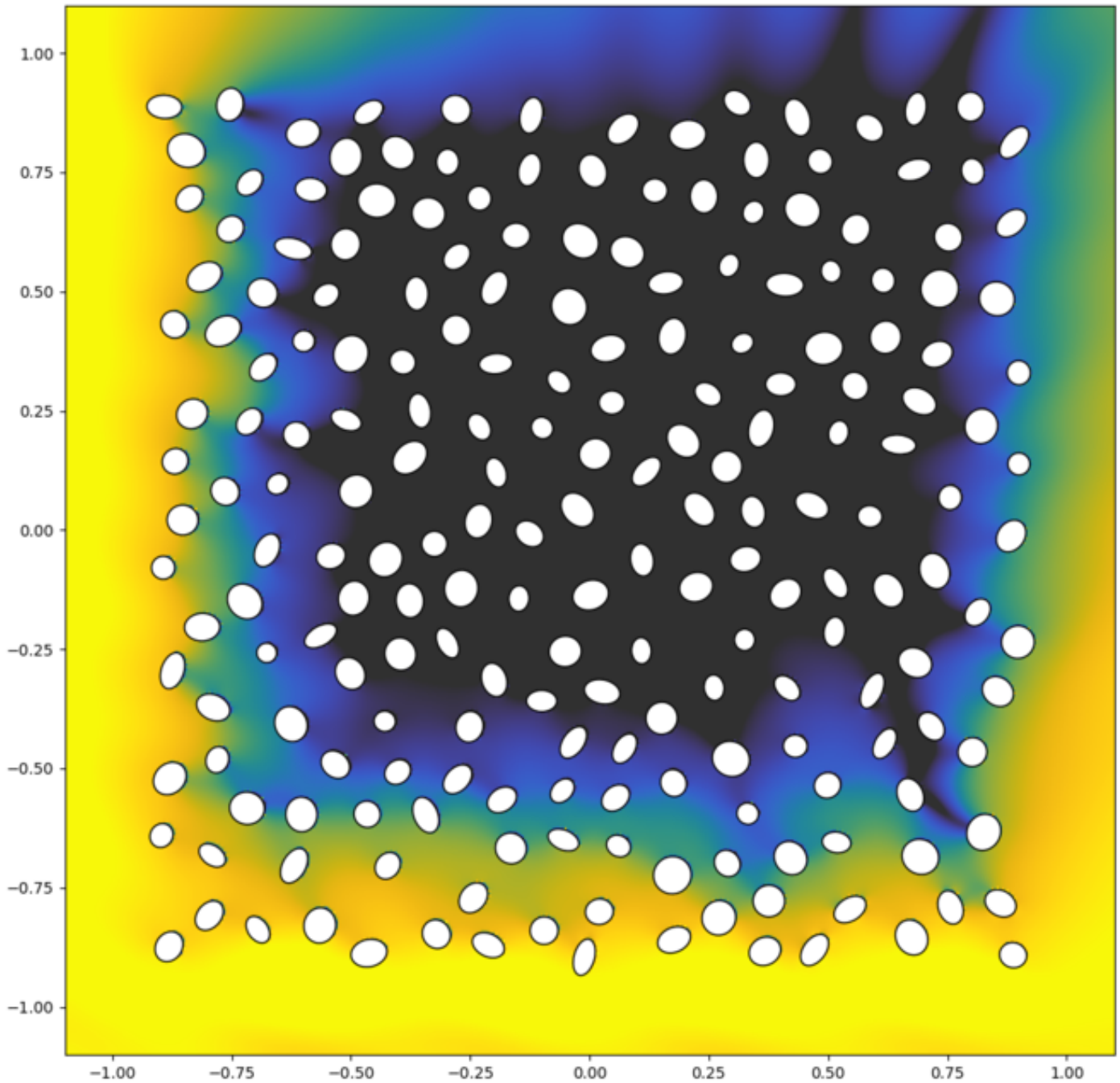
$k = 333$ ($\Re u$)


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

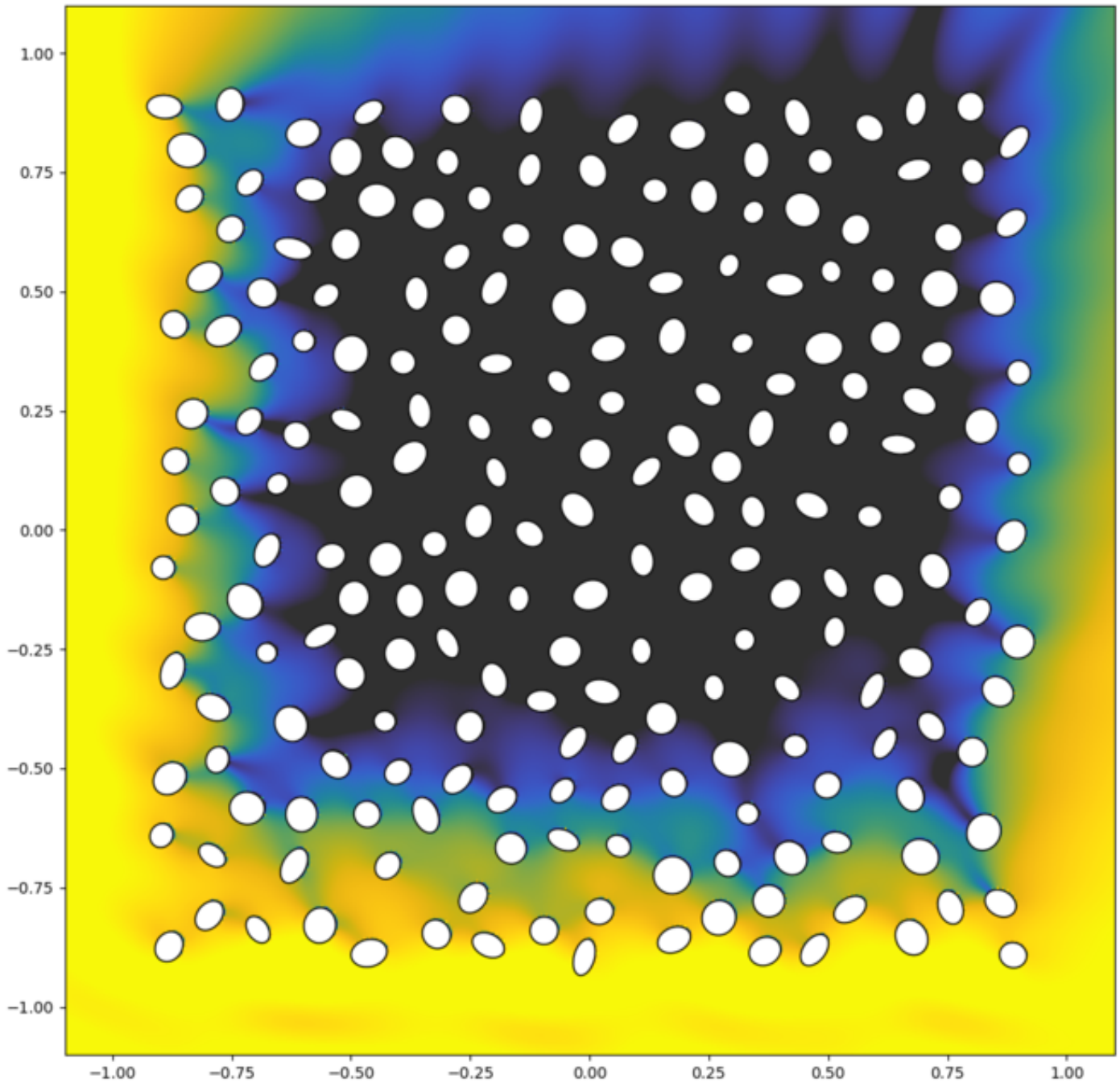
$k = 10$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

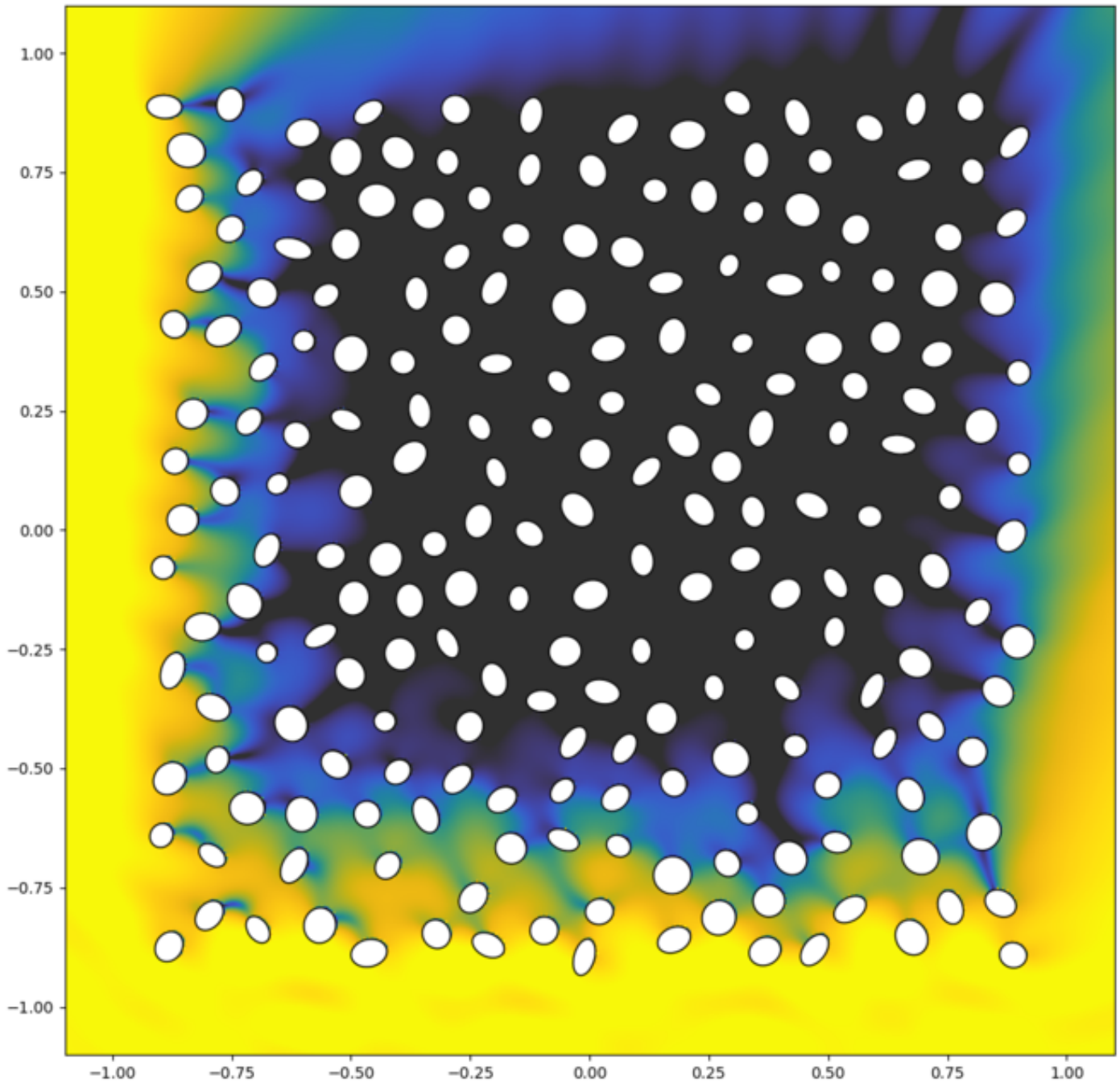
$k = 13$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

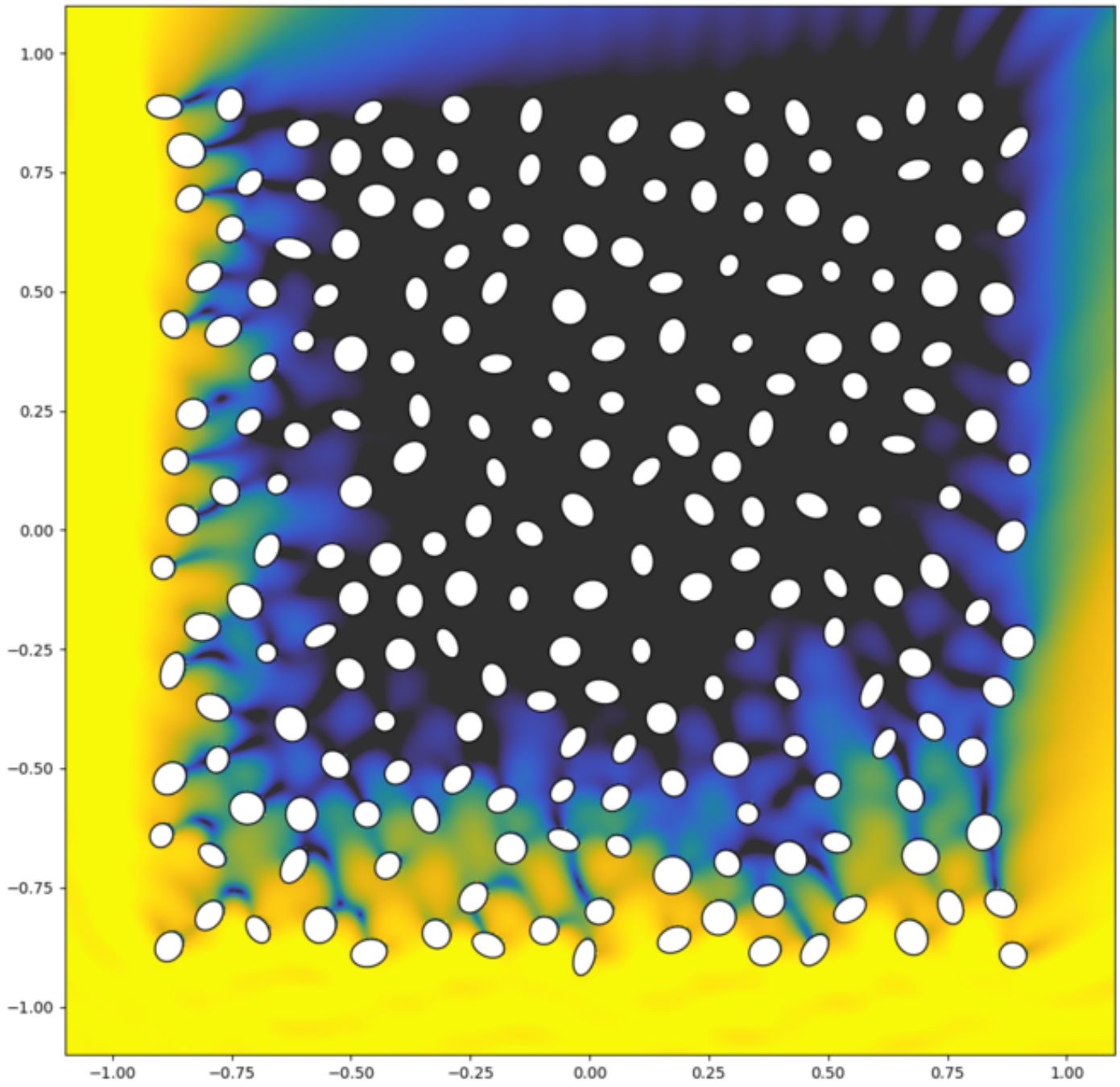
$k = 17$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

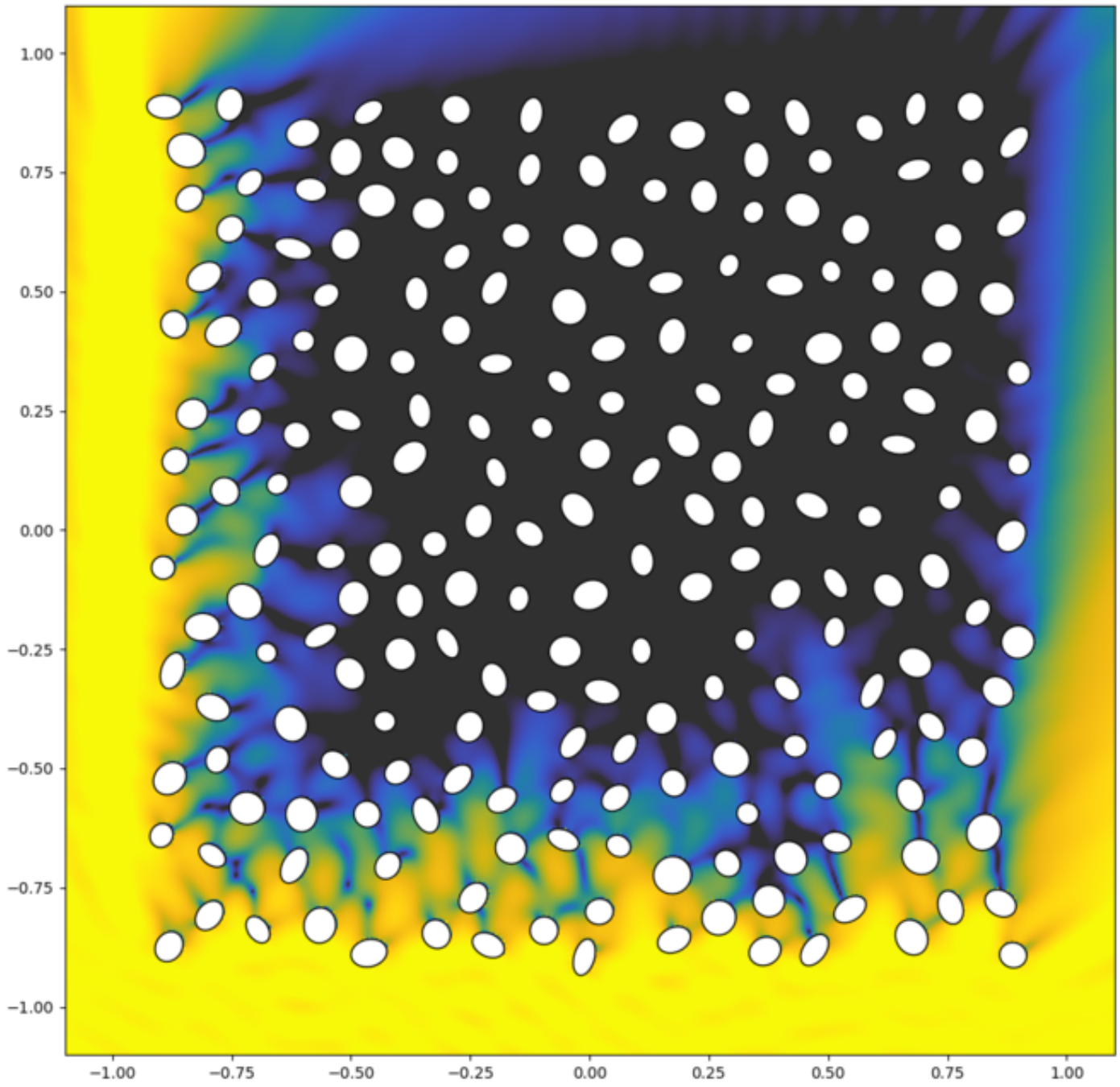
$k = 24$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

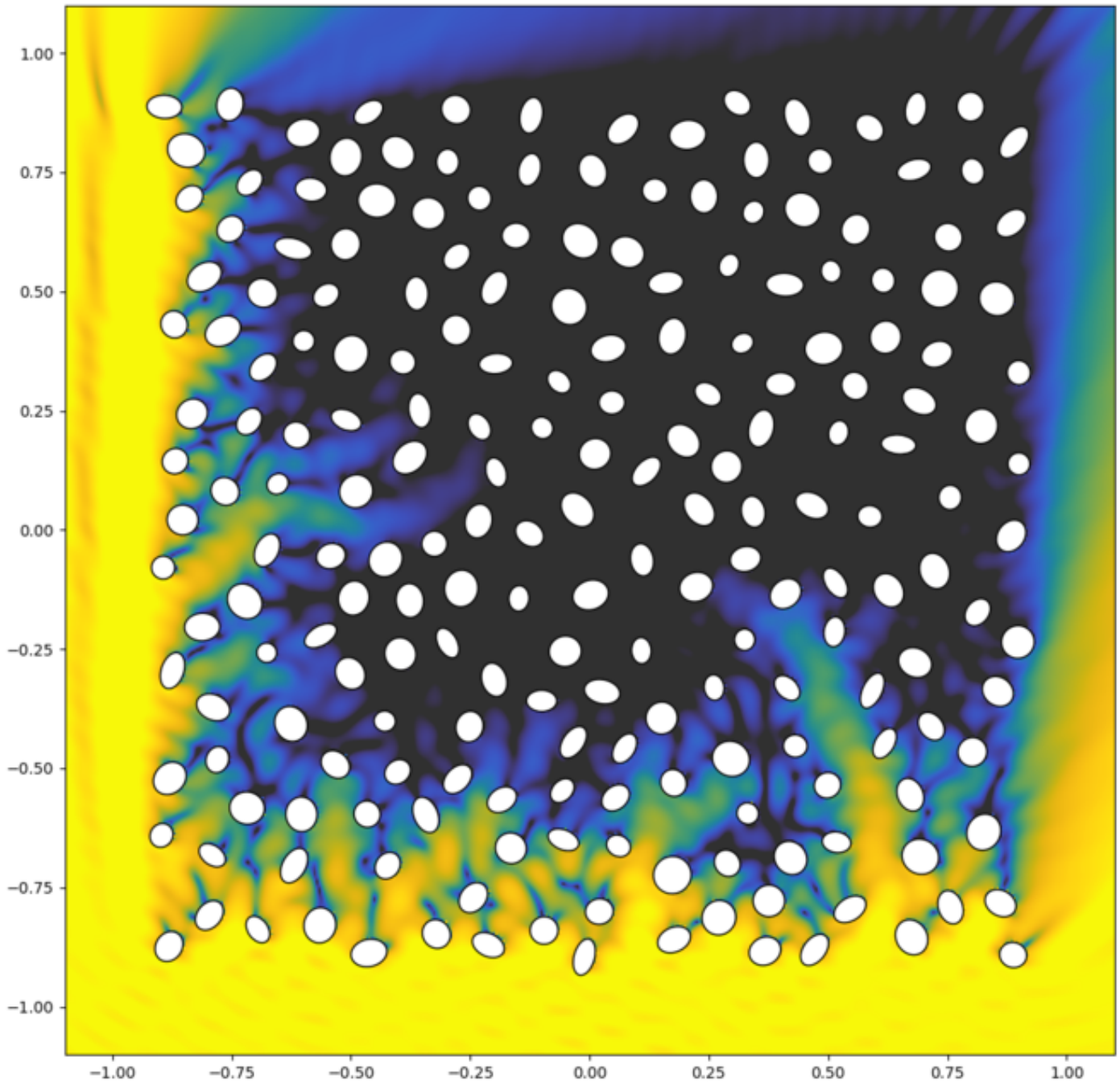
$k = 33$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

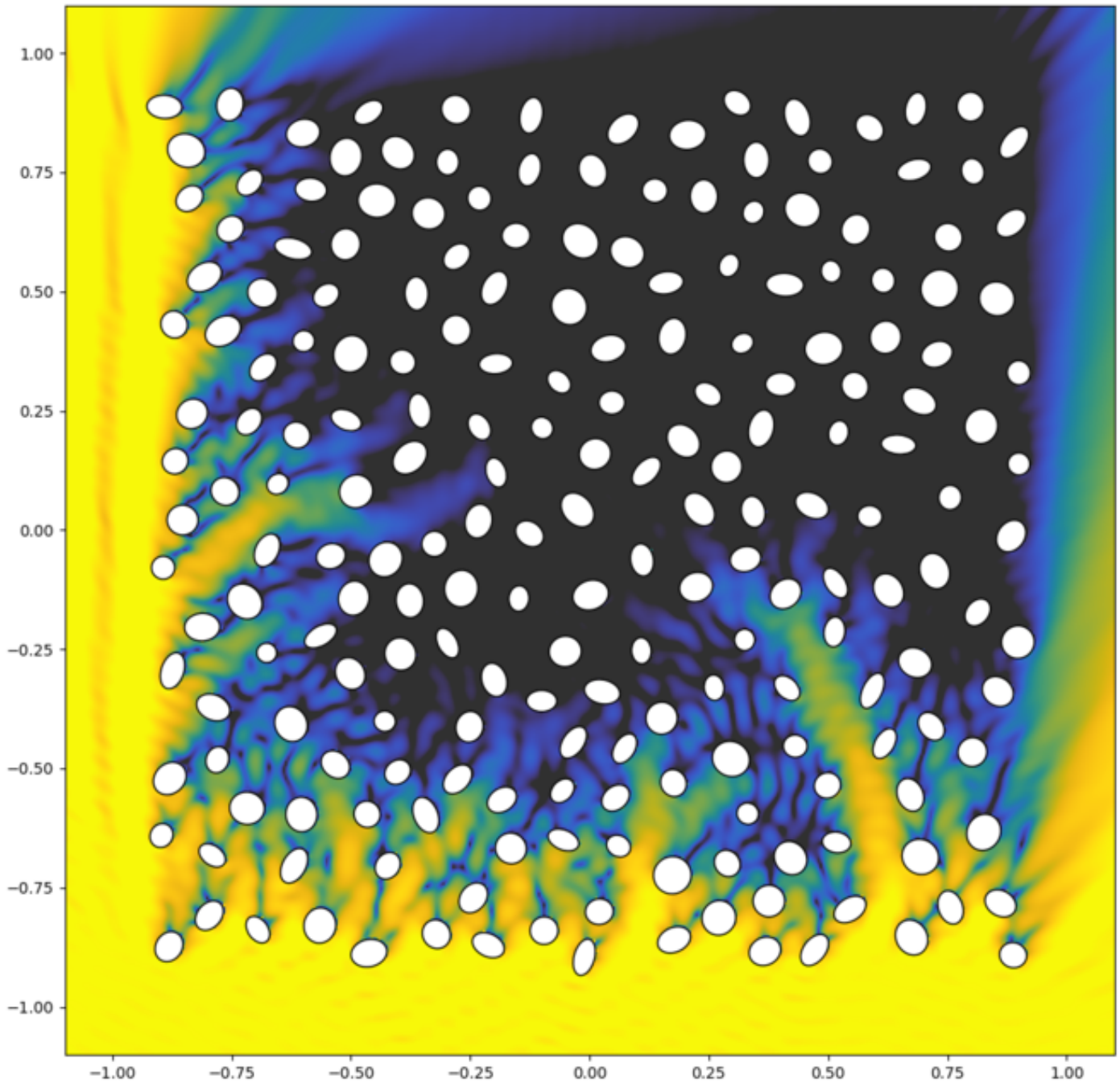
$k = 42$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

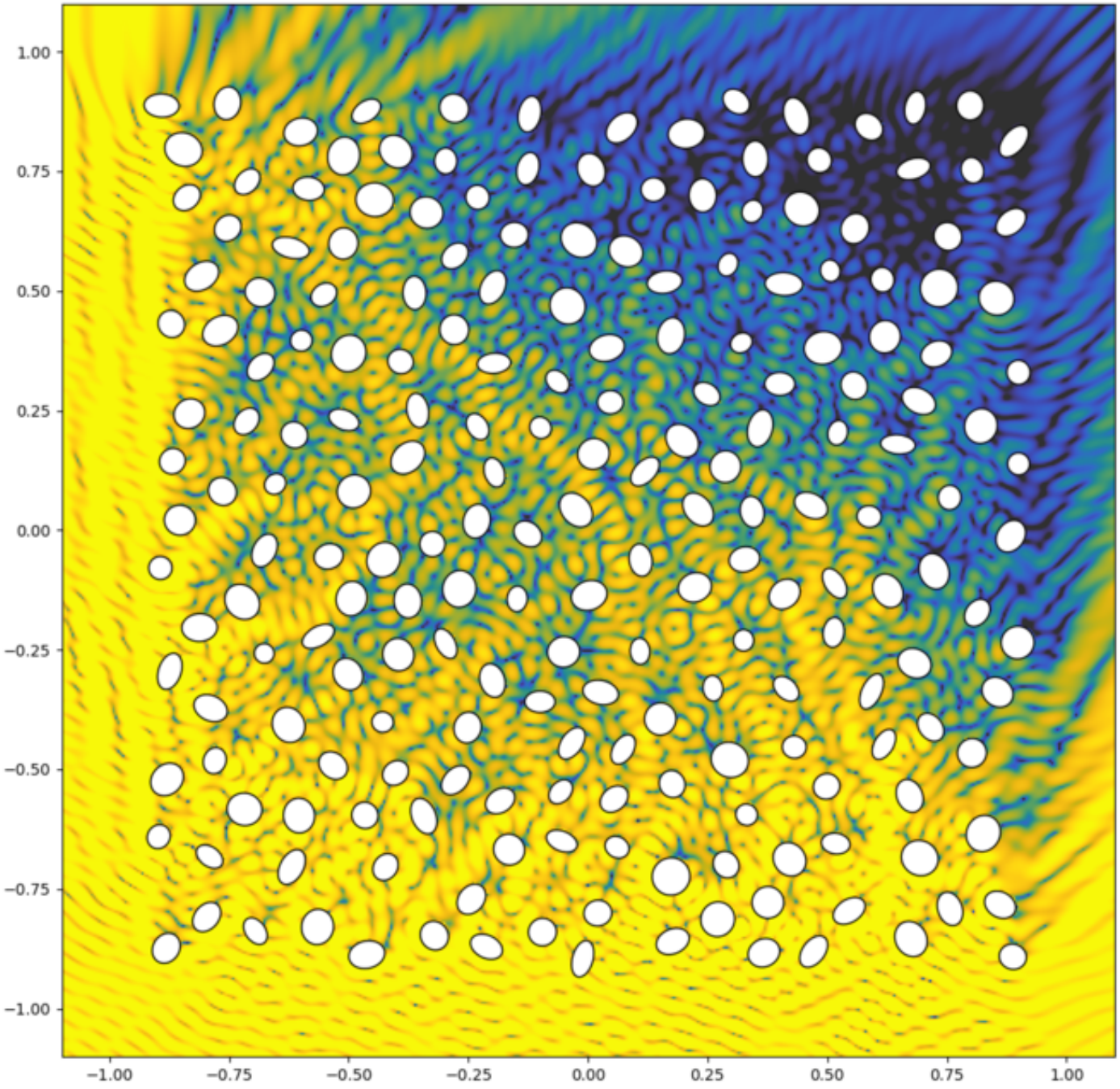
$k = 56$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

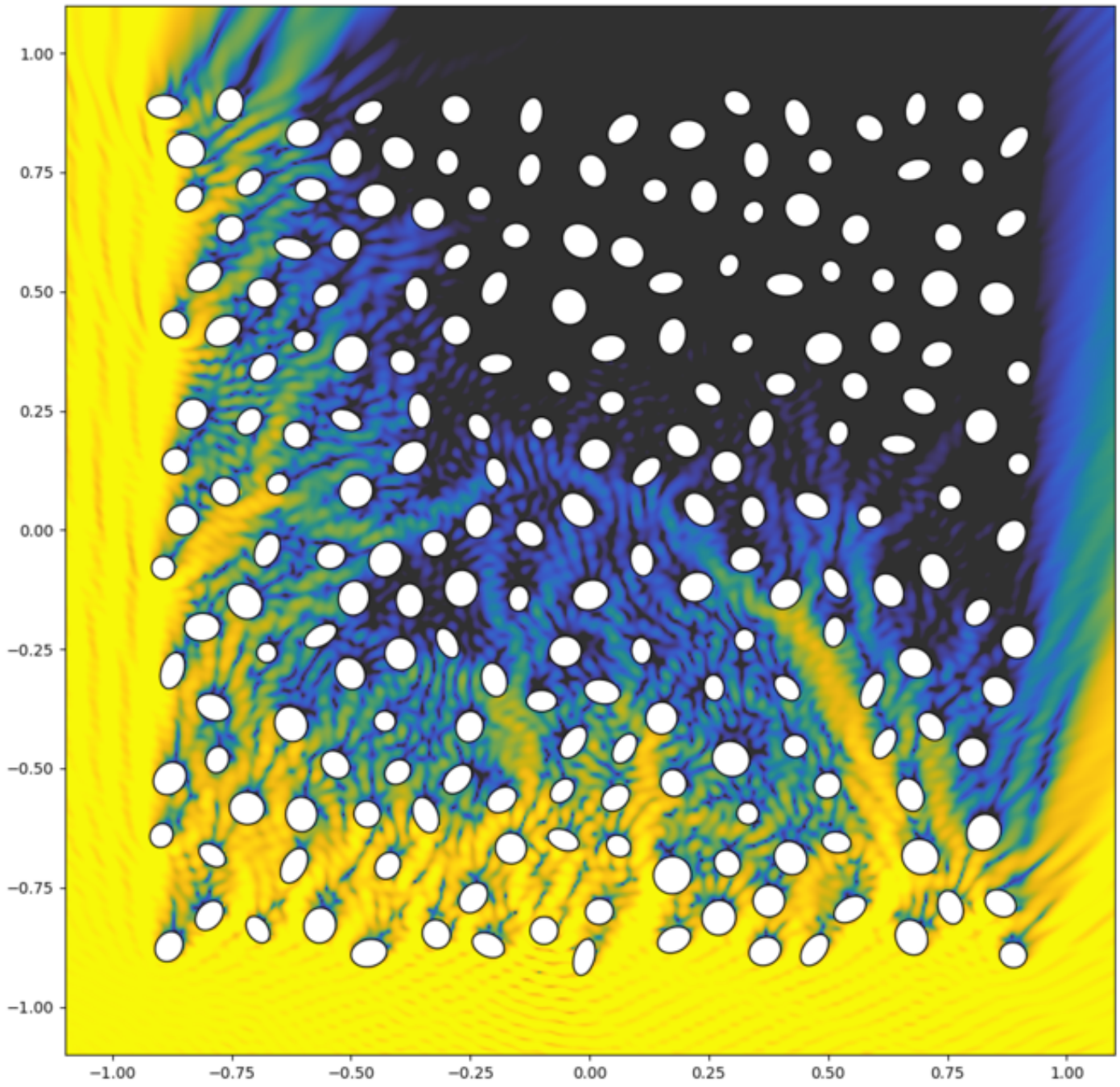
$k = 75$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

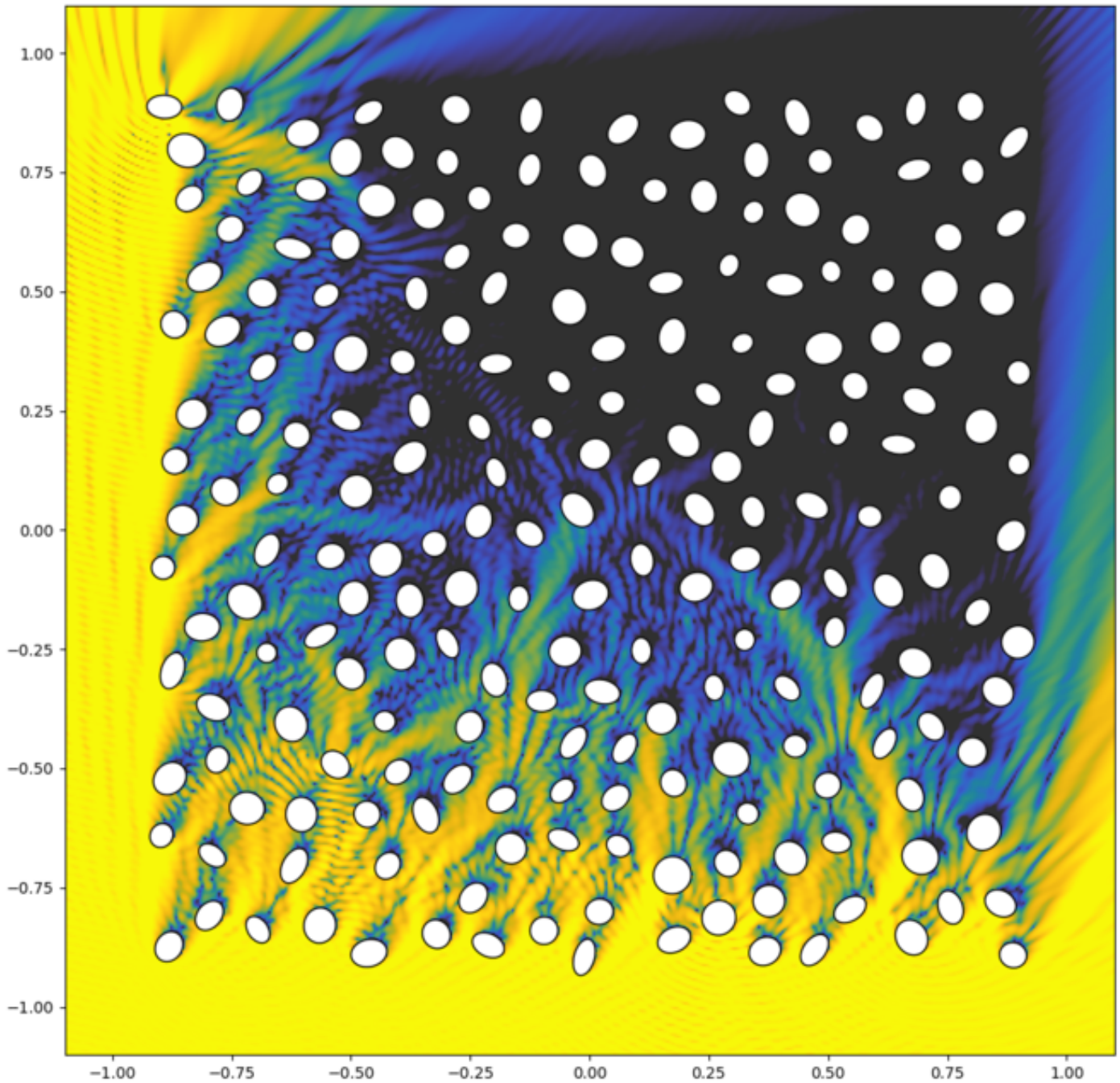
$k = 100$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

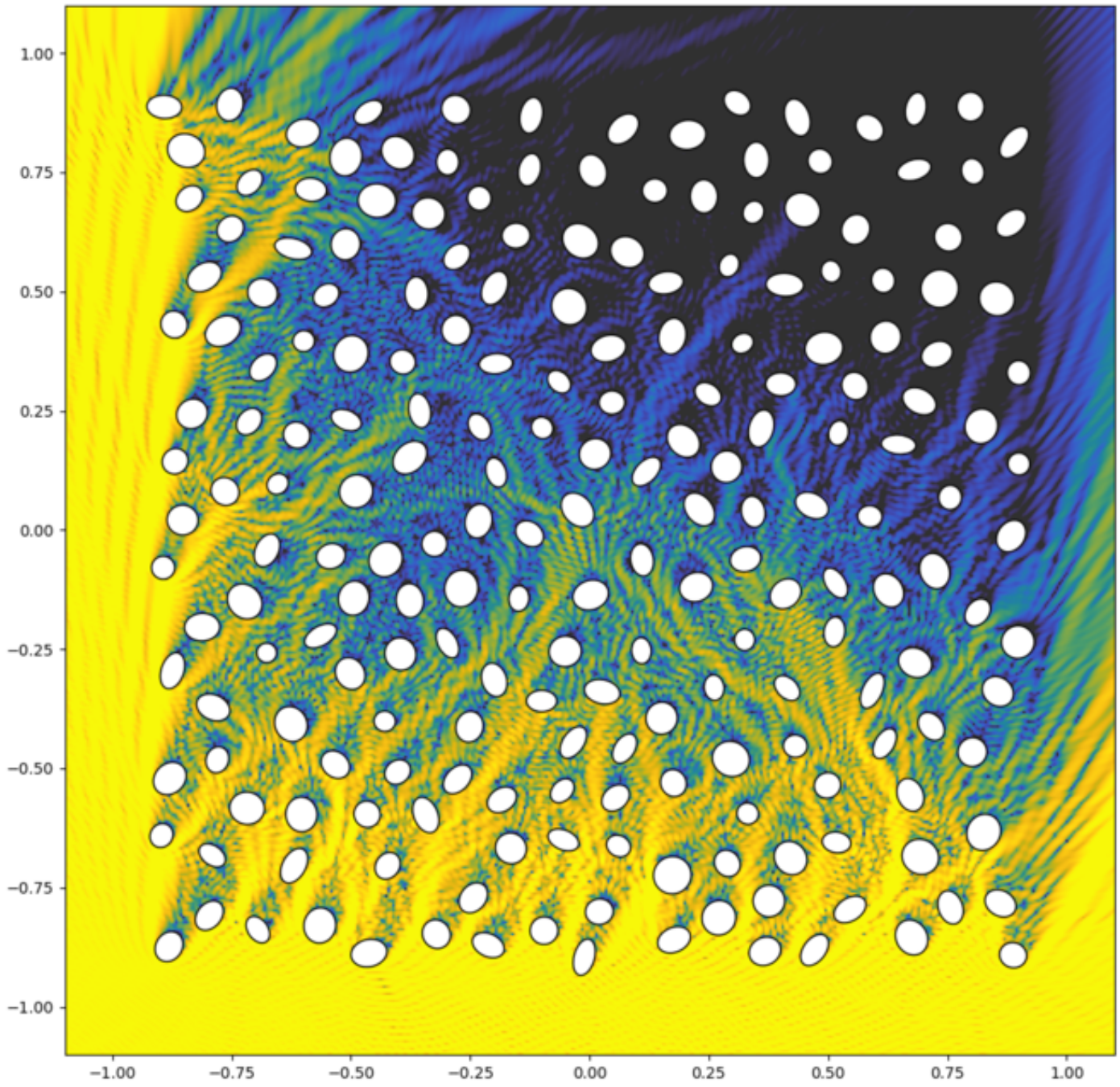
$k = 133$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

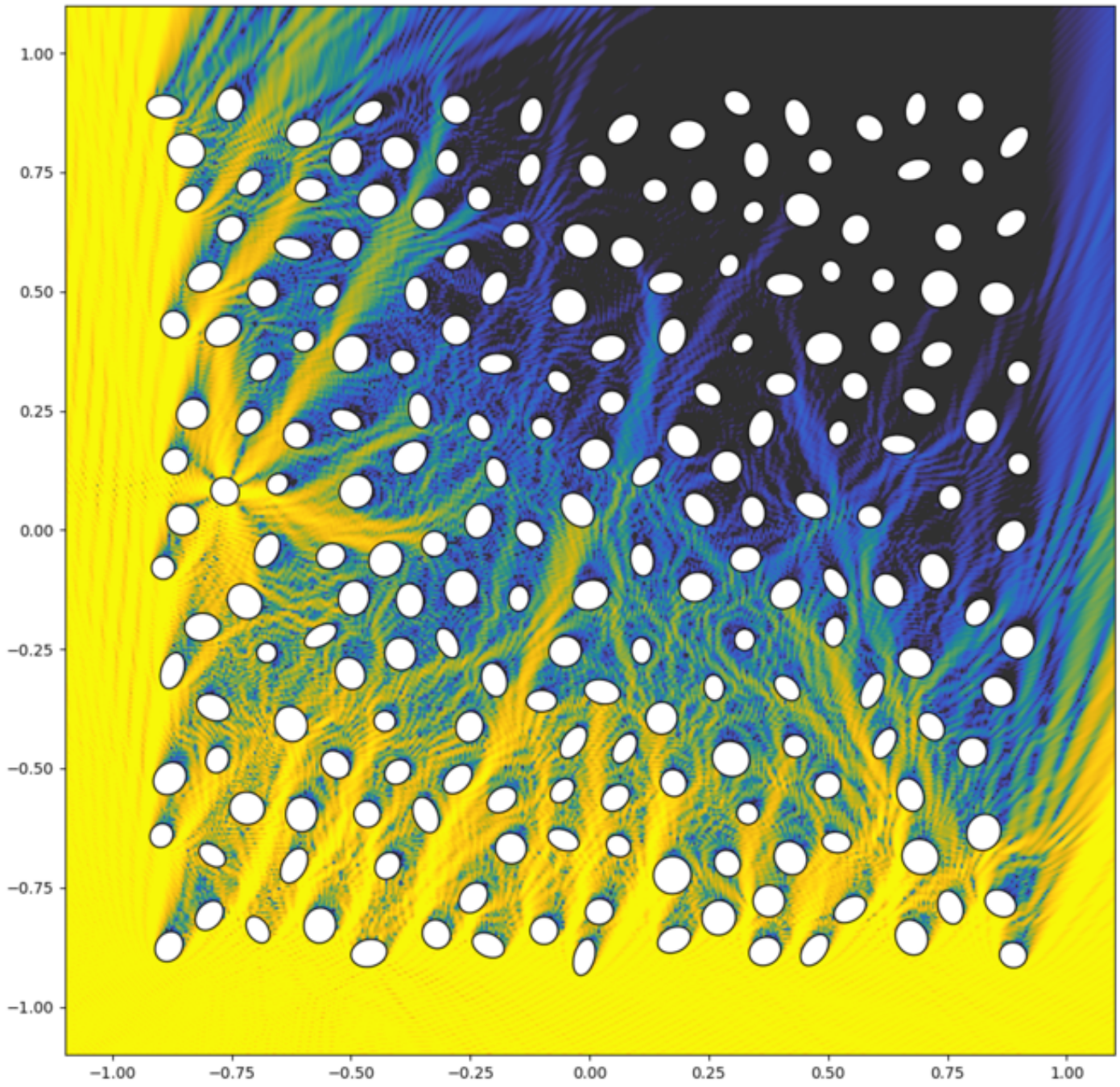
$k = 178$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

$k = 237$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

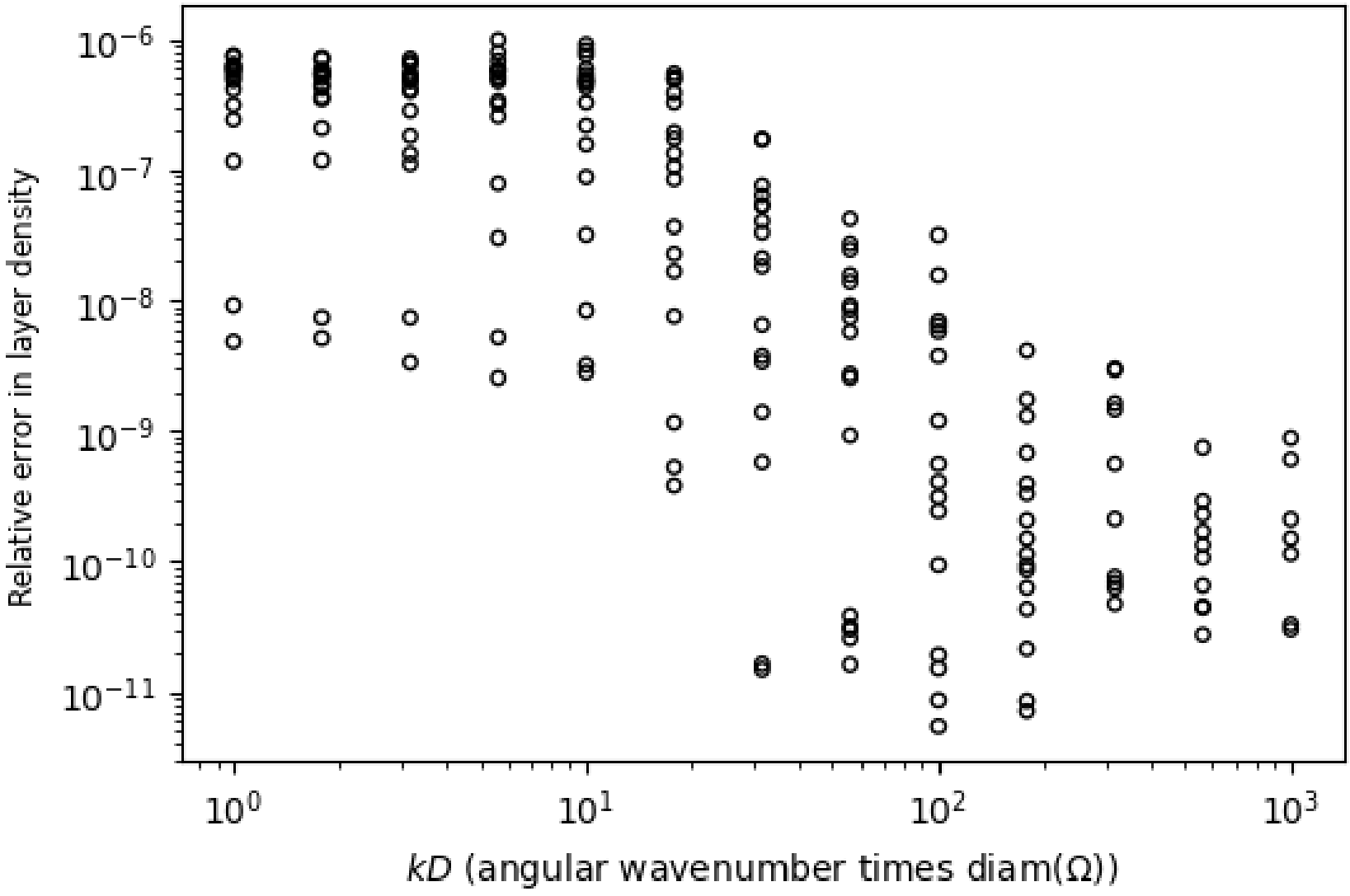
$k = 333$ ($|u|$ (dB))


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00

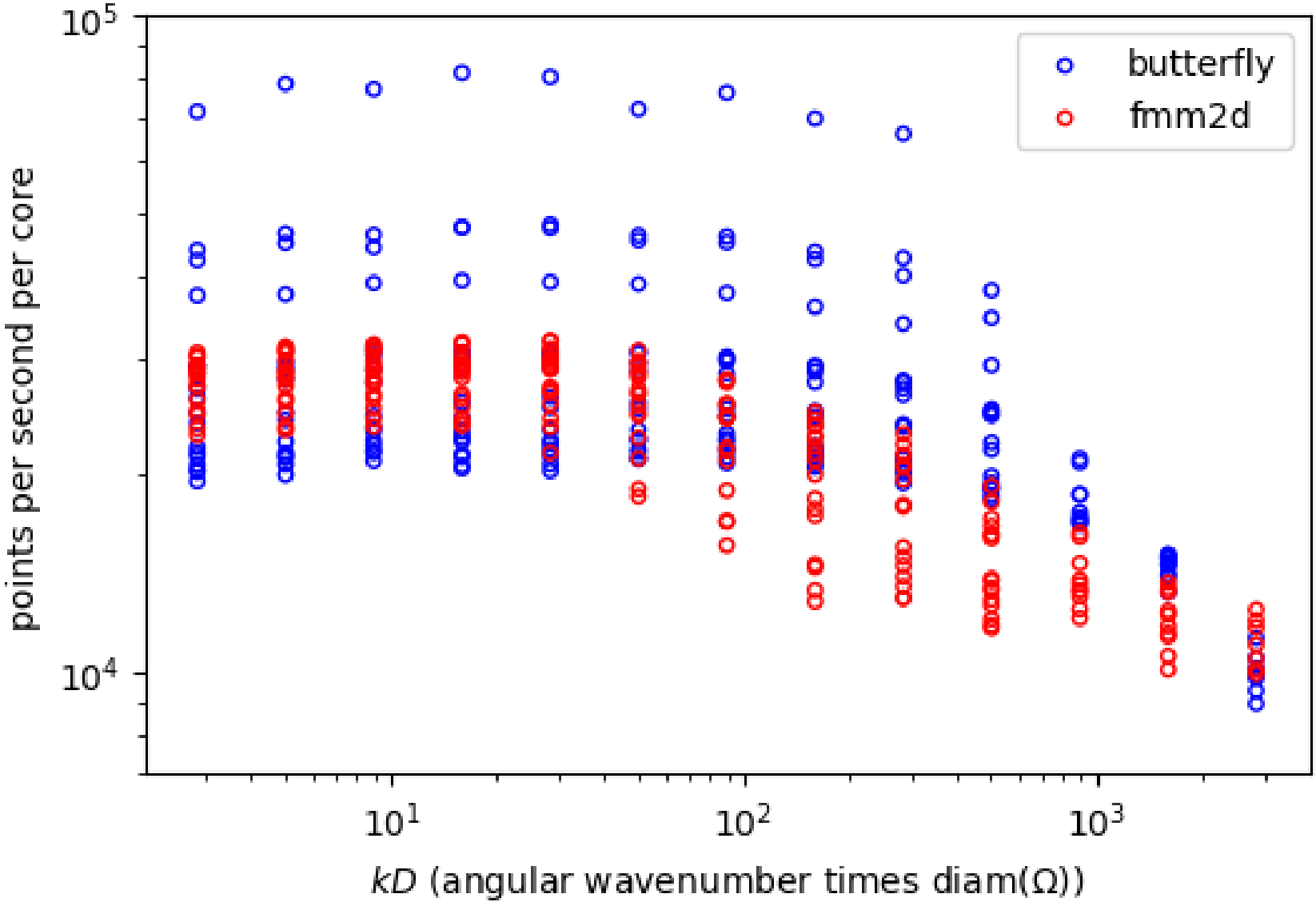
Numerical experiments

- Grid of experiments: k, ellipse radius, discretization fineness (h) and number of points (N)
- Solve scattering problem with ML-BF and HF-FMM (using Flatiron/CCM's `fmm2d`)
- To control for quality of parallelization, everything is 100% single-threaded for these tests (not even multithreaded BLAS, etc.)
- Measure relative accuracy vs dense for σ, time to solve, # iterations, memory use, etc.

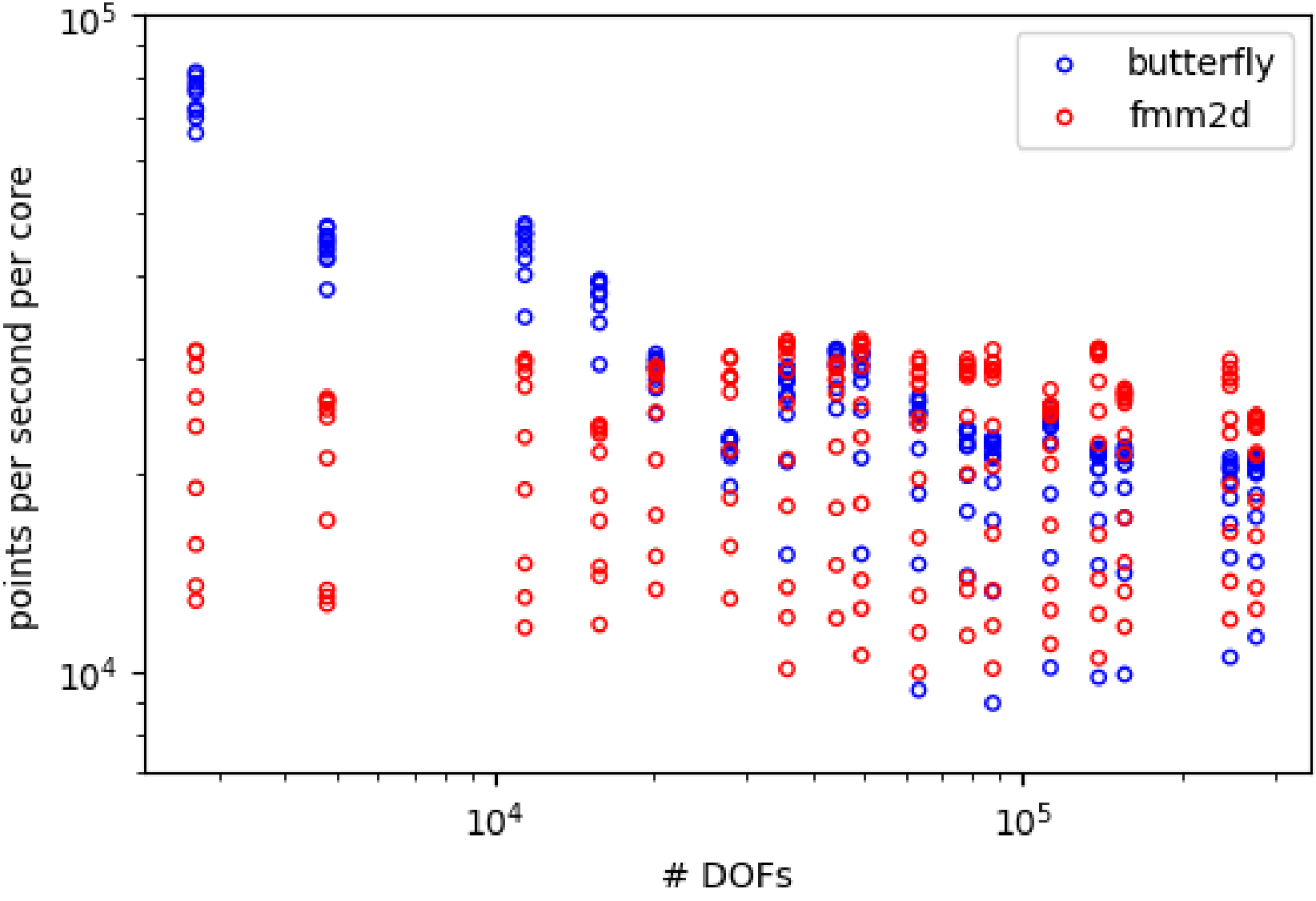
Numerical experiments



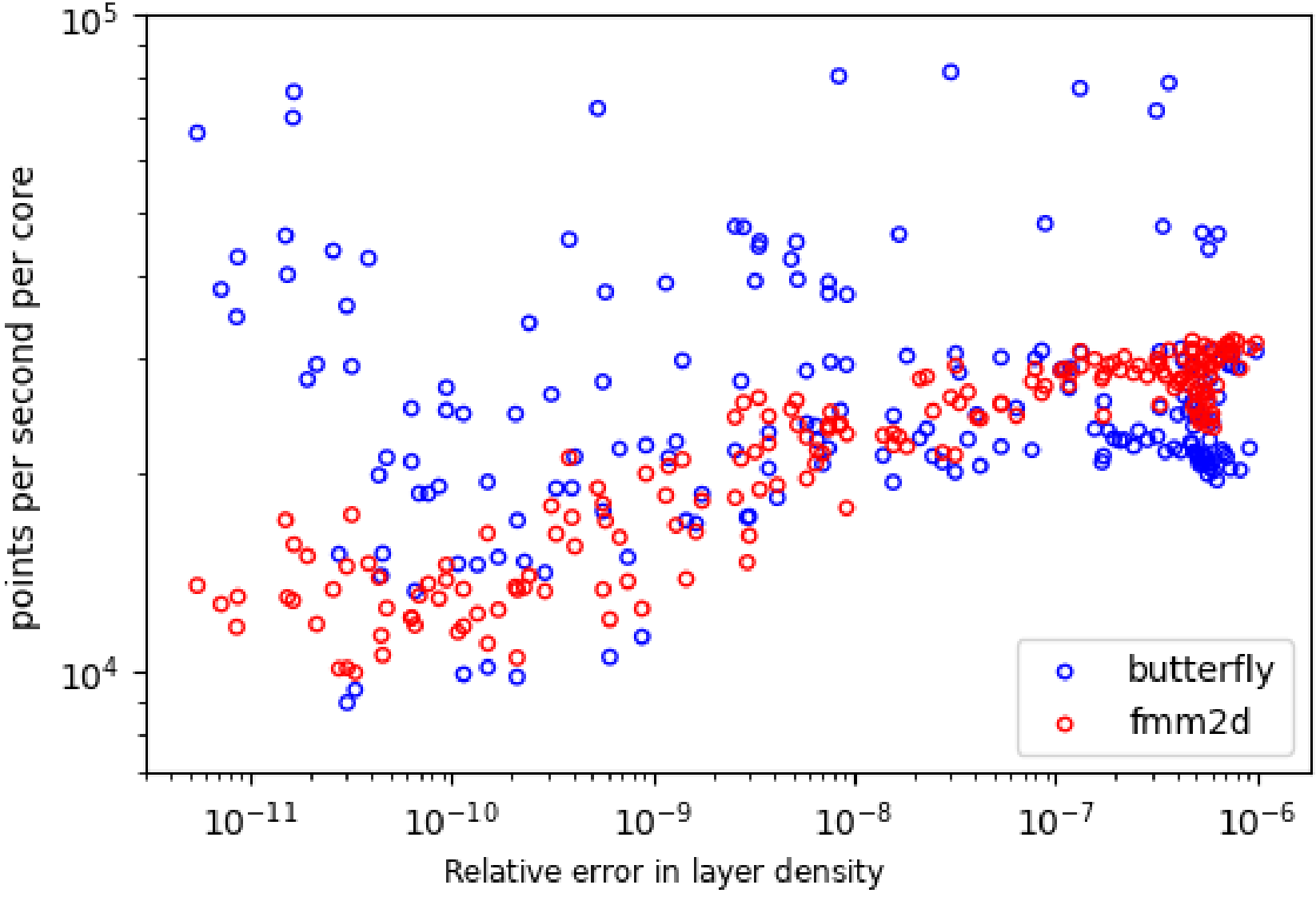
Numerical experiments



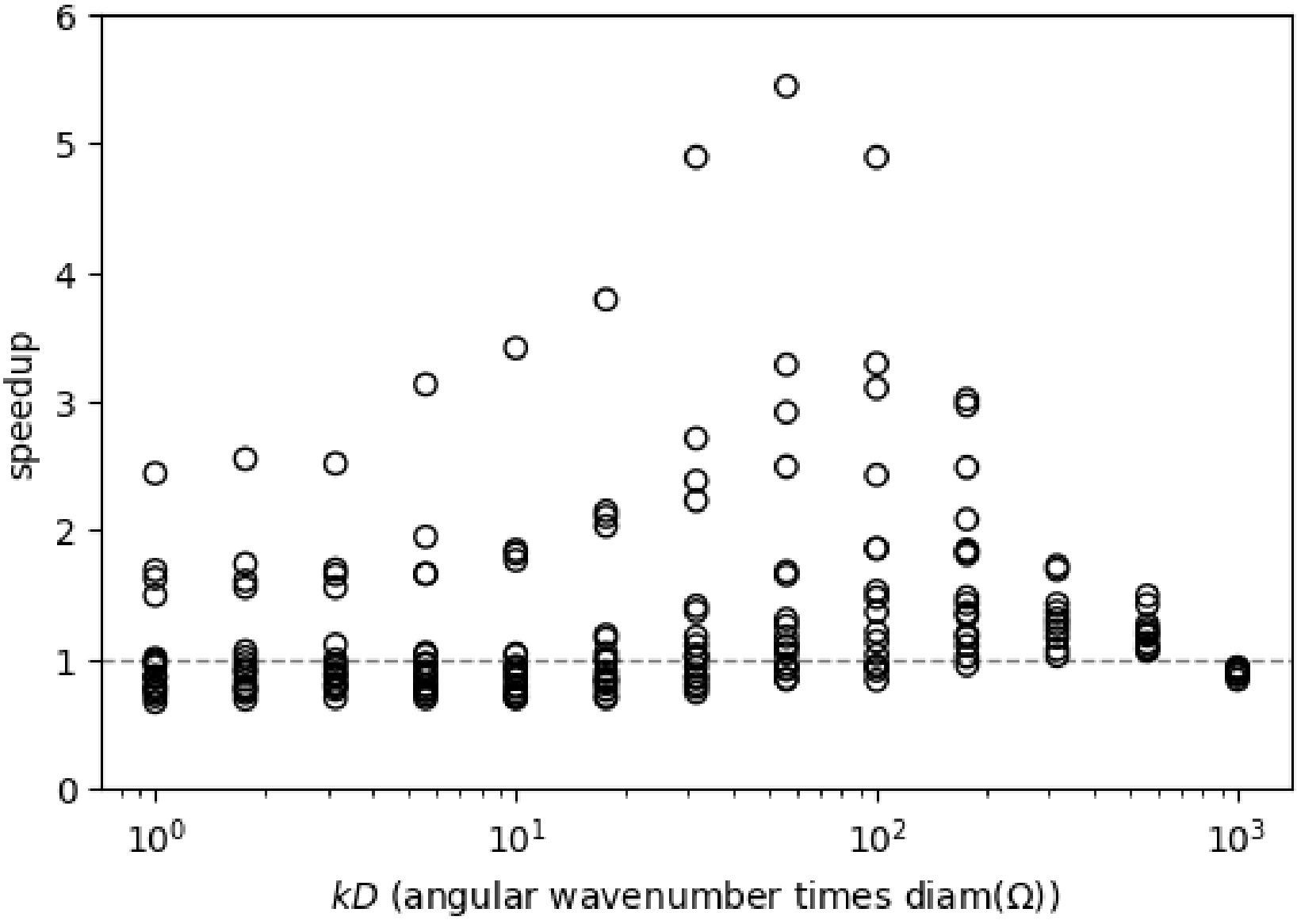
Numerical experiments



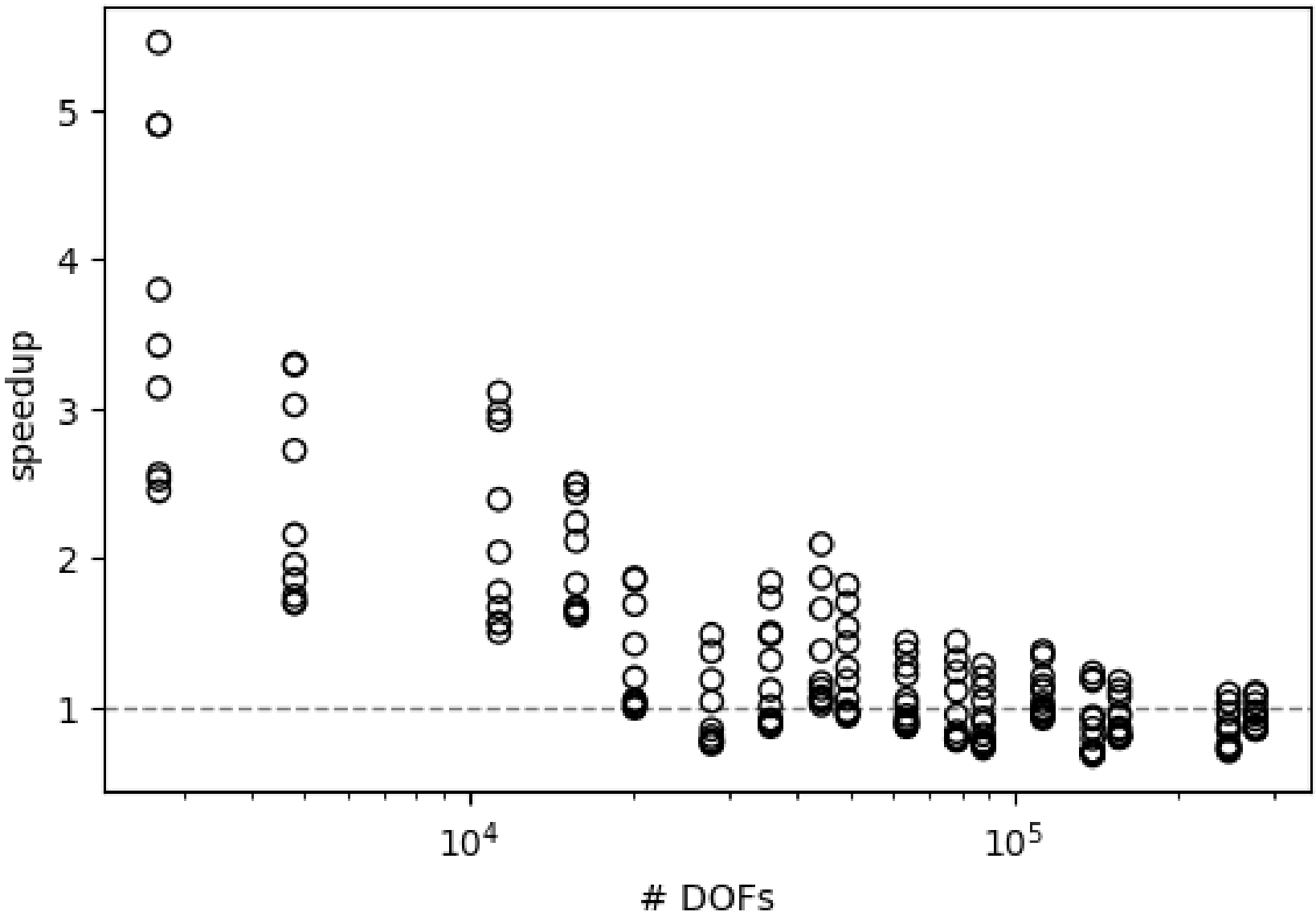
Numerical experiments



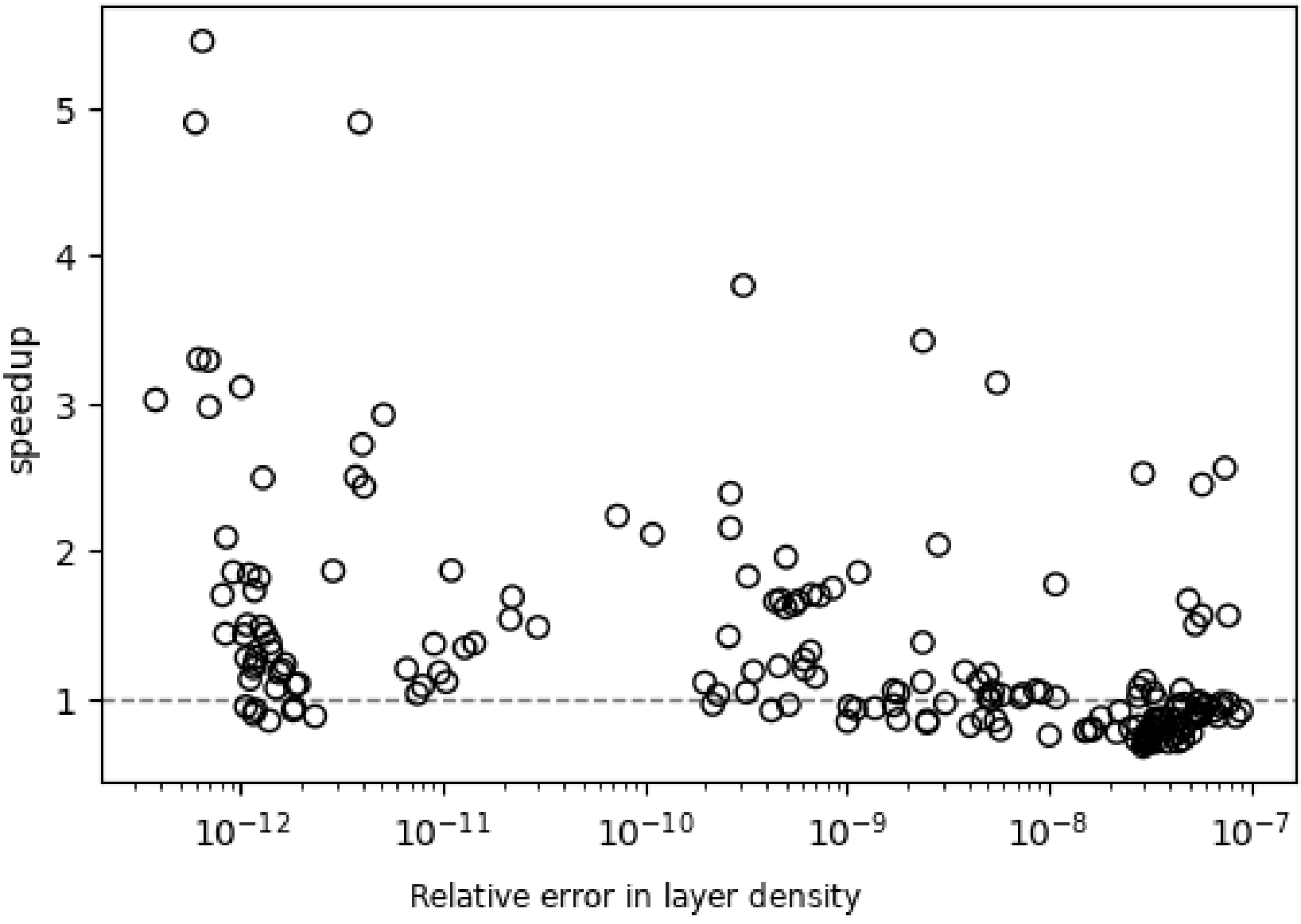
Numerical experiments



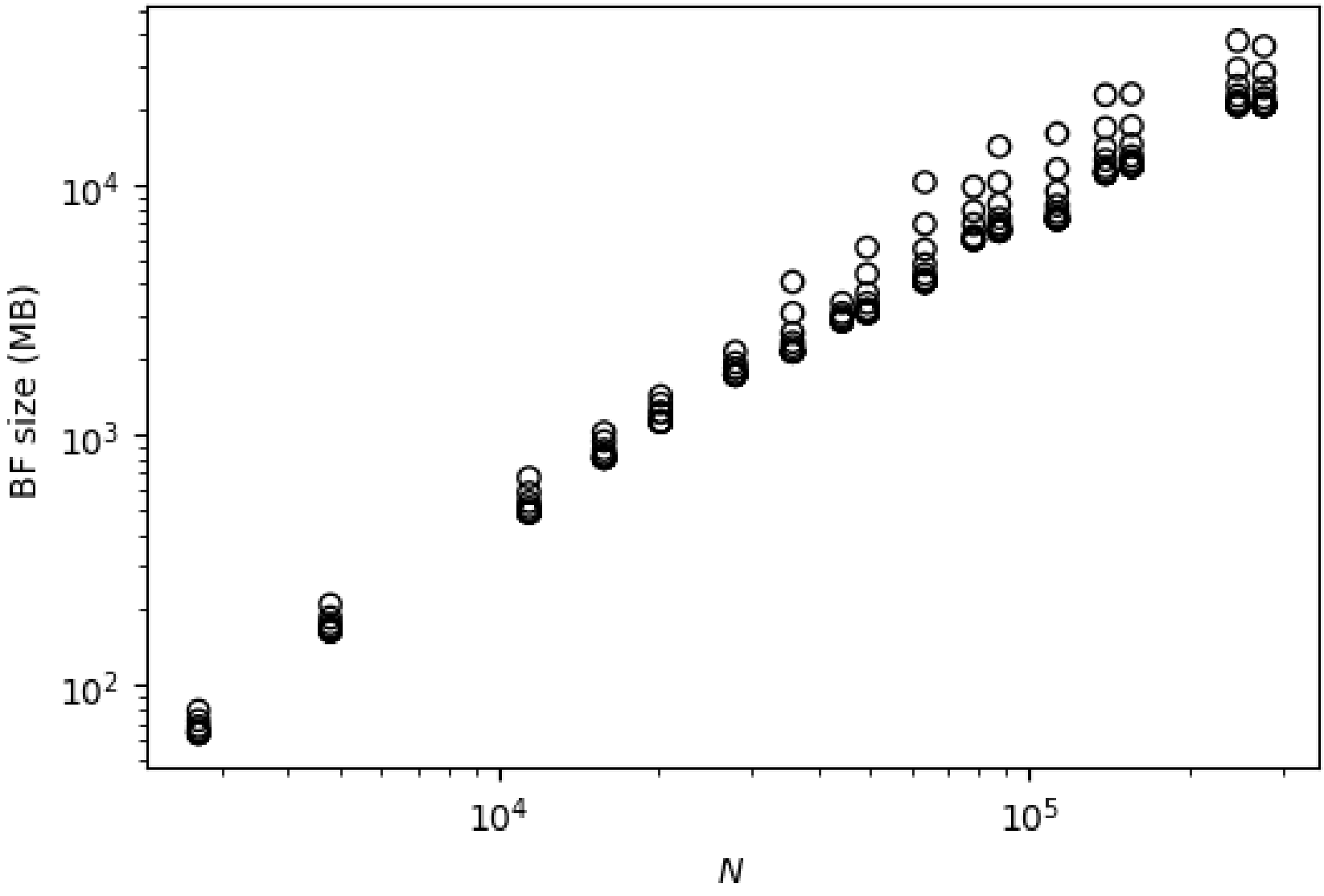
Numerical experiments



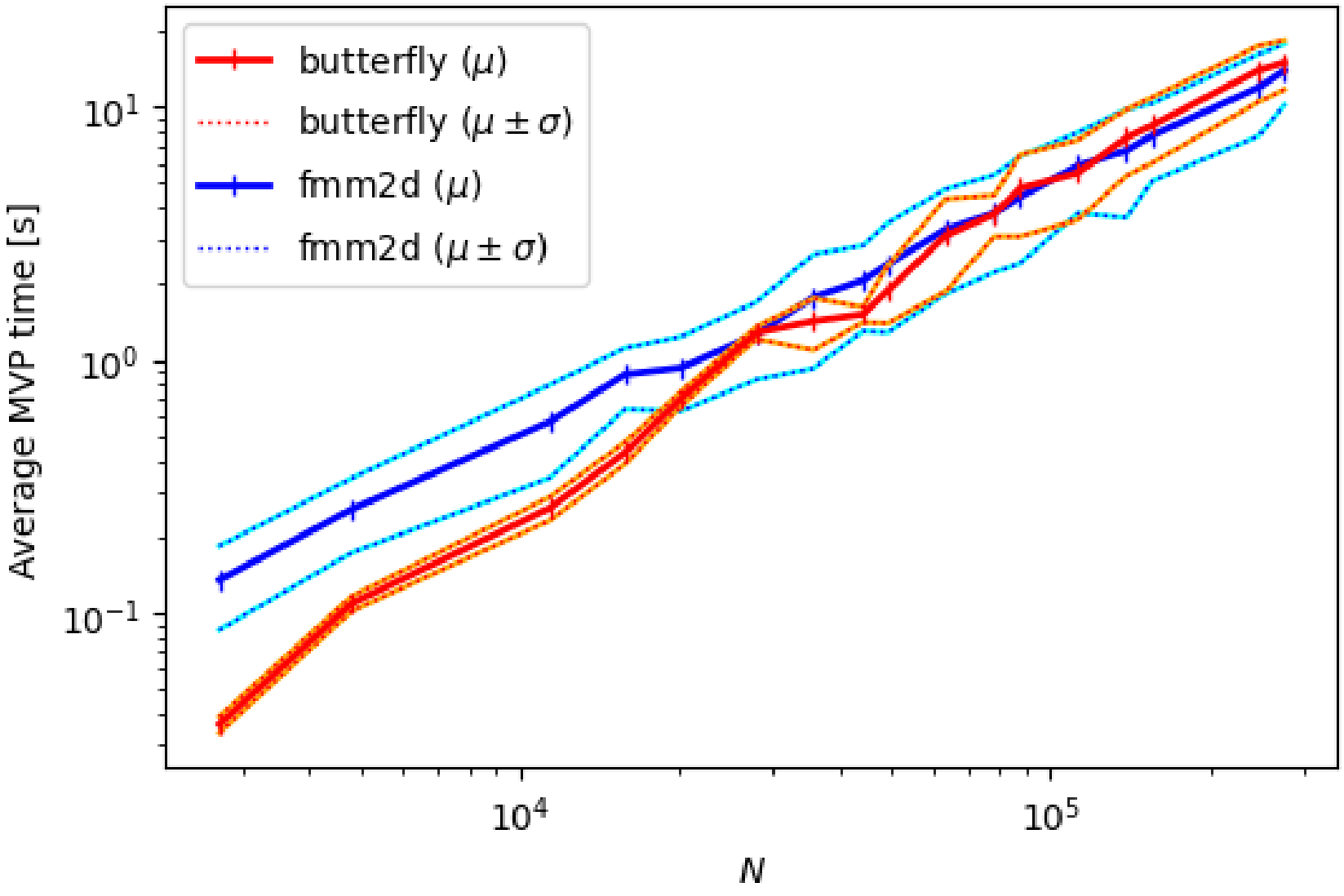
Numerical experiments



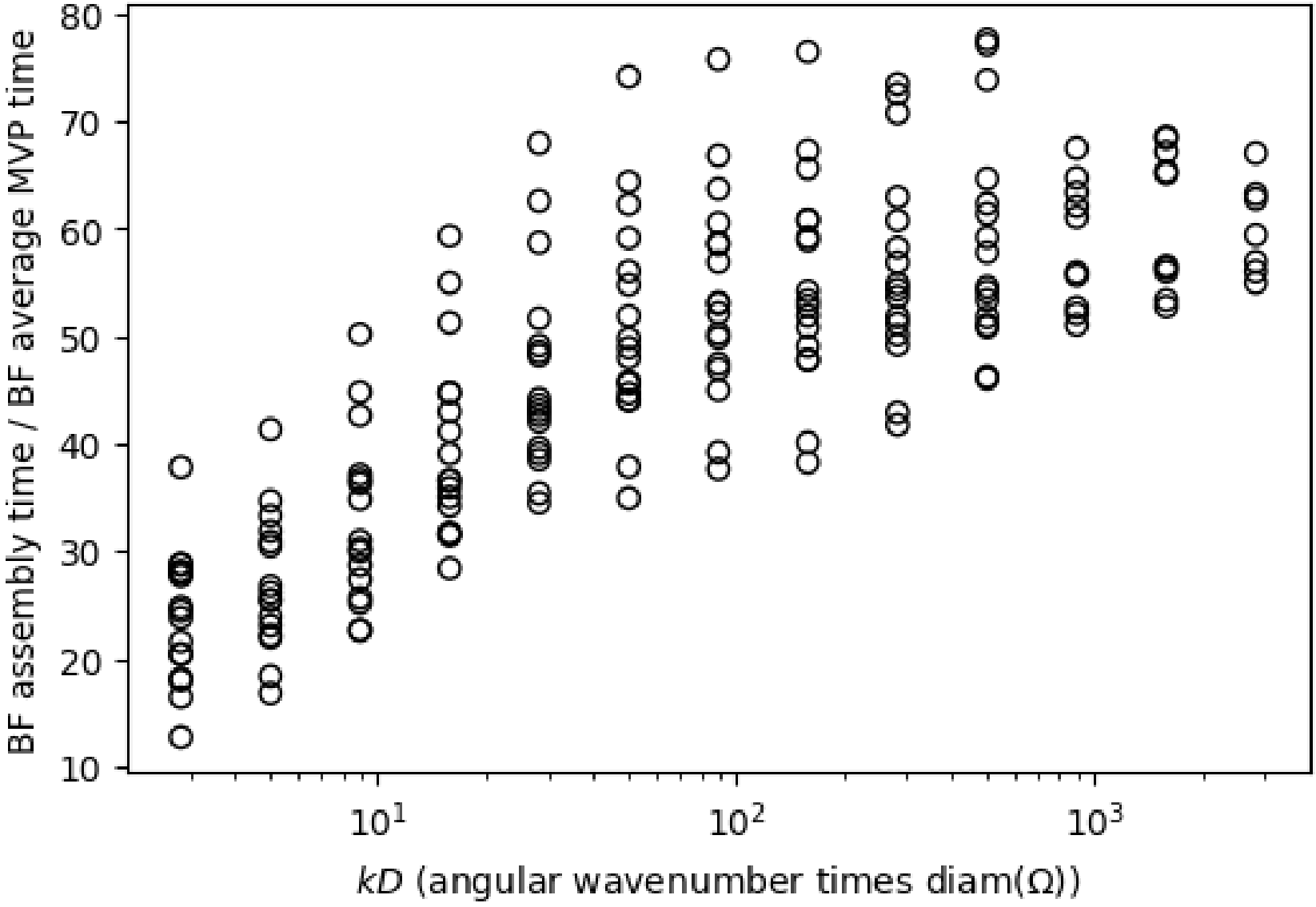
Numerical experiments



Numerical experiments



Numerical experiments



Conclusion

Summary:

- The multilevel butterfly is a surprisingly competitive fast algorithm for 2D Helmholtz. (Time complexity is the same, space complexity is worse.)
- Contrary to previous misconceptions, it is a wideband algorithm.
- Good performance indicates a fast direct solver based on the butterfly will be competitive across a wide range of frequencies.

Conclusion

What's next?

- Fast direct solver (hierarchical LU)
- Reduce space complexity
- Solve 3D scattering problems: should compare favorably w/ 3D HF-FMM
- Task parallelism
- Batch GPU implementation?

C library w/ Python wrapper:

`https://github.com/sampotter/butterfly`

P, Greengard, O'Neil, "Butterfly-accelerated multiple scattering in two dimensions", in preparation.